

Machine Learning for Material Science

Prof. Bai-Xiang Xu, Prof. Karsten Albe

Dr. Vasileios Karanikolas, Dr. Dilshod Durdiev

TECHNISCHE UNIVERSITÄT DARMSTADT

RWTH AACHEN UNIVERSITY

MAT COM COM MAT GRK 2561

FLAIR

DFG Deutsche Forschungsgemeinschaft

NHR4CES NHR for Computational Engineering Science

19-21.10.2026, online

TECHNISCHE UNIVERSITÄT DARMSTADT

Part I: Motivation, Recap ML, Neural Network
Part II: Convolutional Neural Network
Part III: Microstructure representation and Bayesian Optimization

Bai-Xiang Xu, Dilshod Durdiev

NHR4CES Training Event
SDL Materials Design
19.10.2026, online

MECHANICS of FUNCTIONAL MATERIALS

Motivation

Reference:

[1] Goodfellow, Bengio, Courville. Deep Learning. MIT Press. 2016

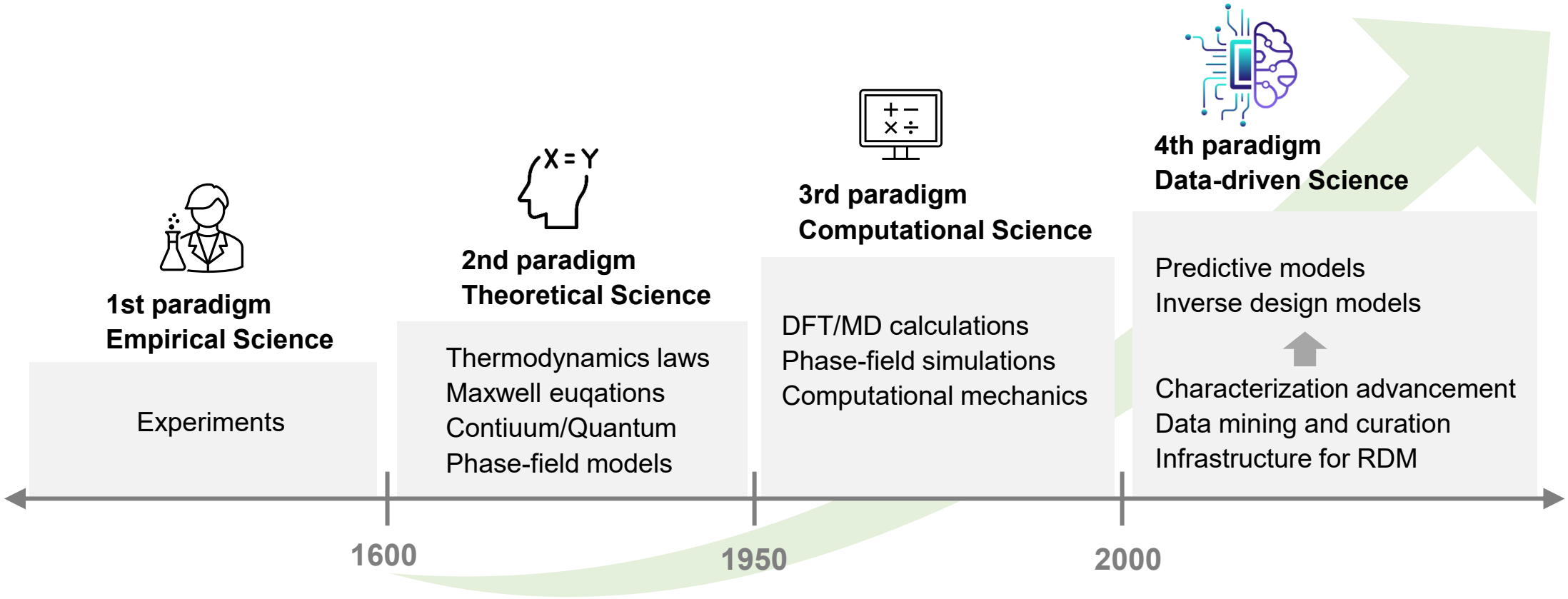
[2] Raschka, Mirjalili. Python Machine Learning. Packt.

[3] Murphy. Machine Learning: a Probabilistic Perspective. MIT Presse. 2012

[4] Bishop. Pattern Recognition and Machine Learning. Springer. 2006

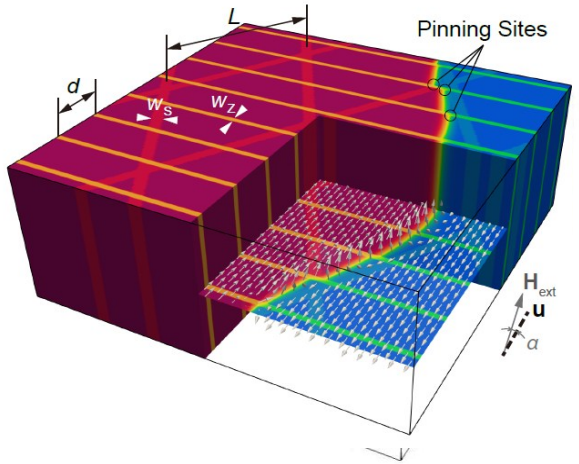
[5] Rasmussen, Williams, Gaussian Processes for Machine Learning, the MIT Press, 2006

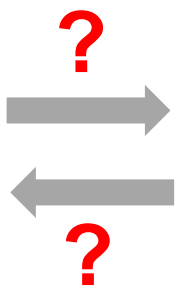
Science Transformation

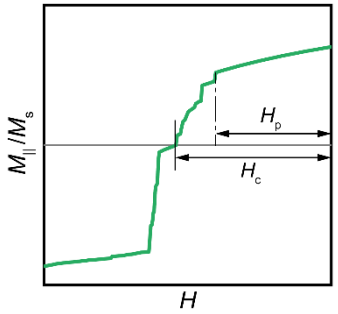


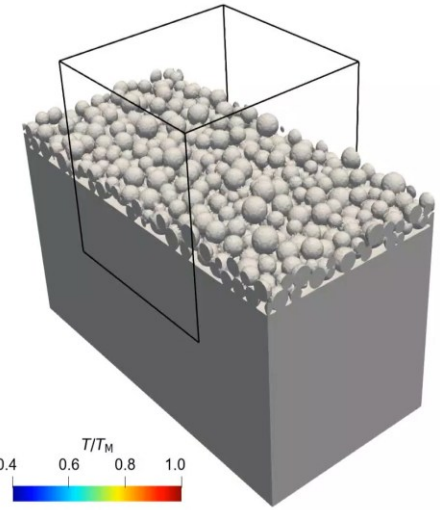
Modified from Schleder et al. J. Phys. Mater. 2019

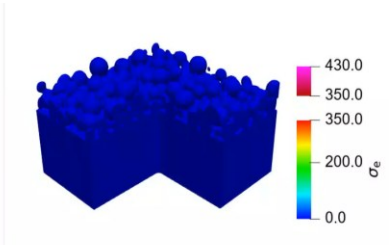
Motivation: Example 1 Property prediction? Inverse design?

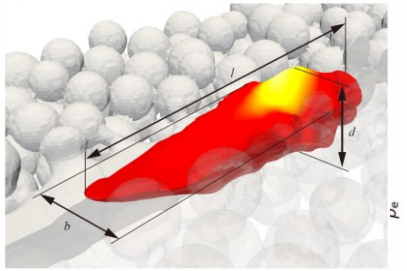


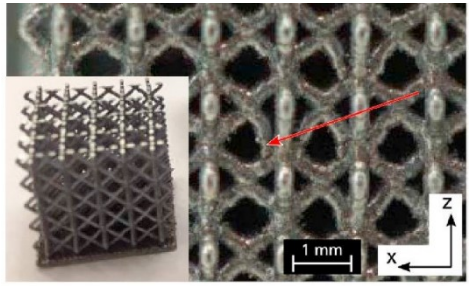








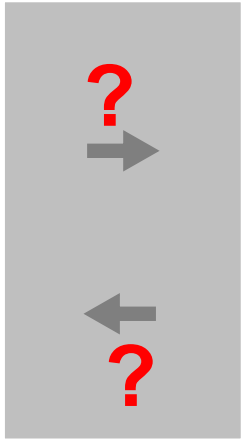




laser and process
P, v, D_l, α ...

powder bed
ψ, d

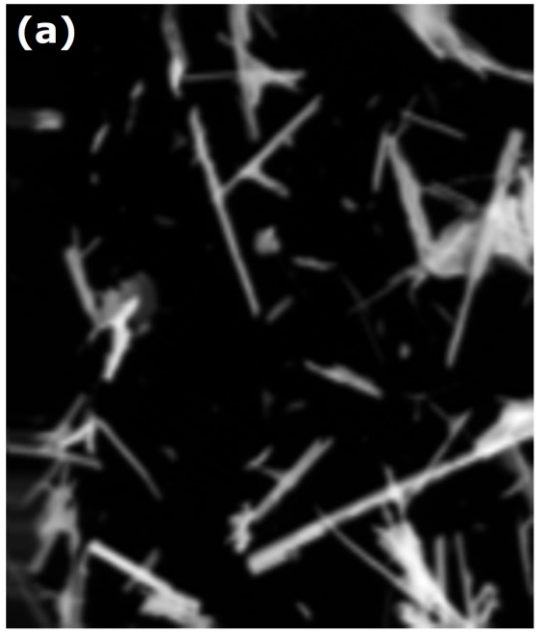
thermal
$k_{pb}^{T_M}, k'_{pb}$,
$c_{pb}^{T_M}, c'_{pb}$



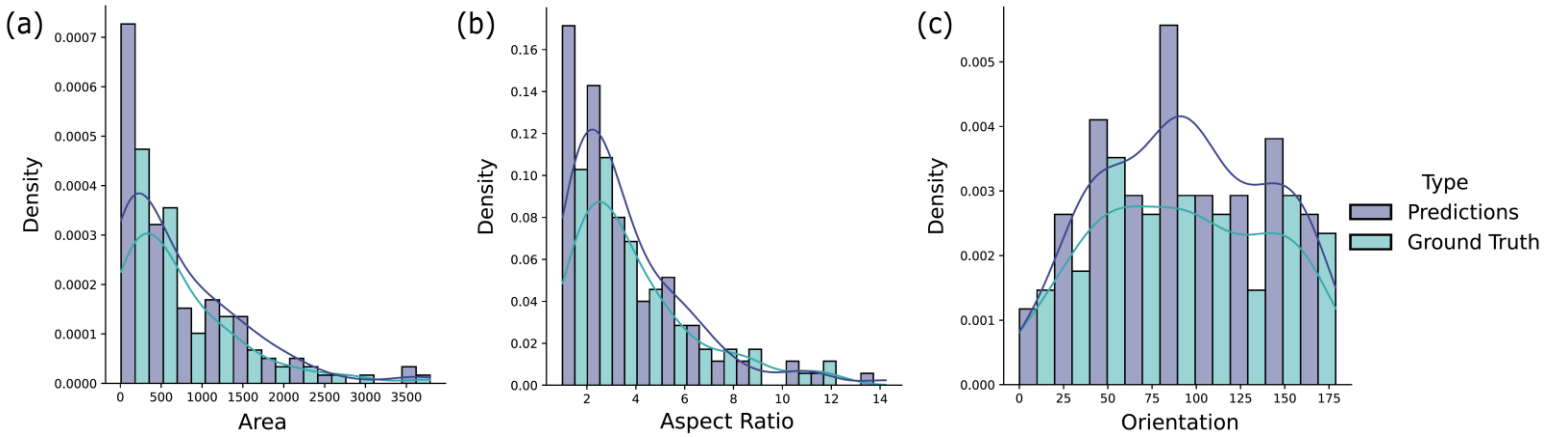
b
melt pool width

30000 datasets of input and output
~ 50 experimental datasets

Motivation: Example 2 Microstructure characterization ?



?



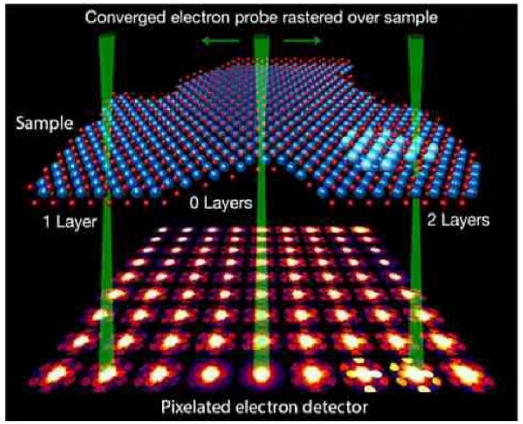
Motivation: Example 3 Crystal structure characterization ?

Conventional template matching

- Templates are generated via crystal structure simulation
- Calculation of *Correlation Index* (*CI*)

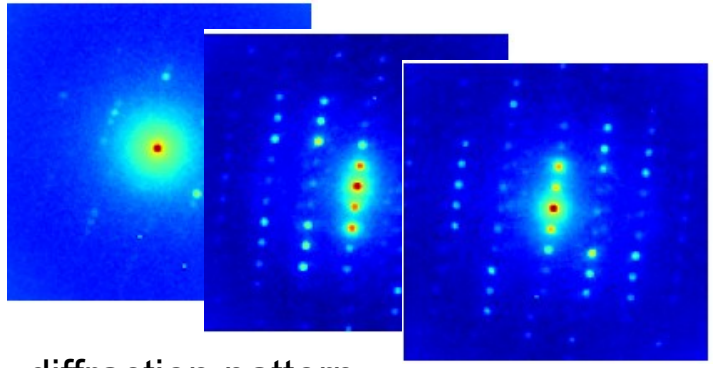
$$CI\ (i) = \frac{\sum_{j=1}^{m} P(x_j, y_j) T_i(x_j, y_j)}{\sqrt{\sum_{j=1}^{m} P^2(x_j, y_j)} \sqrt{\sum_{j=1}^{m} T_i^2(x_j, y_j)}}$$

- Slow, computational expensive and potentially imprecise



Scanning Transmission Electron Microscopy 4DSTEM

Ophus 2019 - Four-Dimensional Scanning Transmission Electron Microscopy.jpg (Ophus 2019, p. 564)



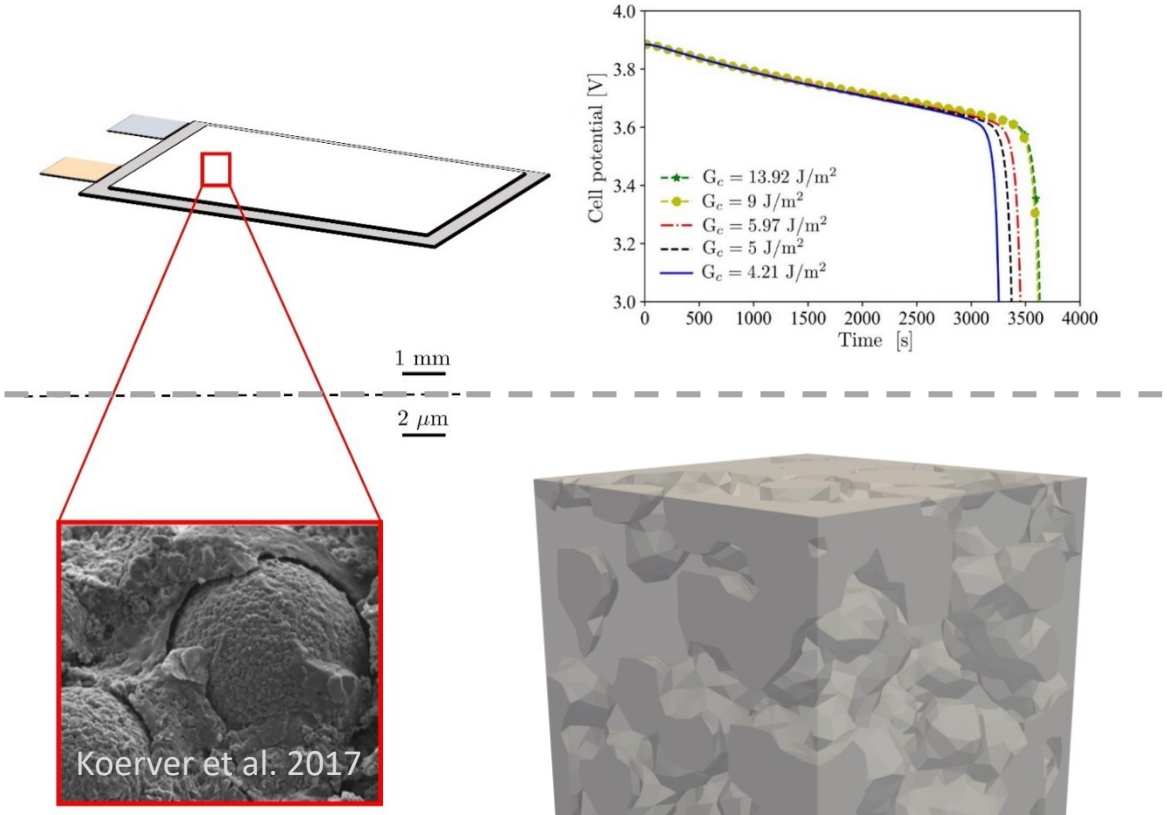
diffraction pattern

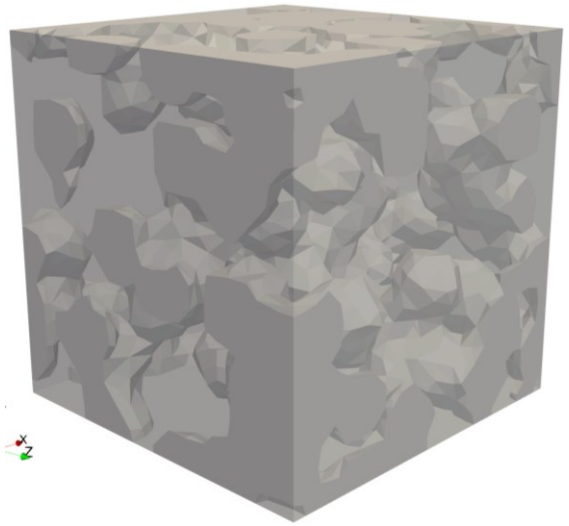
?

Euler angles
grain orientation

MECHANICS of FUNCTIONAL MATERIALS

Motivation: Example 4 Speed up simulations ?





One FE^2 calculation of one charge-discharge cycle takes about 4 days in HPC

→ Can we speed up the simulations by ML?

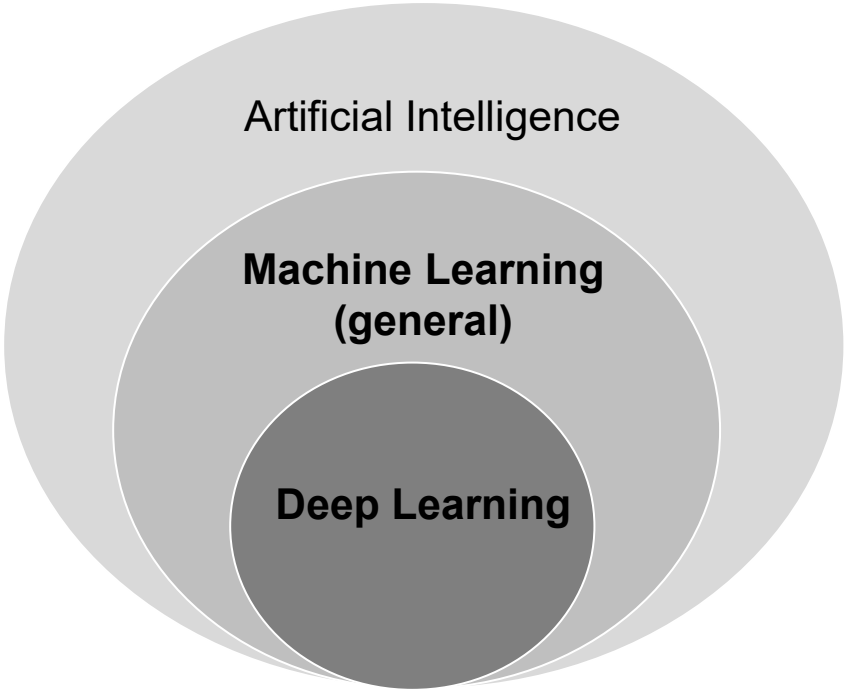
TECHNISCHE UNIVERSITÄT DARMSTADT

Recap ML: from Shallow to Deep Learning
Neural Network

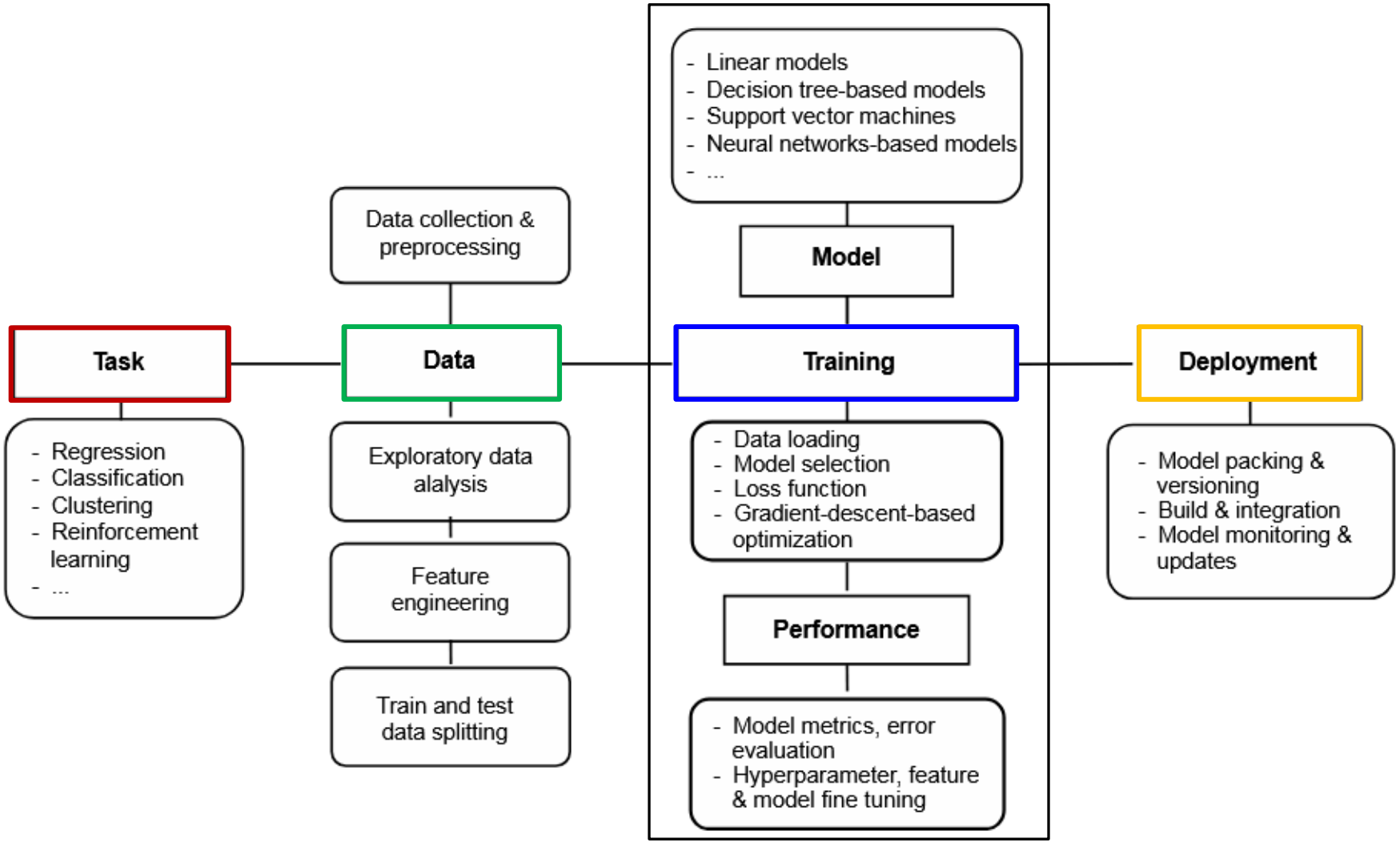
MECHANICS of FUNCTIONAL MATERIALS

Recap ML (general)

Mitchell (1997): A computer program is said to learn from experience *E* w.r.t. some class of tasks *T* and performance measure *P*, if its performance at tasks in *T*, as measured by *P*, improves with experience *E.*



General workflow



Recap ML (general)

Regression **Task** on e.g. the given data

Grain size X [nm]	100	125	158	336	1260	1585
Relative dielectric constant (Y^*)	2750	3250	3750	4000	3500	3000

Comments:

- Large scale DFT calculation based databases on crystal structure property are available, e.g. Materials Project, AFLOW, OQMD and MPDS
- Repositories like NOMAD and Matminer for cleaning, mining and analyzing of the raw data from these DFT-based databases. Some has been equipped with also machine learning (ML) tools
- There is yet unfortunately no available large scale database on microstructure and macroscopic materials property.

From Regression to Machine Learning (shallow)

Model: linear regression

$Y = wX$ w: Parameter t.b.determined

Objective function: Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(Y - Y^*)^2$$

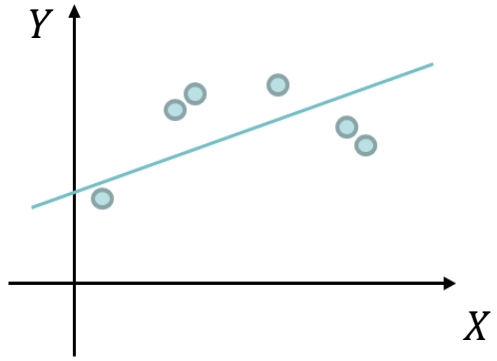
Iterative optimization: Gradient Descent

$$w_{ML} = \underset{X}{\operatorname{argmin}}\, MSE(X^*, Y^*; w)$$

Finding w_{ML} to minimize MSE (iteratively).

Start with initial w_0, update

$$w_{i+1} = w_i + \Delta w_i, \qquad \Delta w_i = \frac{\mathrm{d}(MSE)}{\mathrm{d}w}\mu$$



ML is more than this !

Mitchell (1997): "A computer program is said to learn from experience *E* w.r.t. some class of tasks *T* and performance measure *P*, if its performance at tasks in *T*, as measured by *P*, improves with experience *E*.„

While pure optimization aims at minimizing/maximizing objectives for the given problem/data, ML aims at minimizing/maximizing objectives for unseen data in use of the given data. To decrease the generalization error, it is ready even to scrifice small training error.

From Regression to Machine Learning (shallow)

Model: linear regression

$Y = wX$ w: Parameter t.b.determined

Objective function: Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(Y - Y^*)^2$$

Iterative optimization: Gradient Descent

$$w_{ML} = \underset{X}{\operatorname{argmin}}\, MSE(X^*, Y^*; w)$$

Finding w_{ML} to minimize MSE (iteratively).

Start with initial w_0, update

$$w_{i+1} = w_i + \Delta w_i, \qquad \Delta w_i = \frac{\mathrm{d}(MSE)}{\mathrm{d}w}\mu$$

Model: ML regression

$Y = wX^p$ w: Parameter t.b.determined

Objective function: Mean Squared Error + regularization

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(Y - Y^*)^2 + \lambda w^T w$$

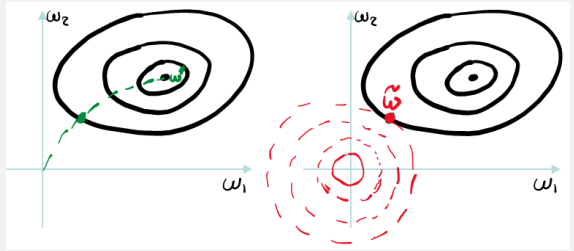
Iterative optimization: Gradient Descent

$$w_{ML} = \underset{X}{\operatorname{argmin}}\, MSE(X, Y^*; w)$$

Finding w_{ML} to minimize MSE (iteratively).

Start with initial w_0, update

$$w_{i+1} = w_i + \Delta w_i, \quad \Delta w_i = \frac{\mathrm{d}(MSE)}{\mathrm{d}w}\mu + m\Delta w_{i-1}$$



Hyperparameters:
w_0: initial slope/weight
μ: learning rate
m: momentum, [0,1]
p: defining the function class
λ: Controlling regularization
($L1$: $|w|$; $L2$: $w^T w$)
others e.g. when iteration stops

Recap ML (general)

Model: ML regression

$Y = wX^p$ w: Parameter t.b.determined

Objective function: Mean Squared Error + regularization

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(Y - Y^*)^2 + \lambda w^T w$$

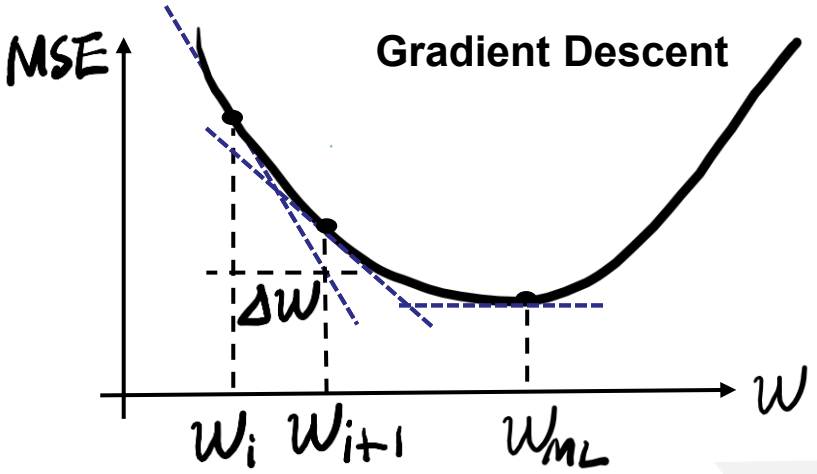
Iterative optimization: Gradient Descent

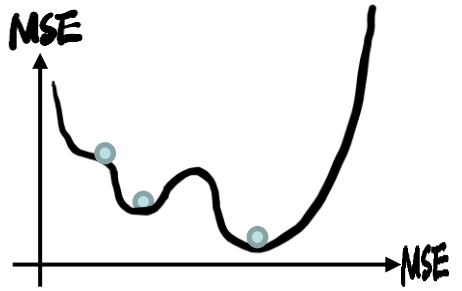
$$w_{ML} = \underset{X}{\operatorname{argmin}}\, MSE(X, Y^*; w)$$

Finding w_{ML} to minimize MSE (iteratively).

Start with initial w_0, update

$$w_{i+1} = w_i + \Delta w_i, \quad \Delta w_i = \frac{\mathrm{d}(MSE)}{\mathrm{d}w}\mu + m\Delta w_{i-1}$$





One model parameter: $w(\rightarrow w_{ML})$

Hyperparameters in optimization:

- w_0: initial slope/weight
- μ: learning rate
- m: momentum, [0,1]
- p: defining the function class
- λ: Controlling regularization ($L1$: $|w|$; $L2$: $w^T w$)

others e.g. when iteration stops

Just like this linear regression model, most Neural Networks also use MSE as objective function and Gradient Descent as the core idea of iterative optimization called *Backpropagation.*

*Thereby **X**, **Y**, **w**, all become higher dimensional vector or tensors:* $Y_i = \sum_j w_{ij} X_j$

Recap ML (general)

Model: **Gaussian Process Regression** (probability distribution of the to-be-determined function)

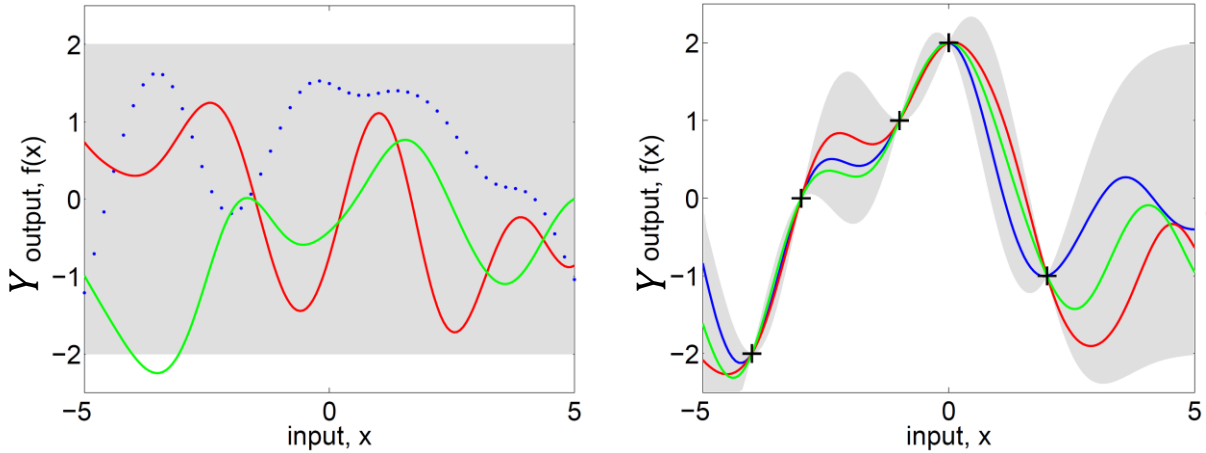
Objective: **maximum a posteriori** (maximum likelihood penalized)

Iterative Graussian Process:

$$P(y_{t+1}|D_{1:t}, x_{t+1}) = \aleph(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1}) + \sigma_{noise}^2)$$

$$\mu_t(x_{t+1}) = \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1} y_{1:t} \quad \sigma_t^2(x_{t+1}) = k(x_{t+1}, x_{t+1}) - \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1}$$

Squared-exponential covariance $\mathbf{K}(x_p, x_q) = \sigma_f^2 \exp\left[-\frac{1}{2l^2}(x_p - x_q)^2\right]$



Distribution regions of the output function: Initial/prior (gray on left) and Posteriori (gray on right) after considering three data points

No model parameter

Hyperparameters in iterative process:

$\mu_0(x), \sigma_0(x)$: initial mean/variance functions
σ_{noise}^2 : noise variance
σ_f^2 : signal variance
l : length-scale

Bayesian Optimization (Part 3), used for searching best property, optimizing hyperparameters (also for NN), inverse design, is mostly based on Gaussian Process Regression.

Thereby the so-called acquisition or utility function should be defined, e.g.

$$u(x) = \mu(x) + \alpha\,\sigma(x)$$

Maximizing $u(x)$ to find the optimum.

Recap ML (general)

TECHNISCHE UNIVERSITÄT DARMSTADT

Classification Task on e.g. the given data

Atom fraction X (continuous) in system $\mathrm{Ba}(\mathrm{Zr}_X\mathrm{Ti}_{1-X})\mathrm{O}_3$	0	0.1	0.2	0.5	0.6	0.9
Ferroelectric state Y^*: Ferroelectrics (FE) Relaxor ferro. (RE) Dielectrics (DE)	FE	FE	RE	RE	RE	DE
One-Hot Encoding P^*: FE→[0,0,1] RE→[0,1,0] DE→[0,0,1]	[1,0,0]	[1,0,0]	[0,1,0]	[0,1,0]	[0,1,0]	[0,0,1]

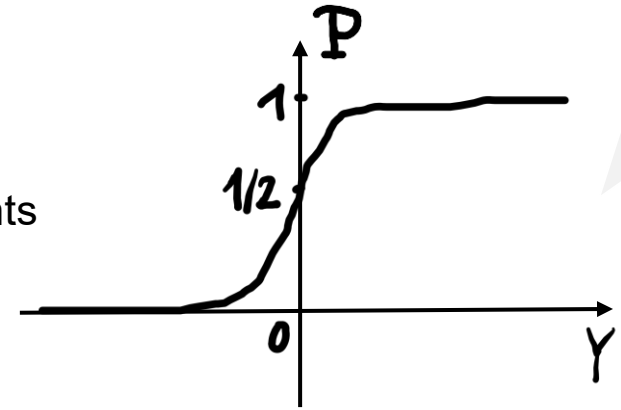
Recap ML (general)

Model: **linear logistic regression** (via linear function)

$$P = \mathrm{sig}(Y) = \frac{1}{1+\exp(-Y)},\ Y = wX \qquad \text{for a binary system}$$

$$P = \mathrm{softmax}(Y_i \mid \boldsymbol{Y}) = \frac{\exp(Y_i)}{\sum_{l=1}^{L}\exp(Y_l)},\quad Y_i = \sum_j w_{ij}X_j \quad \text{for } L\text{-variants}$$

w or w_{ij} : Model Parameters to be determined



Similarly, Gaussian Process regression model can also be extended for Gaussian Process Classification via sigmoid function or softmax functions.

Objective function: standard **Cross-Entropy loss**

$$CE = \sum_{i=1}^{L} Y_i \log(Y_i^*) + \lambda\ (\text{Regularization model})$$

Iterative optimization: **Gradient Descent**

$$w_{ML} = \arg\min CE(X, Y^*; w)$$

Finding w_{ML} to minize CE (iteratively): Start with initial w_0, and update via

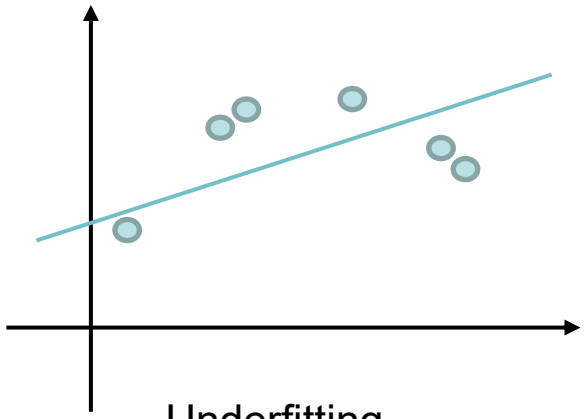
$$w_{i+1} = w_i + \Delta w_i,\quad \Delta w_i = \frac{\mathrm{d}(CE)}{\mathrm{d}w}\mu + m\Delta w_{i-1}$$

Model parameter: $w(\rightarrow w_{ML})$

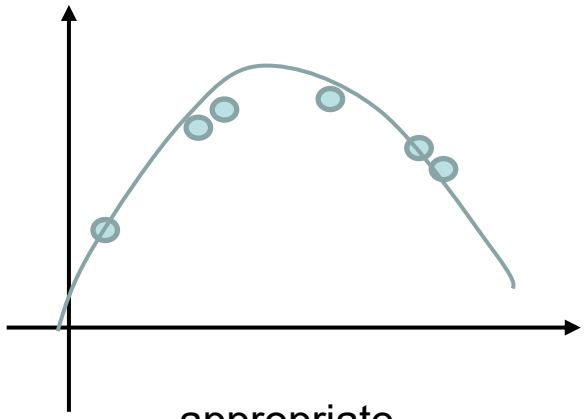
Hyperparameters in optimization:

- w_0: initial slope/weight
- μ: learning rate
- m: momentum, [0,1]
- λ: Controlling regularization ($L1$: $|w|$; $L2$: $w^T w$)

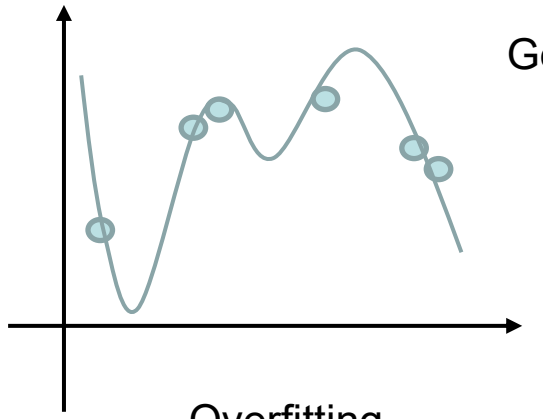
Recap ML (general): Regression



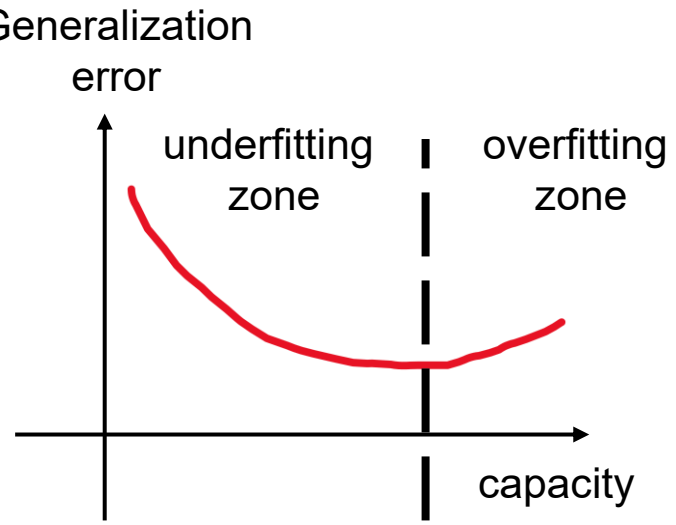
Underfitting
excessive λ



appropriate
medium λ



Overfitting
small λ



Good Practitioners can

- data processing techniques (Bucketize; Normalization; Augmentation; Validation; Cross-validation)
- rich experience on hyperparameter tuning and optimization (e.g. how to avoid local minimum and how to avoid overfitting by reducing feature numbers, regularization, early stopping, dropout)
- decision on whether/how to gather more data and how to increase regularization
- rich debugging skills: visualize the model in action, analyze the worst mistakes; monitor histograms of activations and gradients; compate derivatives from different approaches…

……

Recap ML (general)

TECHNISCHE UNIVERSITÄT DARMSTADT

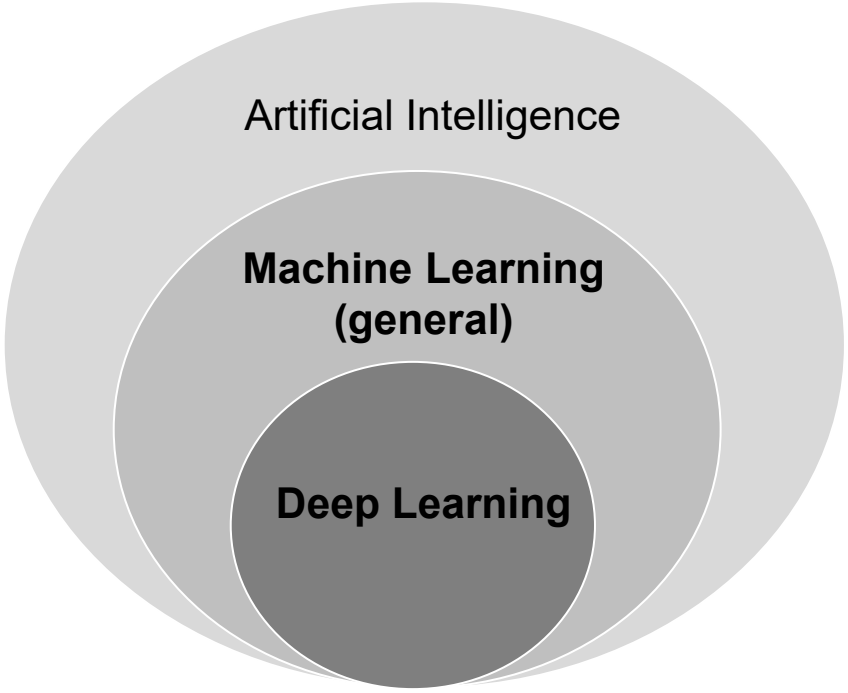
Tasks (*T*)	Models (*E*)	Learning algorithms (*E*)	Performance (*P*)
• **Regression** (continous output *Y* as function of *X*) • **Classification** (discrete output *Y* as function of *X*, also called logistic regression) ○ Clustering (no explicit output *Y*, but grouping *X*) ○ Recommendation (no explicit output *Y*, but rules on *X*) …	Supervised: – Restricted Function class – Gaussian Process – Support Vector Machines – Random Forest – Decision Tree – *k*-nearest Neighbours – Neural Networks (DL) Unsupervised: – Principle Components Analysis – *k*-means Clustering – Neural Networks (D Belief L) Reinforced: – Feedback loop btw. learning and experience …	Nearly all learing algorithms share a fairly simple recipe: ✓ a specification of **data** (*X, Y* if supervised learing) ✓ a **model** ✓ An **objective** function, e.g. *mean squared error* or *cross entropy*. Depending on the situations, it can be also called error or cost function (the average loss over the entire training dataset) ✓ an **iterative optimization** procedure (e.g. *gradiend descent*)	Evaluation: ▪ **Training Error**, **Test** (generalization) **Error.** (**Both** errors should be **small)** ▪ **Underfitting** (large training error); **Overfitting** (large test error, i.e. bad generalization) ▪ … Improvement: ▪ **Regularization** (i.e. modifying the cost function) to improve generalization ▪ **Validation**: one subset of training data to learn **hyperparameter**, while the other to learn **parameter** ▪ ***k*-fold cross-validation**: divide training dataset into *k* subsets; on the i-th trial, the i-th subset is used as test data, and the others for training; Test error is then the average test error across *k* trials. ▪ …

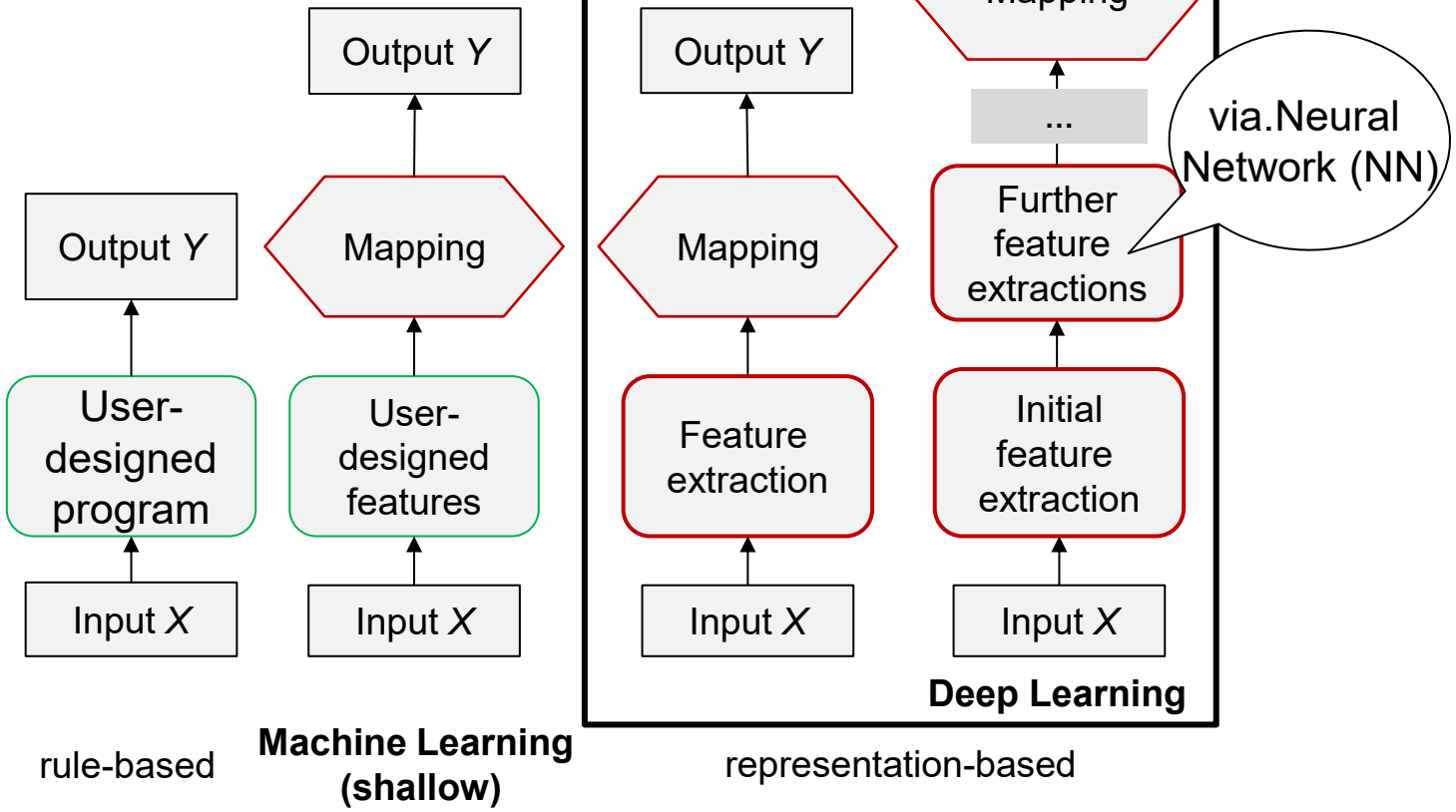
Challenges motivating Deep Learning

- Curse of Dimensionality ($\sim 10^d$)
- In order to generalize well, ML algorithms need to be guided by prior beliefs about what kind of function they should learn. This is difficult, and more flexible models are needed
- Manifold, hierarchical and unknown features
- Need for a universal model with high flexibility to handle the above challenges

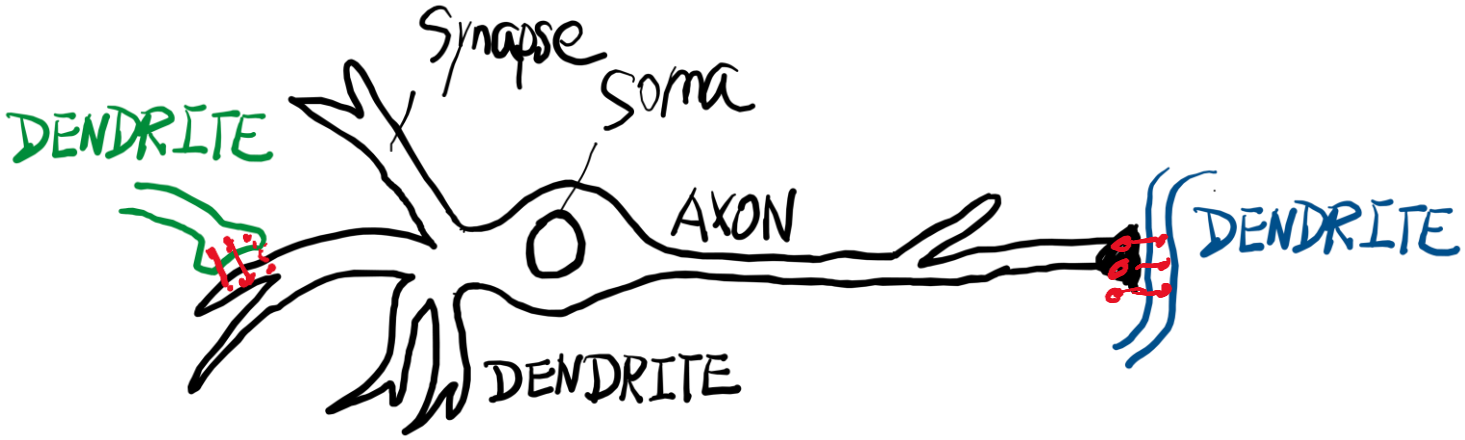
Machine Learning (ML shallow) vs. Deep Learning (DL)

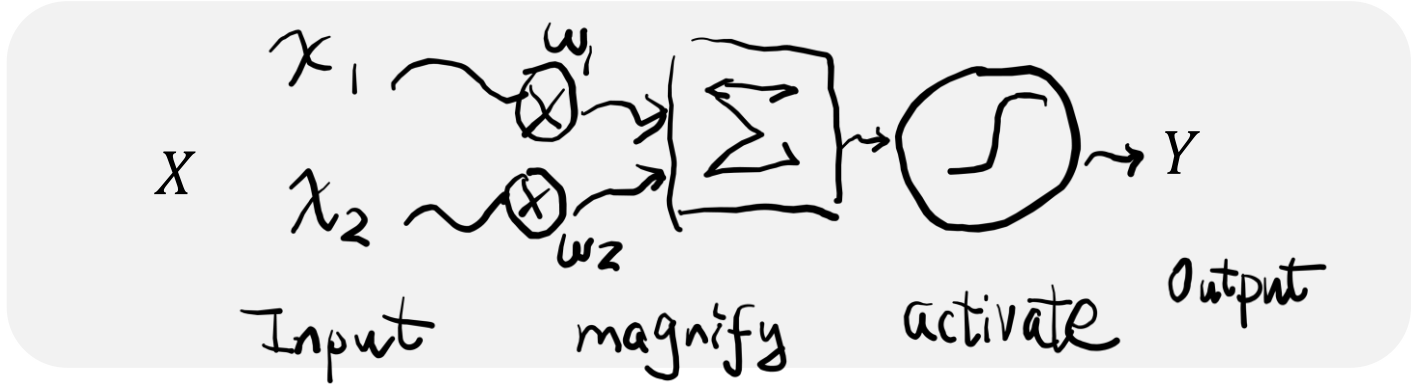
- Mitchell (1997): “A computer program is said to learn from experience *E* w.r.t. some class of tasks *T* and performance measure *P*, if its performance at tasks in *T*, as measured by *P*, improves with experience *E*.„
- LeCun: “**Deep Learning = Learning Representations/Features**”



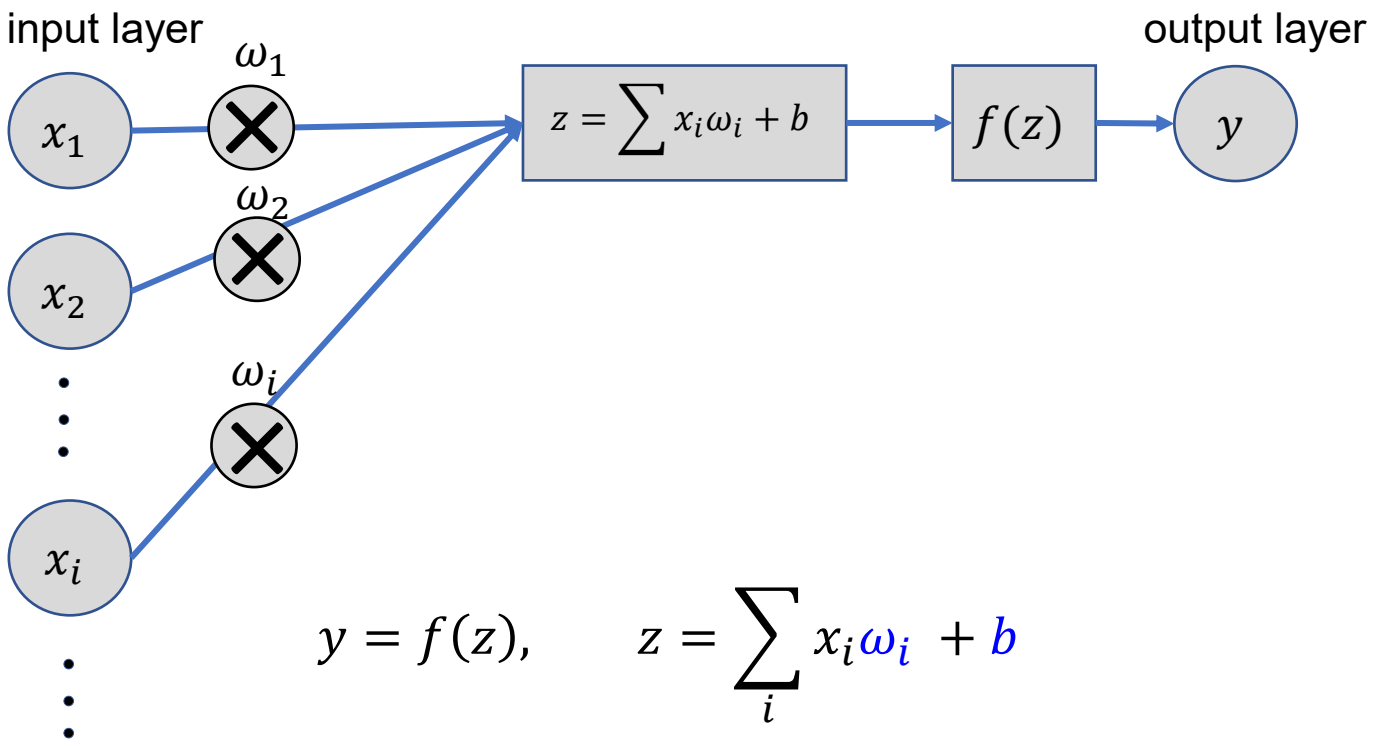


Neural Network



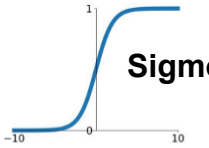


Perceptron

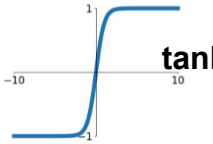


$$y = f(z), \qquad z = \sum_i x_i\omega_i + b$$

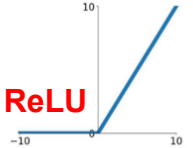
f: activation function (unit)
There are many options.



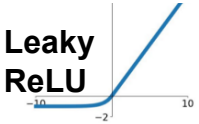
Sigmoid $f(z) = \mathrm{sig}(z) = \dfrac{1}{1+\exp(-z)}$



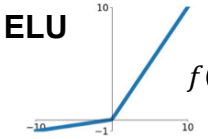
tanh $f(z) = \tanh(z)$



ReLU $f(z) = \max(0, z)$



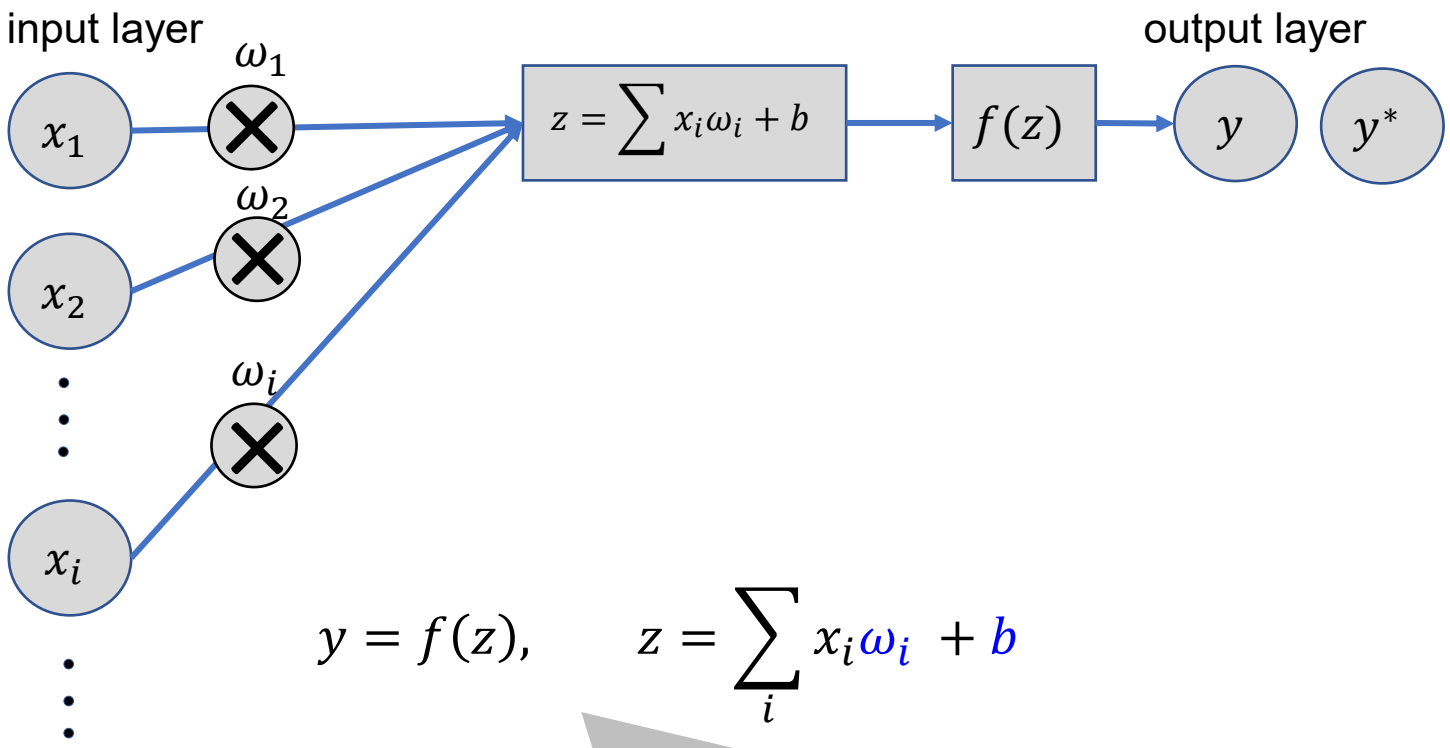
Leaky ReLU $f(z) = \max(0.1z, z)$



ELU $f(z) = \begin{cases} z, & z \geq 0 \\ \alpha(\exp(z) - 1), & z < 0 \end{cases}$

Maxout $\max(z_j)$

Perceptron



$$y = f(z), \qquad z = \sum_i x_i\omega_i + b$$

Chain rule $\frac{\partial J}{\partial \omega_i} = \frac{\partial J}{\partial y}\frac{\partial y}{\partial \omega_i} = \frac{\partial J}{\partial y}\frac{\partial f}{\partial z}\frac{\partial z}{\partial \omega_i} = 2y\frac{\partial f}{\partial z}x_i$

If $f = \mathrm{sig}(z)$, $\frac{\partial f}{\partial z} = f(1-f)$. Then $\frac{\partial J}{\partial \omega_i} = 2y^2(1-y)x_i$

Objective function: **Mean Squared Error**

$$J = J(\omega_i, b) = (y - y^*)^2 = (f(x_i\omega_i + b) - y^*)^2$$

Compare that of the Linear regression:

$$J = J(\omega_i, b) = (y - y^*)^2 = (x_i\omega_i + b - y^*)^2$$

Iterative optimization: **Gradient Descent**

$$\omega_{ML} = \arg\min J(x_i, y^*; \omega_i, b)$$

Start with initial ω_i, b, update

$$\omega_i|_{t+1} = \omega_i|_t + \Delta\omega_i|_t, \quad \Delta\omega_i|_t = \frac{\partial J}{\partial \omega_i}\mu + m\Delta\omega_i|_{t-1}$$

Model parameter: $\omega_i(\rightarrow \omega_{i_{ML}}), b(\rightarrow b_{ML})$

Hyperparameters in optimization:

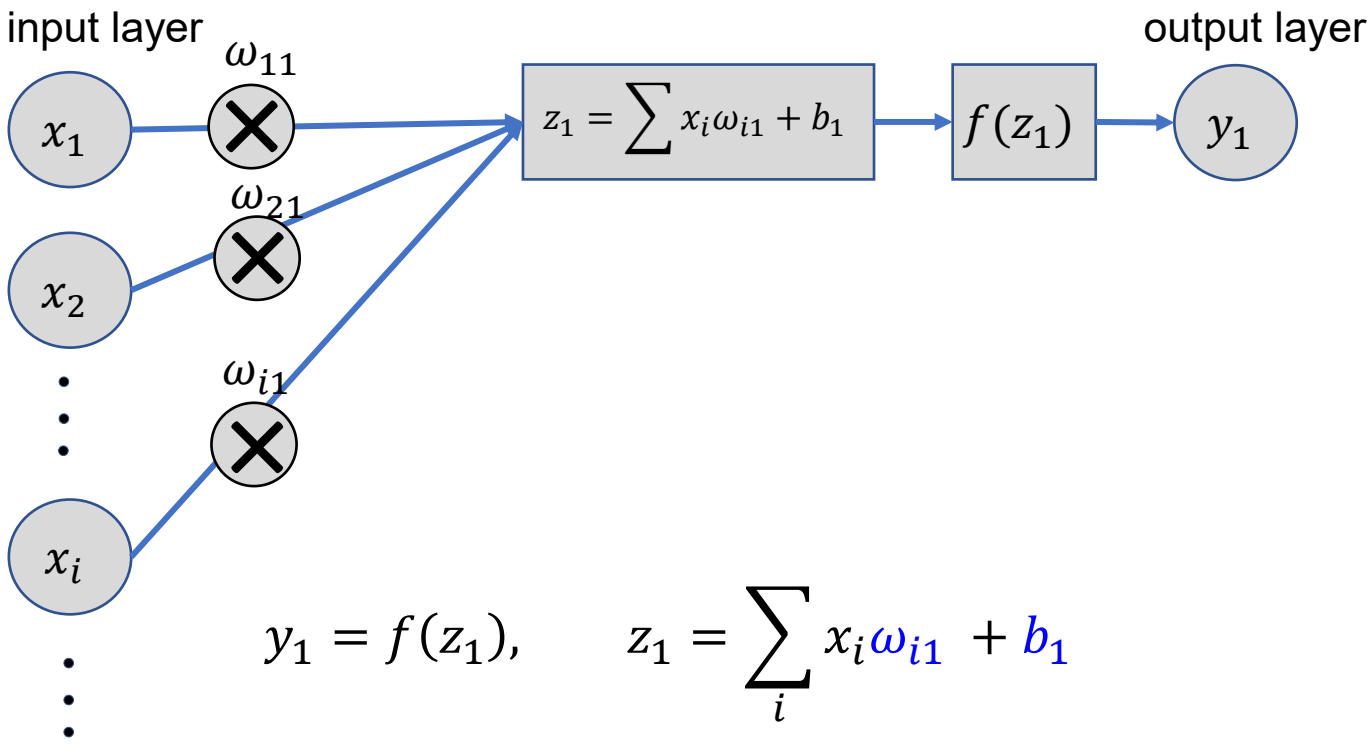
Initial ω_i, b

μ: learning rates

m: momentums, [0,1]

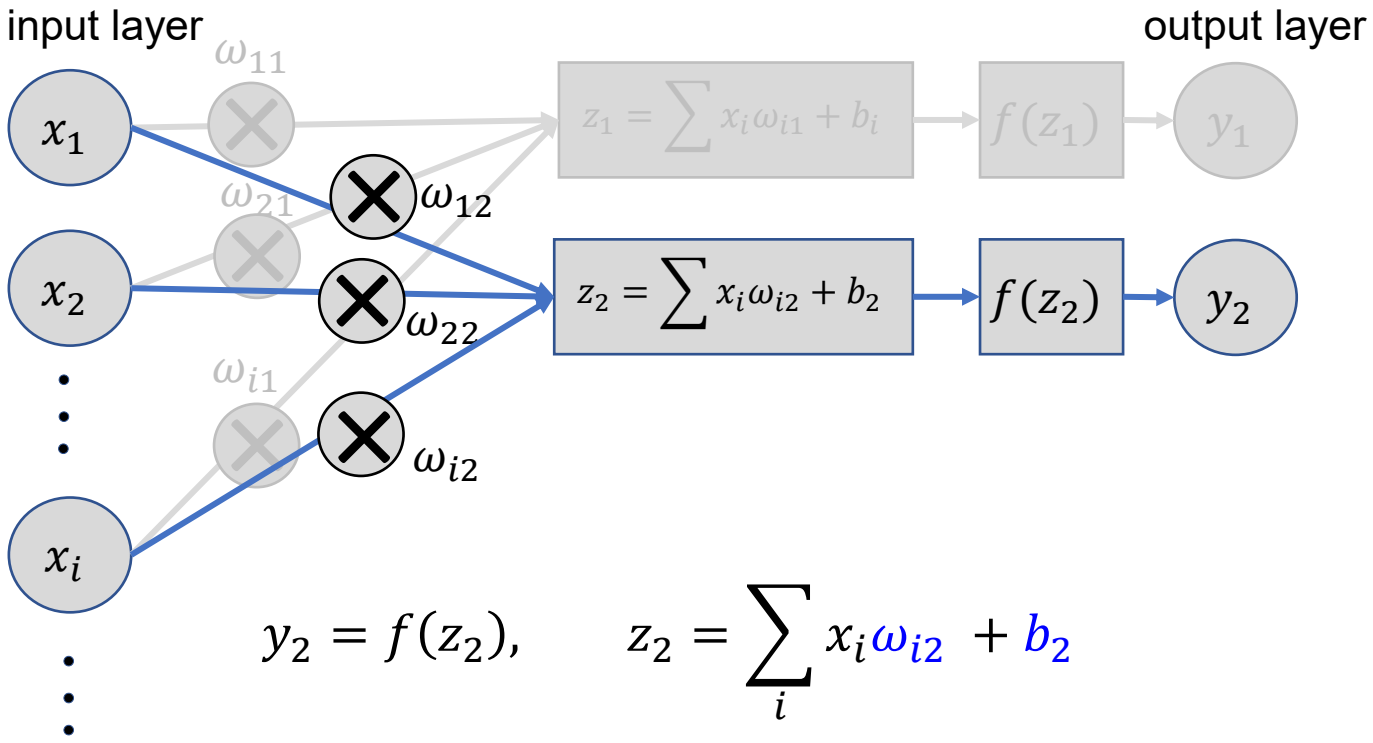
λ: regularization parameters and more

Perceptron



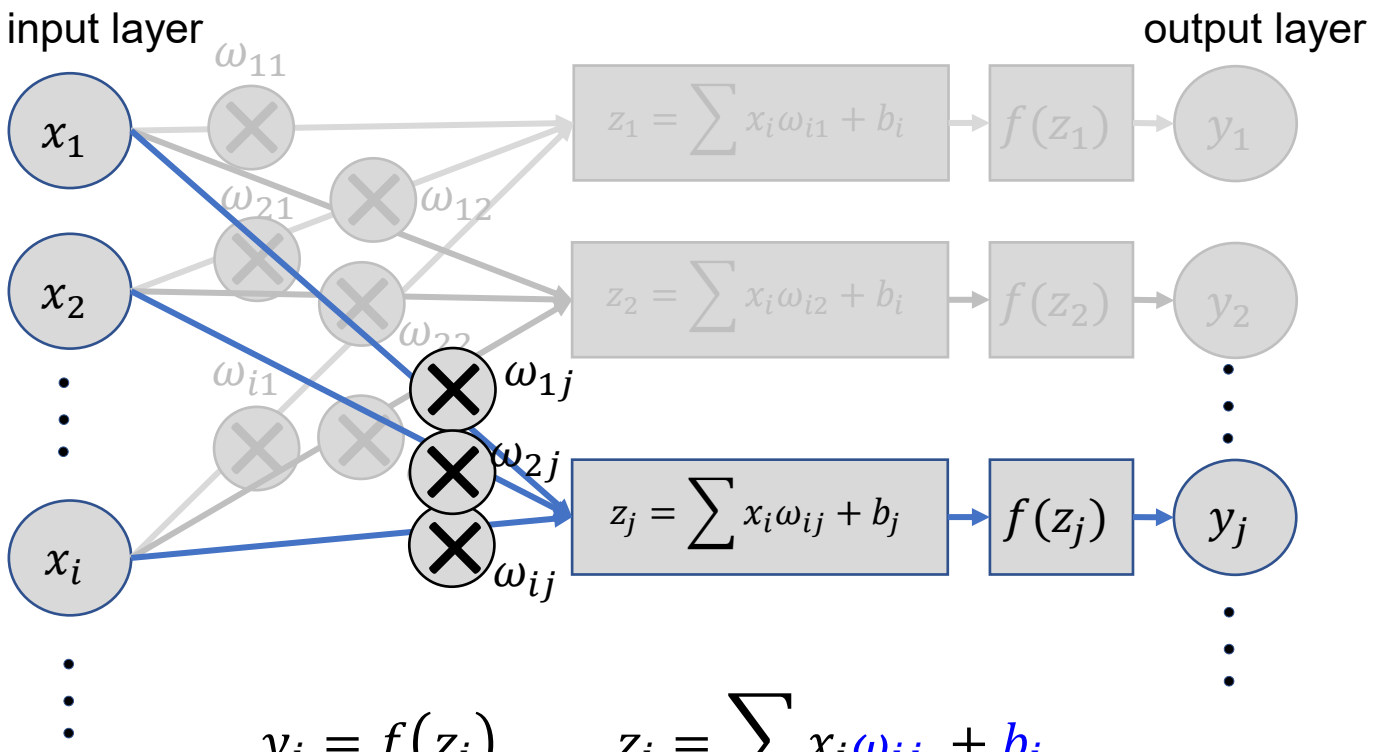
$$y_1 = f(z_1), \qquad z_1 = \sum_i x_i \omega_{i1} + b_1$$

Perceptron



$$y_2 = f(z_2), \qquad z_2 = \sum_i x_i \omega_{i2} + b_2$$

Perceptron



$$y_j = f(z_j), \qquad z_j = \sum_i x_i\omega_{ij} + b_j$$

Chain rule $$\frac{\partial J}{\partial \omega_{ij}} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial \omega_{ij}} = \frac{\partial J}{\partial y_j}\frac{\partial f}{\partial z_j}\frac{\partial z_j}{\partial \omega_{ij}} = 2y_j\frac{\partial f}{\partial z_j}x_i$$

Objective function: **Mean Squared Error**

$$J = J(\omega_{ij}, b_j) = \left|\left|y_j - y_j^*\right|\right|^2$$

Iterative optimization: **Gradient Descent**

$$\omega_{ML} = \arg\min J(x_i, y_j^*; \omega_{ij}, b_j)$$

Start with initial ω_{ij}, b_j, update

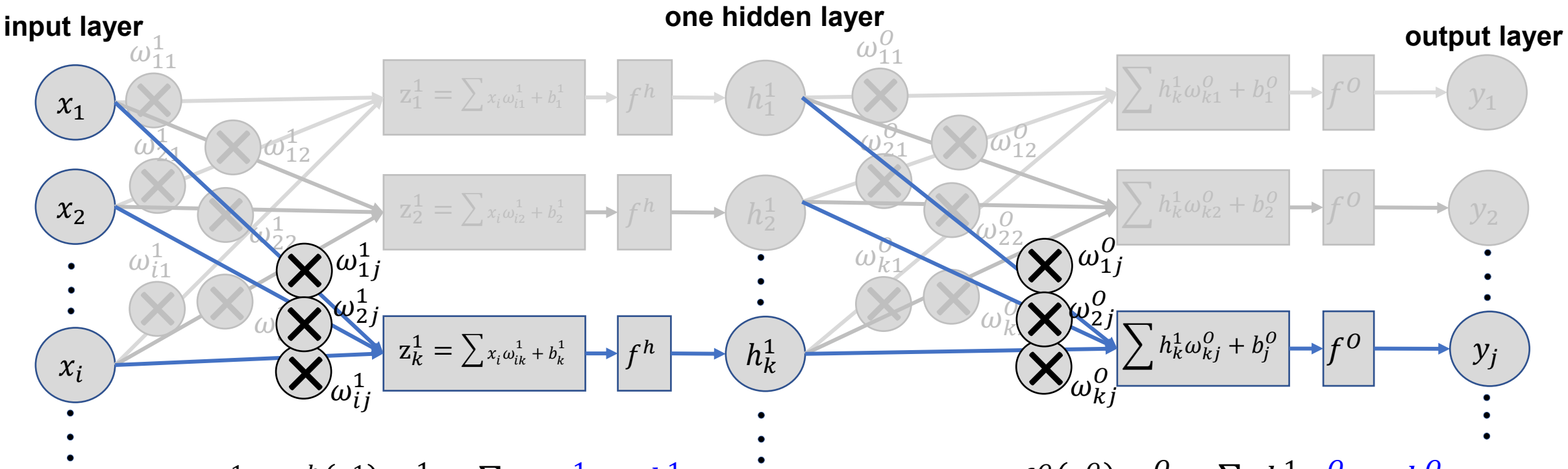
$$\omega_{ij}|_{t+1} = \omega_{ij}|_t + \Delta\omega_{ij}|_t, \quad \Delta\omega_{ij}|_t = \frac{\partial J}{\partial \omega_{ij}}\mu + m\Delta\omega_{ij}|_{t-1}$$

Model parameter: $\omega_{ij}\left(\rightarrow \omega_{ij_{ML}}\right), b_j(\rightarrow b_{j_{ML}})$

Hyperparameters in optimization:

- Initial ω_{ij}, b_j
- μ: learning rates
- m: momentums, [0,1]
- λ: regularization parameters and more

Multilayer Perceptron



$$h_k^1 = f^h(z_k^1),\ z_k^1 = \sum_i x_i \omega_{ik}^1 + b_k^1 \qquad y_j = f^o(z_j^o),\ z_j^o = \sum_k h_k^1 \omega_{kj}^o + b_j^o$$

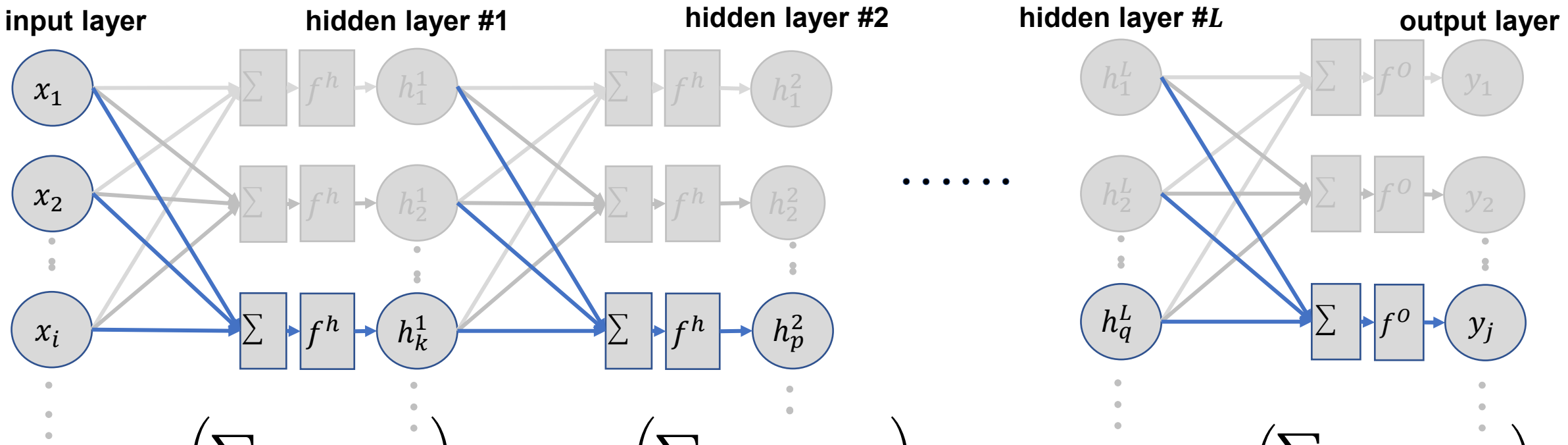
f^h is the hidden layer unit (sig, tanh, **ReLU**, Maxout…)

f^o the out layer unit (e.g. linear for regression, sig or softmax for classification).

$$\frac{\partial J}{\partial \omega_{ik}^1} = \sum_j \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial \omega_{ik}^1} = \sum_j \frac{\partial J}{\partial y_j}\frac{\partial f^o}{\partial z_j^o}\frac{\partial z_j^o}{\partial \omega_{ik}^1} = \sum_j \frac{\partial J}{\partial y_j}\frac{\partial f^o}{\partial z_j^o}\omega_{jk}^o\frac{\partial h_k^1}{\partial \omega_{ik}^1} = \sum_j \frac{\partial J}{\partial y_j}\frac{\partial f^o}{\partial z_j^o}\omega_{jk}^o\frac{\partial f^h}{\partial z_k^1}\frac{\partial z_k^1}{\partial \omega_{ik}^1}$$

$$\frac{\partial J}{\partial \omega_{kj}^o} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial \omega_{kj}^o} = \frac{\partial J}{\partial y_j}\frac{\partial f^o}{\partial z_j^o}\frac{\partial z_j^o}{\partial \omega_{kj}^o} = \frac{\partial J}{\partial y_j}\frac{\partial f^o}{\partial z_j^o}h_k^1$$

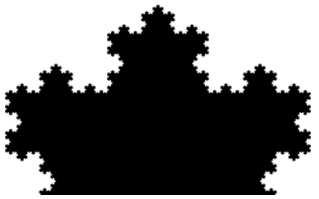
Multilayer Perceptron: Neural Net or Deep Feedforward Net



$$h_k^1 = f^h\left(\sum_i x_i \omega_{ik}^1 + b_k^1\right) \qquad h_p^2 = f^h\left(\sum_k h_k^1 \omega_{kp}^2 + b_p^2\right) \ldots\ldots \qquad y_j = f^o\left(\sum_q h_q^L \omega_{qj}^o + b_j^o\right)$$

Model parameters: $\omega_{ik}^1, b_k^1; \omega_{ip}^2, b_p^2; \ldots; \omega_{qj}^o, b_j^1$.

These hidden layers are kind of feature abstraction/extraction or implicit pattern recognition. NN are universal approximators using simple non-linearities and (self-similar) hierarchy.



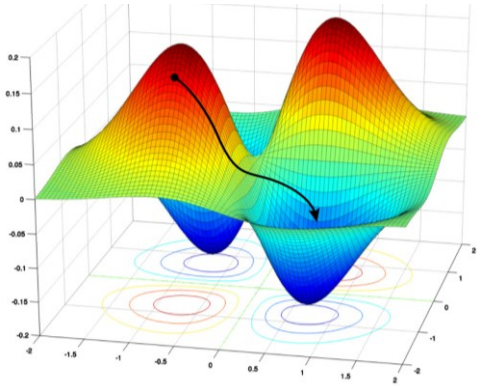
Architecture variation of NN

- In layered structure: the depth of the network (particularly number of hidden layers) and the width of each layer (i.e. number of neurons in each hidden layer)
- Tradeoff btw. Depth and Width. A feedfowrd NN with one single hidden layer is generally capable of representing any function. But the width of the single layer may be infeasibly large and fail to learn and generalize correctly.
- Sparsely connected vs. fully connected.
- Jumping over layers
- Recurrent and recursive nets vs. Feedforward

Optimization

It is a very challenging optizimation problem !!

ill-conditioning, local minima, saddle points, plateaus, ultrahigh dimension…

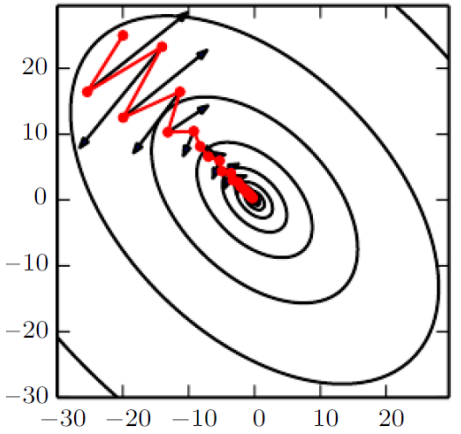


math.stackexchange.com

Basic algorithms:

- Back-Propagation (backprop), the algorithm for training the model parameters based on gradient descent, a type of automatic differentiation
- Stochastic Gradient Descent (SGD): unbiased estimate of gradient by averaging gradient on a minibatch of subsets of examples.
- Momentum approach for poor conditioning and stochastic gradient variance
- Adaptive learning rates (LR) during training
  - delta-bar-delta: decreasing LR if derivative changes sign, otherwise increases
  - AdaGrad: adapts LR of all model parameters by scaling them inversely proportional to the square root of the sum of all historical squared values of the gradient.
  - RMSProp: modification of AdaGrad for nonconvexity by using an exponentially decaying average to discard history from extreme past.
  - Adam: RMSProp + momentum

To be continued…



Goodfellow et al. 2016

Optimization

- Second-Order Methods
  - Newton's Method

    $J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$

    $\rightarrow \boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$, $\boldsymbol{H}$: the Hessian of J with respect to $\boldsymbol{\theta}$ evaluated at $\boldsymbol{\theta}_0$.

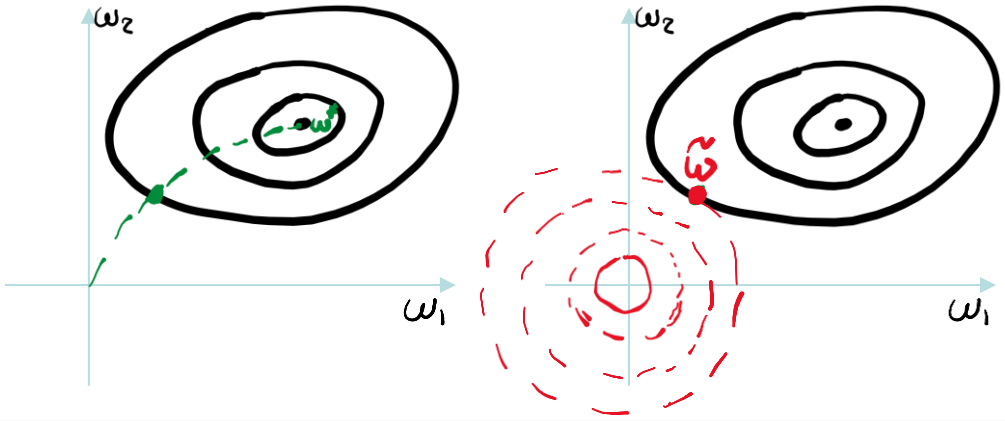
  - Conjugated Gradients: an approach to avoid the calculation of $\boldsymbol{H}^{-1}$

    $\boldsymbol{d}_t = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \beta_t \boldsymbol{d}_{t-1}$, where $\boldsymbol{d}_t^T \boldsymbol{H} \boldsymbol{d}_{t-1} = \boldsymbol{0}$ and β_t given explicitly by gradients (Fletch-Reeves or Polak-Ribiere)
  - Broyden-Fletcher-Goldfarb-Shanno (BFGS): an approach to avoid the calculation of $\boldsymbol{H}^{-1}$
- Overall strategies
  - Batch normalization: an elegant way of reparameterization
  - Coordinate Descent: to solve the optimization fast by breaking it into separate pieces (a subset of variables)
  - Greedy supervised pretraining: break supervised learning problems into other simpler supervised ones
- Intiatial values: mostly heuristically.
  - random (Gauss or normal) initial weights, not too large (to avoid saturation) and not too small (to ensure strong symmetry-breaking)
  - constant bias
  - ML guided choice

Traing Error vs. Generalization Error

It is NOT only a pure optimization problem !!

Whereas pure optimization aims at minimizing/maximizing objectives for the given problem/data, ML aims at minimizing/maximizing objectives for unseen data in use of the given data. To decrease the generalization error, it is ready even to scrifice small training error.

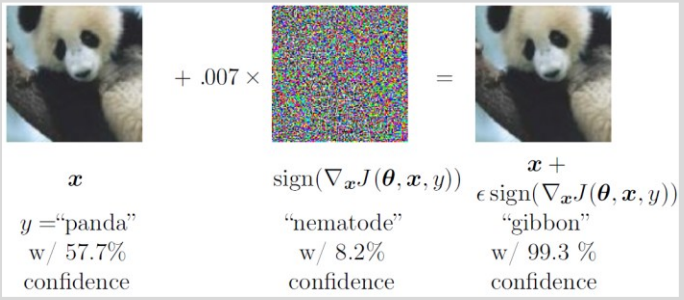


Strategies to decrease generalization error (better performance of a ML model)

- **Regularization:** Modification to the basic objective function on the basis of prior knowledge of the problem

$J = J(\boldsymbol{\omega}, \boldsymbol{b}) + \lambda\Omega(\boldsymbol{\omega})$ L^1-Regularization $\Omega(\boldsymbol{\omega}) = \lambda|\boldsymbol{\omega}|$ L^2-Regularization $\Omega(\boldsymbol{\omega}) = \lambda\boldsymbol{\omega}^{\boldsymbol{T}}\boldsymbol{\omega}$

- **Enlarge Datasets**
- **Dataset Augmentation** e.g. via data transformation and injecting noise
- **Ensemble Methods** by combining several models
- **Dropout**, ensemble all subnetworks after removing nonoutput units from base nets
- **Adversarial Training**

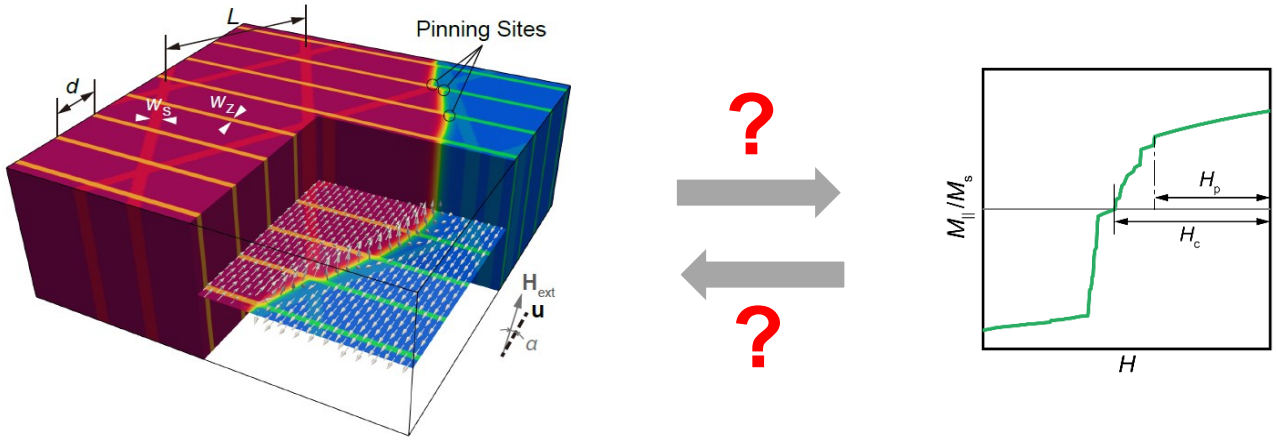


Goodfellow et al. 2014

Questions

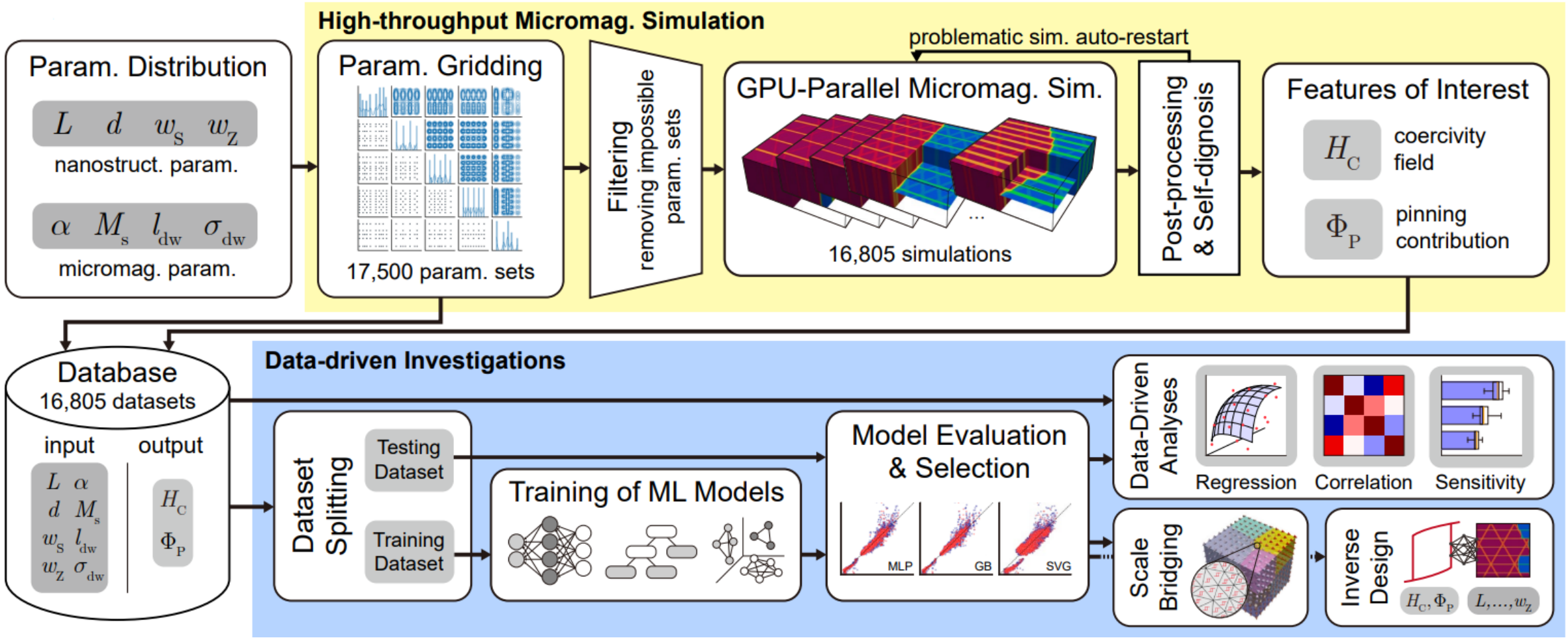
- What hyperparameters do you know about Neural Nets (Networks)?
- What is the difference between hyperparameters and model parameters (weights and bias)?
- Why do we need activation functions?
- What are the advantages/disadvantages of NN compared to the shallow models?
- In principle, we can make the neural nets extremly large to build an arbitary complex model, why we don‘t do that and what are the bottlenecks?
- What if the input is an image?
- How does ML differ from conventional optimization?

Motivation: Example 1 Property prediction? Inverse design?

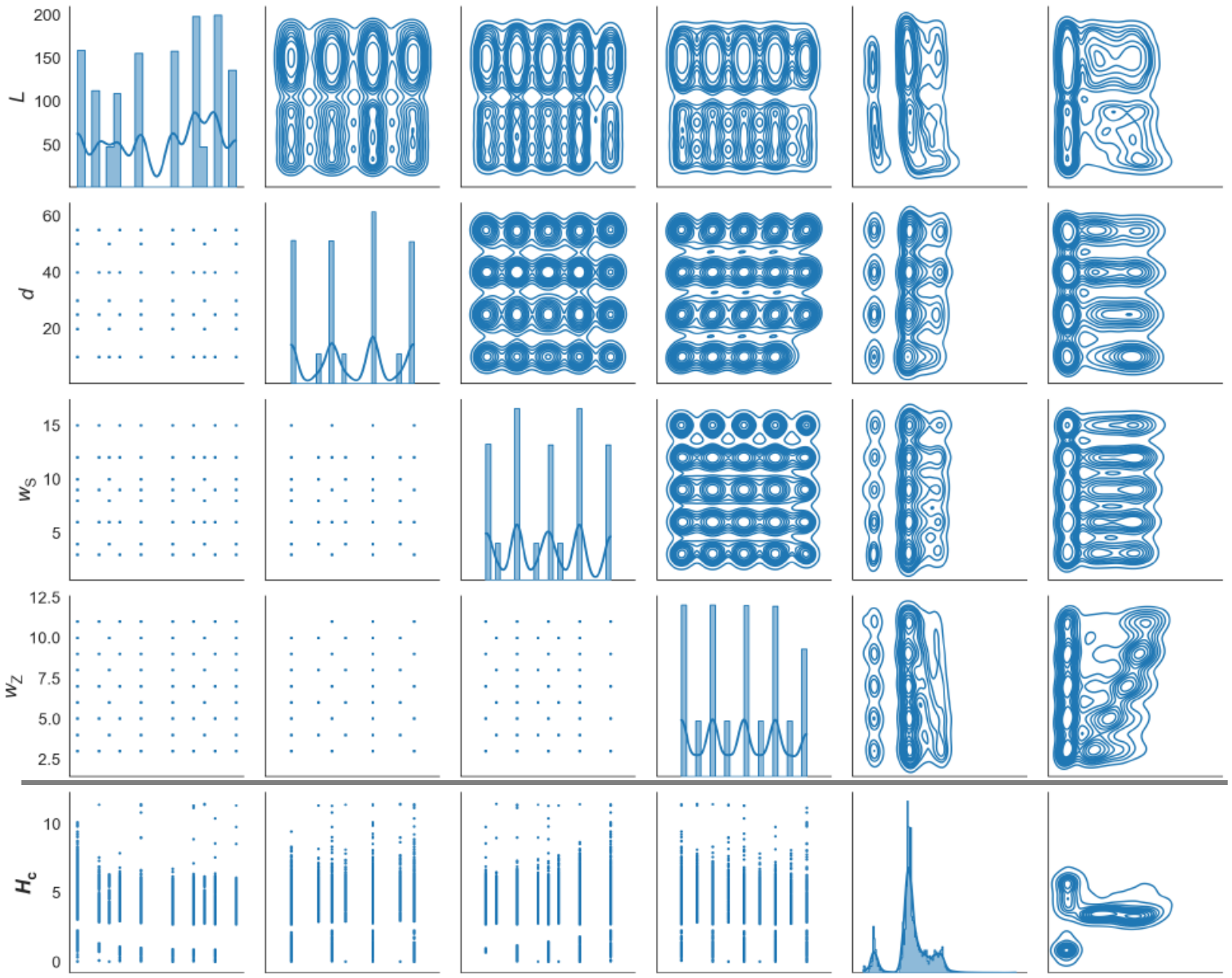


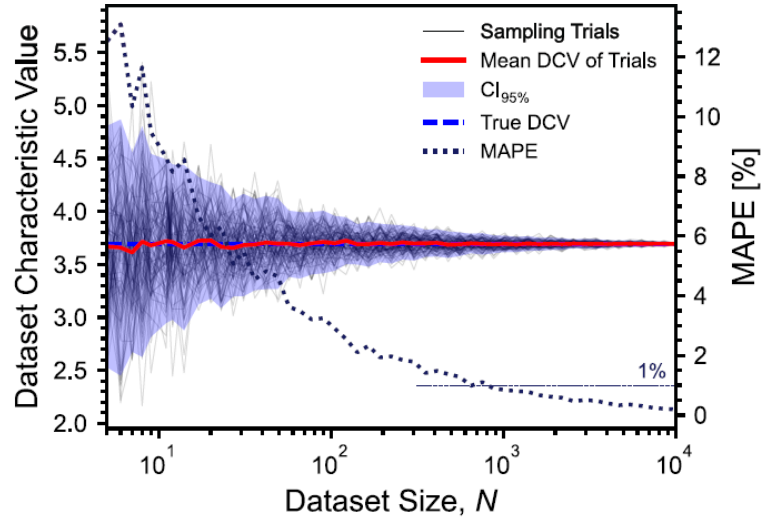
Yang, Kühn, …Gutfleisch, Zhang, Xu. http://arxiv.org/abs/2408.03198

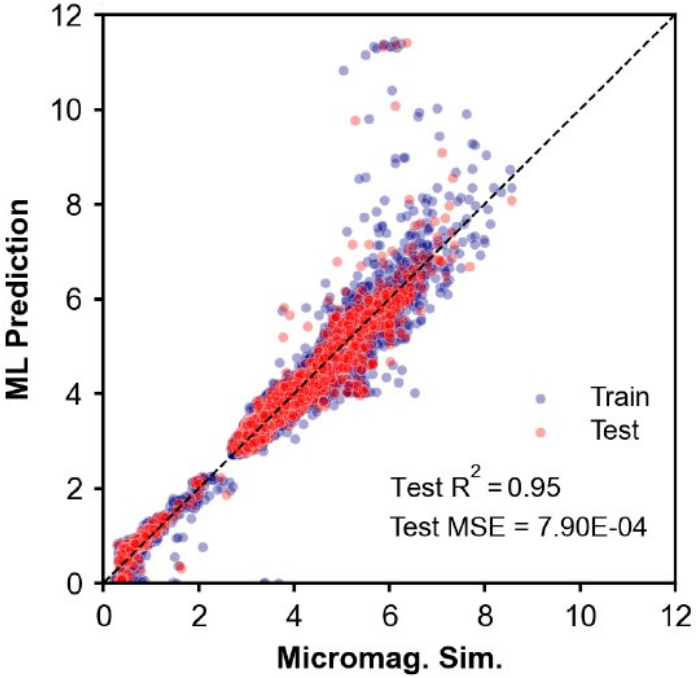
Machine learning of high-throughput micromagnetic simulation



Yang, Kühn, …Gutfleisch, Zhang, Xu. http://arxiv.org/abs/2408.03198





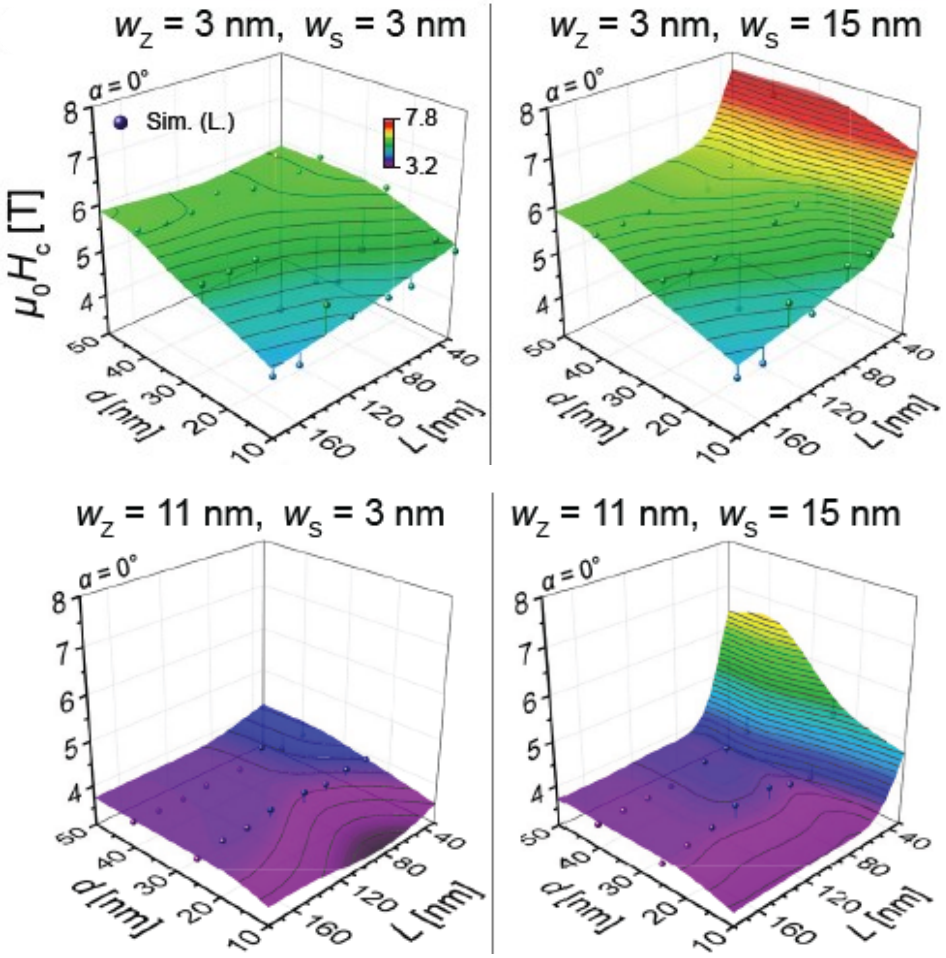


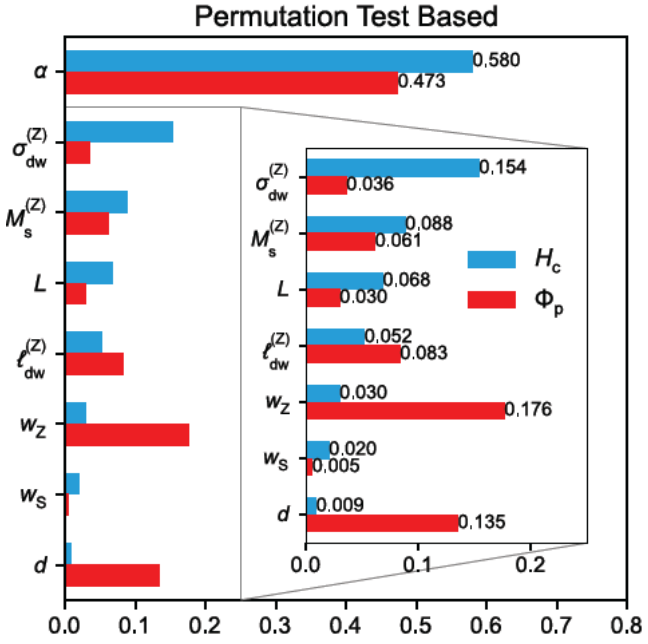
MECHANICS of FUNCTIONAL MATERIALS

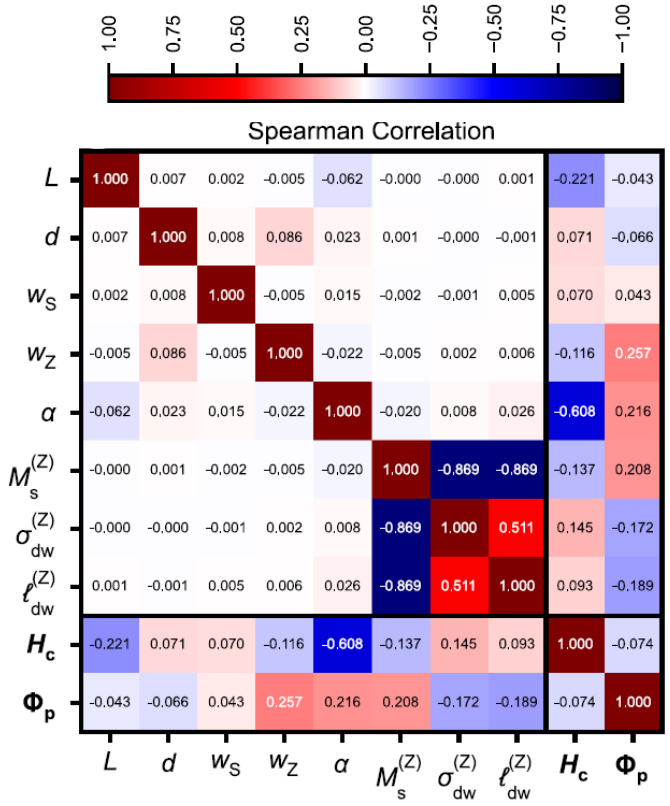
Forword prediction neural network (f-NN)

$$\mathcal{L} = \min_{\omega} \frac{1}{n} \sum^{n} \| \mathcal{F}_{\omega}[\boldsymbol{X}] - \boldsymbol{\Upsilon} \|^2$$

$$\boldsymbol{\Upsilon} = \{H_c, \Phi_p\} \quad \boldsymbol{X} = \{L, d, w_S, w_Z, \alpha, M_s^{(Z)}, \sigma_{dw}^{(Z)}, l_{dw}^{(Z)}\}$$

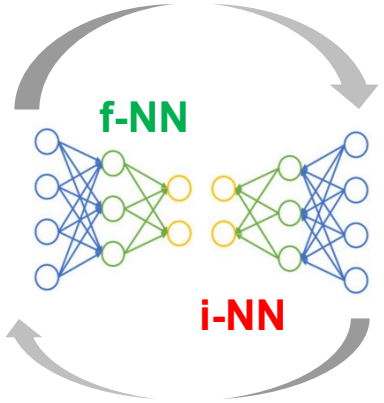


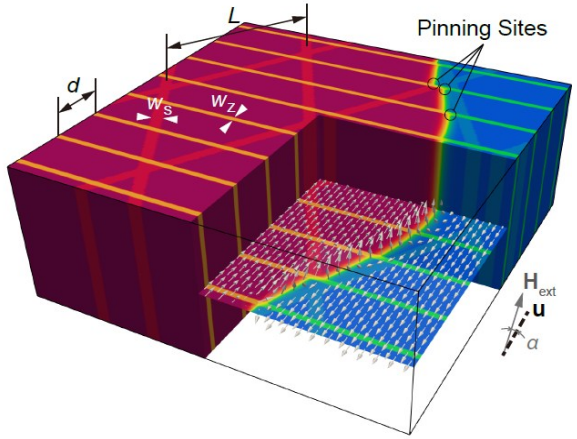


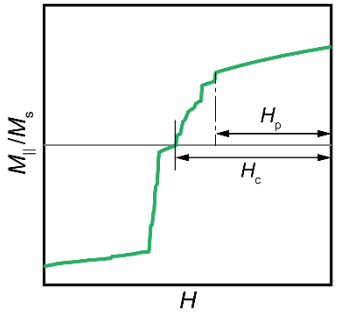


MECHANICS of FUNCTIONAL MATERIALS

DATA-DRIVEN MATERIALS RESEARCH
INVERSE DESIGN







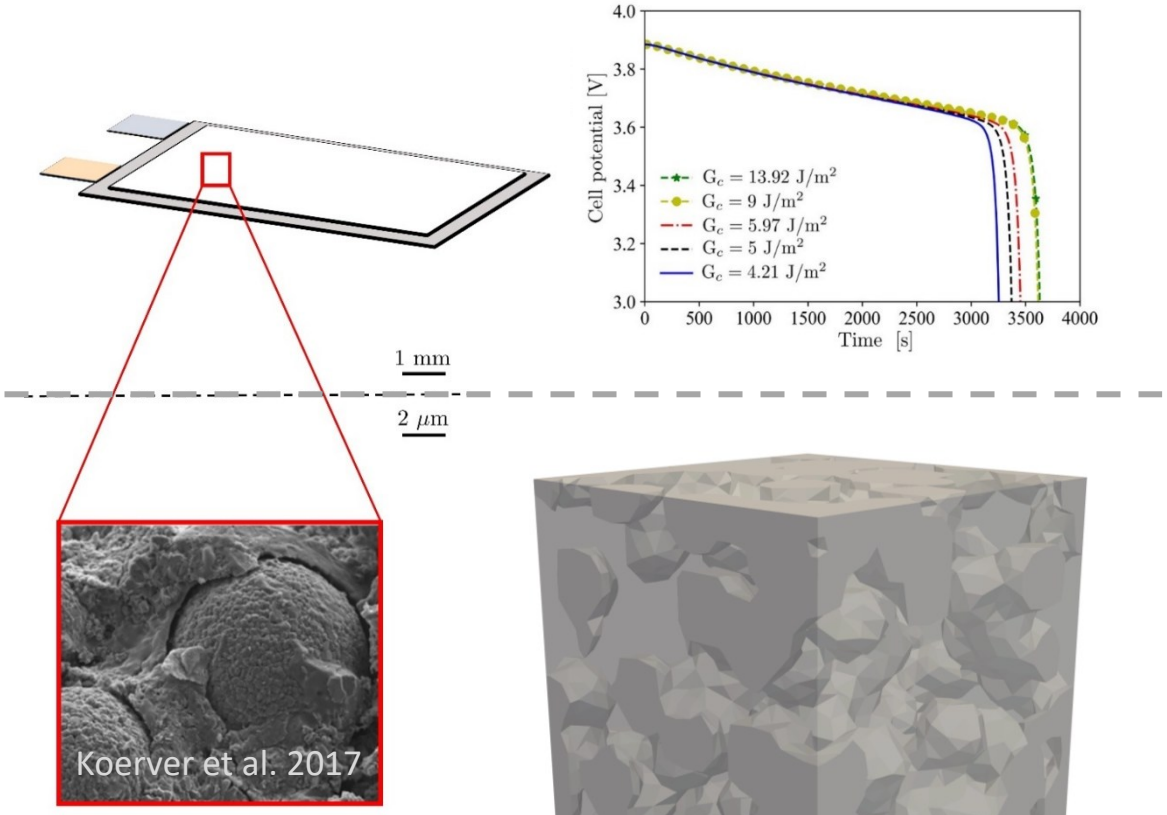
$$\mathcal{L} = \min_{v} \frac{1}{n} \sum^{n} (\lambda \underbrace{\|\mathcal{I}_v[Y] - X\|^2}_{\text{inverse pred.}} + \underbrace{\|\mathcal{F}_\omega\,[\mathcal{I}_v[Y]] - Y\|^2}_{\text{forward verif.}} \quad \text{with} \quad \lambda \geq 0$$

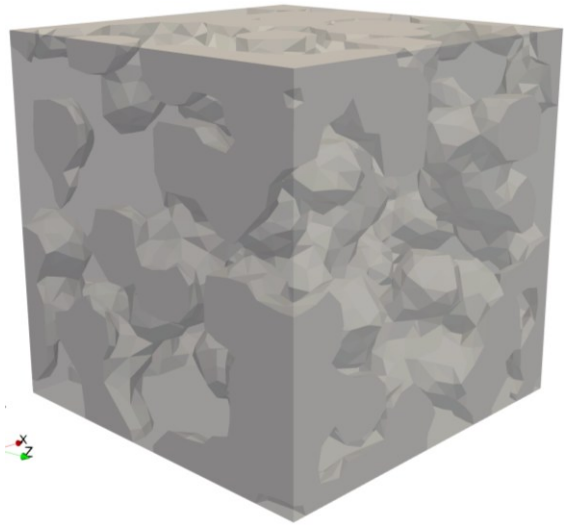
Queried Properties (*i*-NN iutput)		Designed Nanostructural Parameters (*i*-NN output)				Forward Cross-c
$\mu_0 H_c^{\mathrm{Q}}$ (T)	$\Phi_{\mathrm{P}}^{\mathrm{Q}}$	L (nm)	d (nm)	w_{S} (nm)	w_{Z} (nm)	$\mu_0 H_c^{\mathrm{I}}$ (T)
7.0000	0.0500	10.5512	34.4364	7.2424	4.2517	7.0060
5.0000	0.0100	93.6572	21.5065	6.3650	9.6443	5.0007
4.0000	0.0000	120.7757	13.0958	6.5392	12.0948	4.149

Yang et al. npj Comput. Mater. (under review)



Motivation: Example 4 Speed up simulations ?

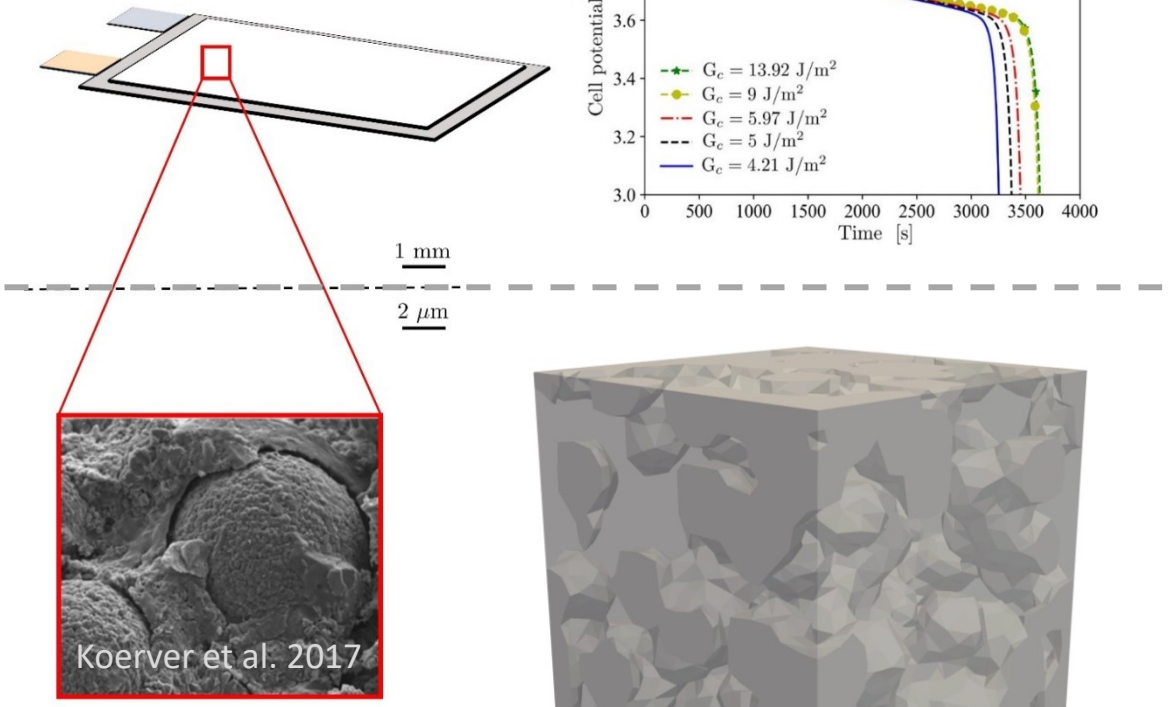


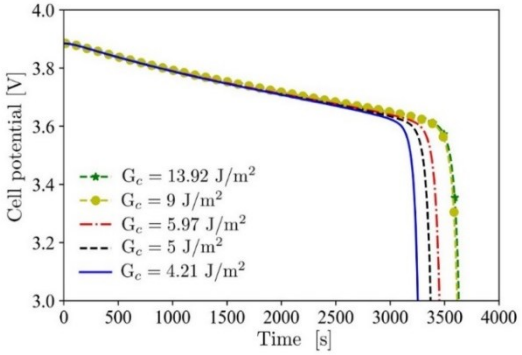


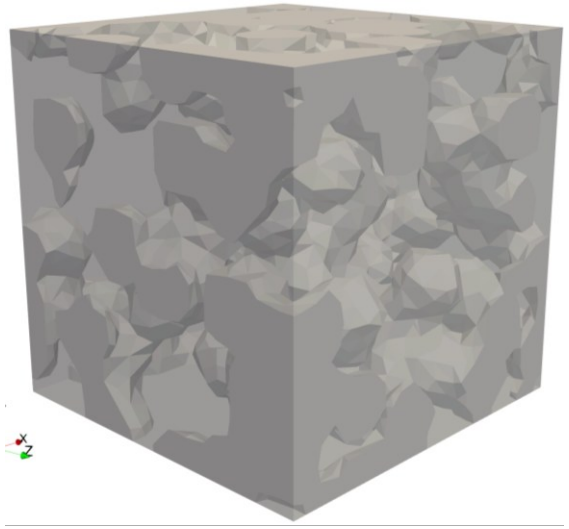
One FE^2 calculation of one charge-discharge cycle takes about 4 days in HPC

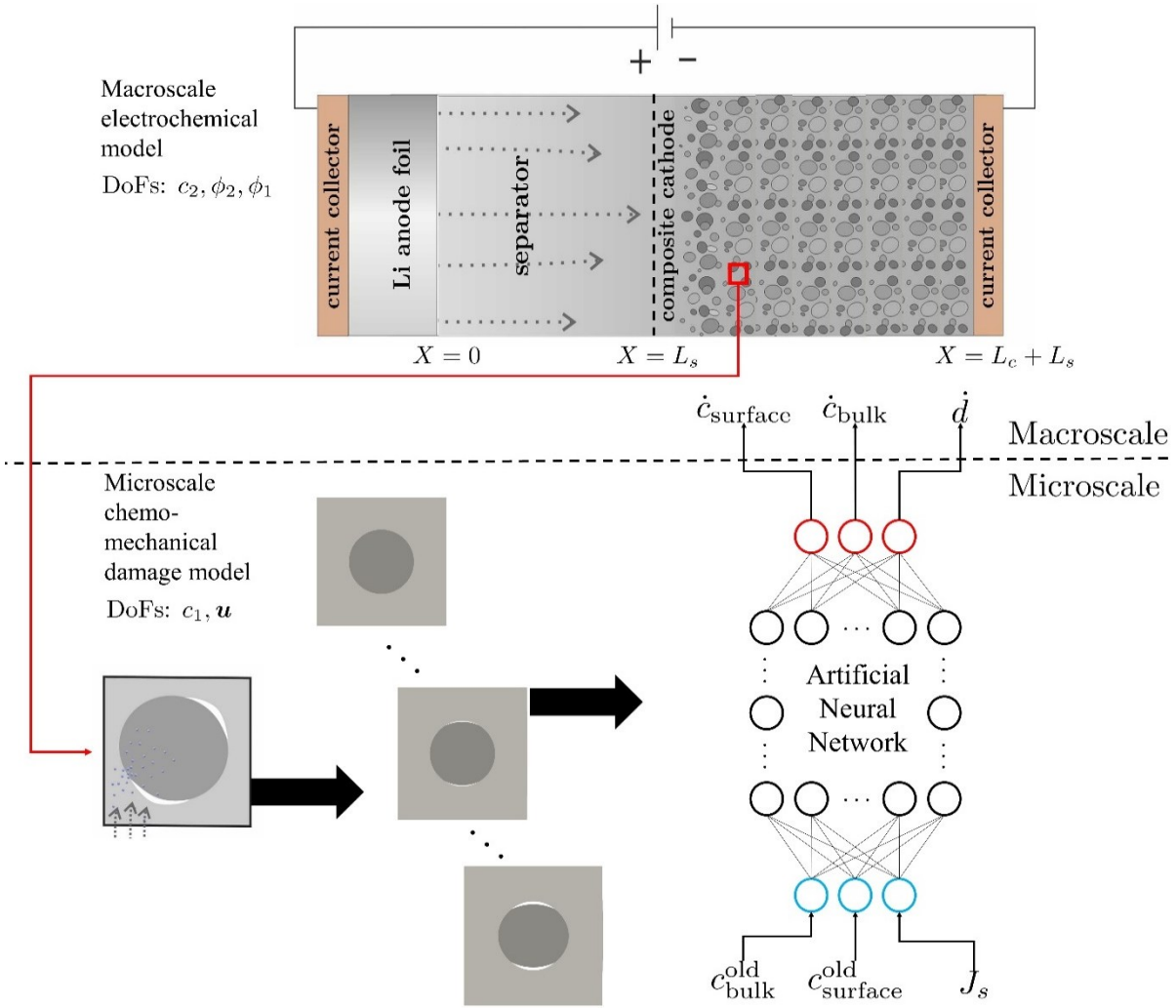
→ Can we speed up the simulations by ML?

FE^2 vs. FE+NN

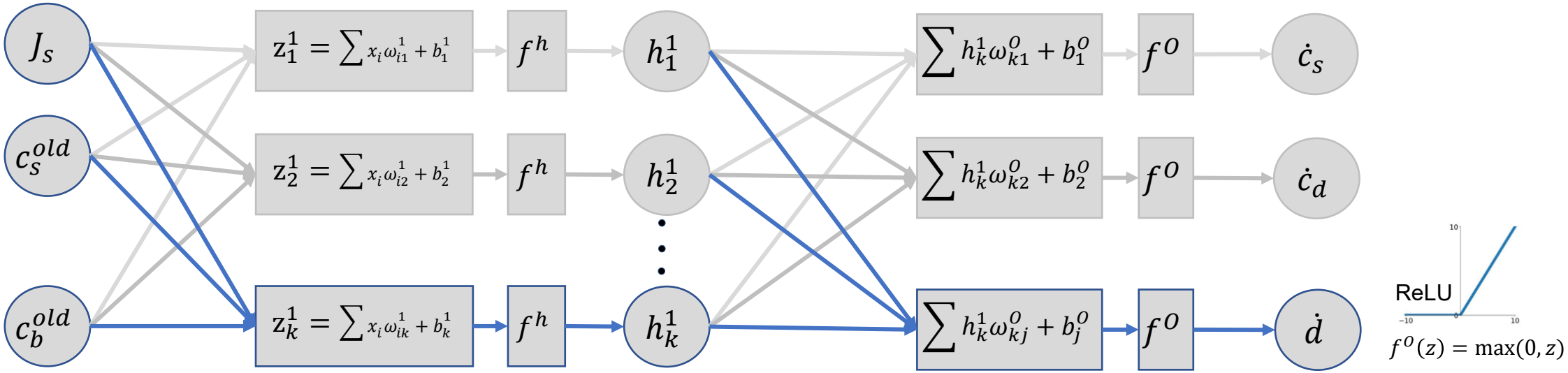

1 mm
2 μm
Koerver et al. 2017


Cell potential [V]
Time [s]
$G_c = 13.92$ J/m^2
$G_c = 9$ J/m^2
$G_c = 5.97$ J/m^2
$G_c = 5$ J/m^2
$G_c = 4.21$ J/m^2




Macroscale electrochemical model
DoFs: c_2, ϕ_2, ϕ_1
current collector
Li anode foil
separator
composite cathode
current collector
$X = 0$
$X = L_s$
$X = L_c + L_s$
$\dot{c}_{surface}$
$\dot{c}_{bulk}$
$\dot{d}$
Macroscale
Microscale
Microscale chemo-mechanical damage model
DoFs: $c_1, \boldsymbol{u}$
Artificial Neural Network
c^{old}_{bulk}
$c^{old}_{surface}$
j_s

NN for particle simulation



$$h_k^1 = f^h(z_k^1),\ z_k^1 = \sum_i x_i \omega_{ik}^1 + b_k^1$$

$$y_j = f^O(z_j^O),\ z_j^O = \sum_k h_k^1 \omega_{kj}^O + b_j^O$$

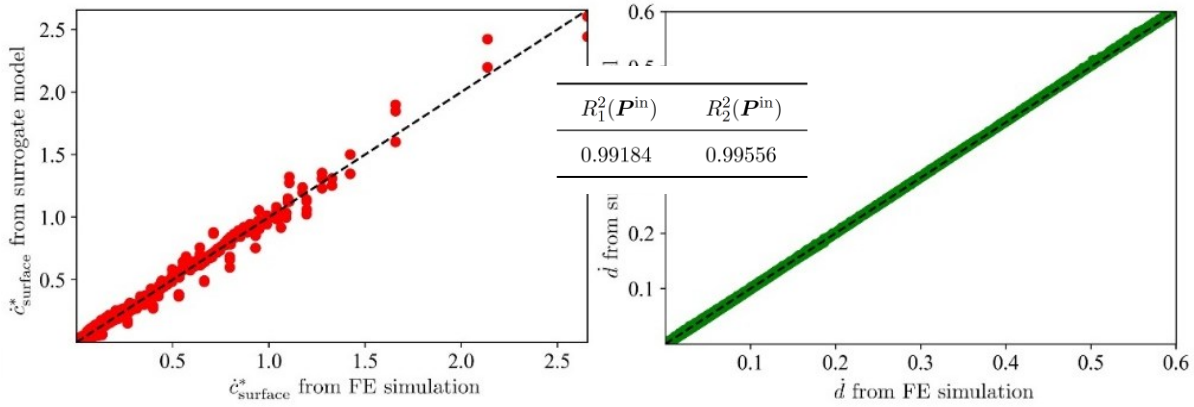
- 1 hidden layer of nodes 128, 2048, 3072, respectively
- $G_c = [4.2, 5.9, 13.9]\ \mathrm{J/m^2},\ \ J_s = [2.8\mathrm{e}-7 \sim 8.3\mathrm{e}2]\ \mathrm{mol/(m^2.s)}$
- 12304 data sets (training 80%, test 10%, validation 10%)
- PyTorch, C++

Loss function: MSE $\|y_j - y_j^*\|$

Model parameters: $\omega_{ik}^1, b_k^1;\ \omega_{qj}^O, b_j^O$

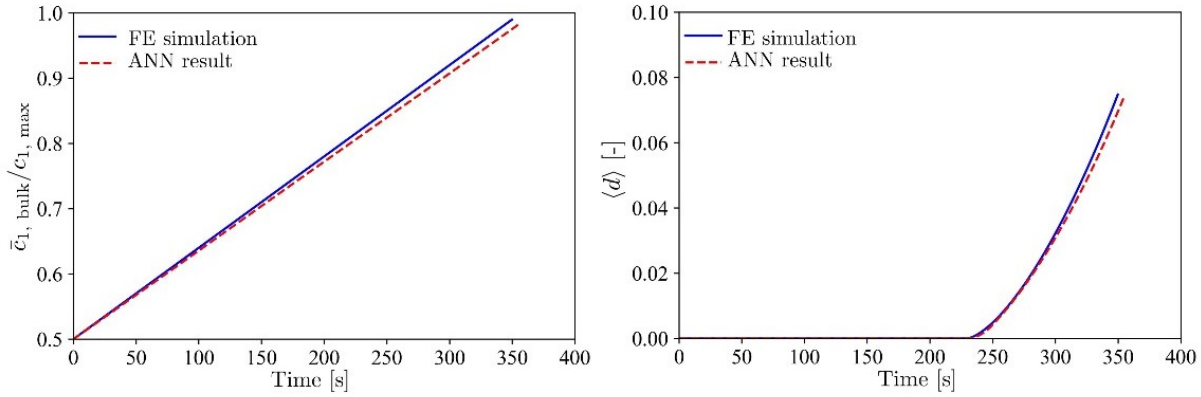
Hyperparameters: layer and node numbers

Performance of the trained NN model



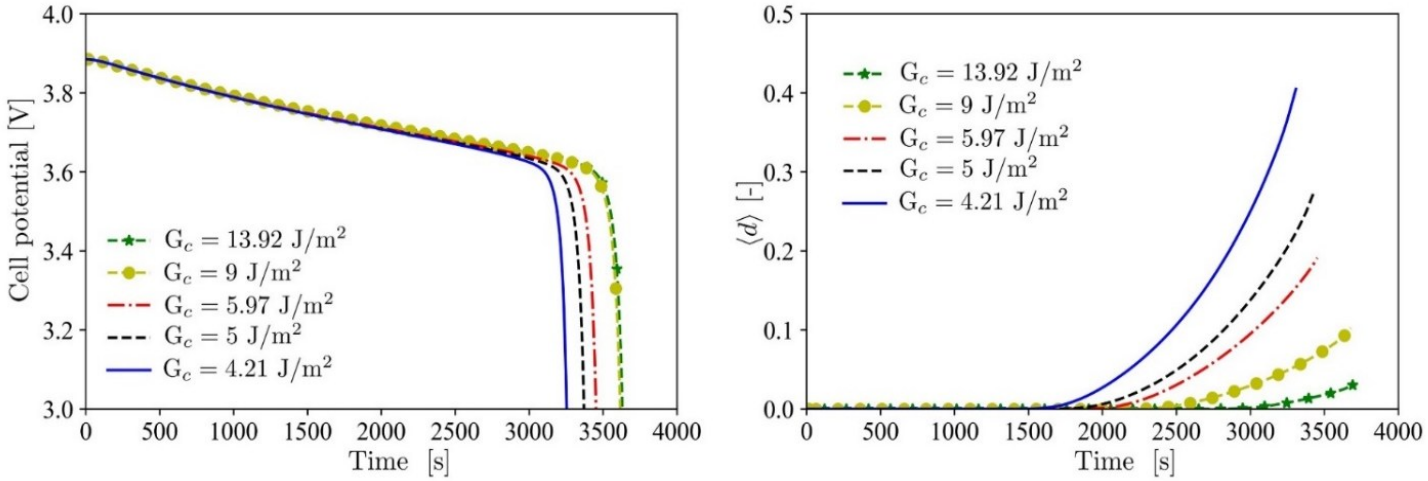
$R_1^2(\boldsymbol{P}^{\text{in}})$	$R_2^2(\boldsymbol{P}^{\text{in}})$
0.99184	0.99556

Unseen case 2: G_c =10J/m^2, J_s=183 mol/(m^2.s)



- Offline training of the NN model for the chemomechanical damage model on the particle level is high.
- Soon after the ANN is trained, the online access to the NN model in the FE+NN simulations is extremely efficient.
- With increasing data, the performance of the NN can grow.

FE+NN multiscale simulations



on the same machine:

4 days for FE² vs. 20 minutes for FE+NN

Asheri, Rezaei, Xu, *Comp. Mat. Sci., 226, 112186, 2023*

MECHANICS of FUNCTIONAL MATERIALS

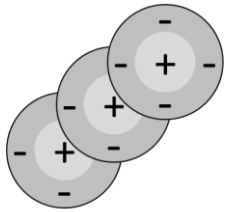
TECHNISCHE UNIVERSITÄT DARMSTADT

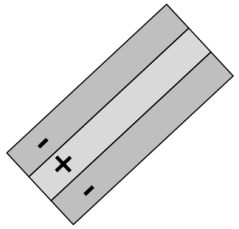
Convolutional Neural Network (CNN)

MECHANICS of FUNCTIONAL MATERIALS

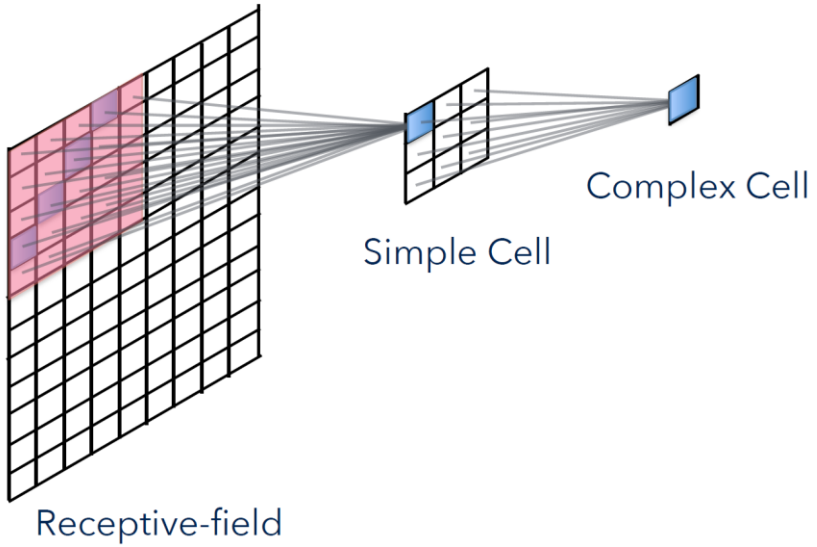
Motivation

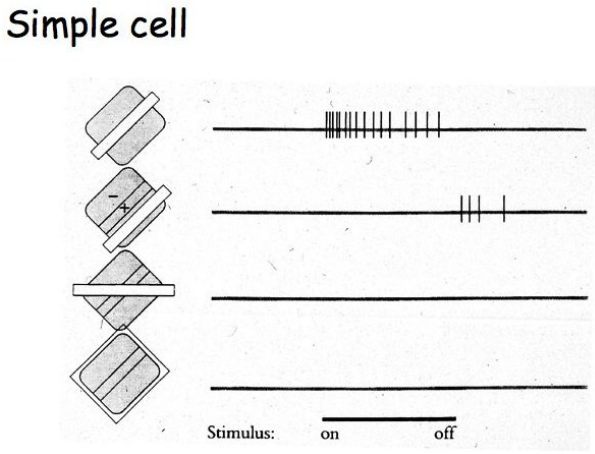
Simple cells & complex cells in our primary visual area (V1)





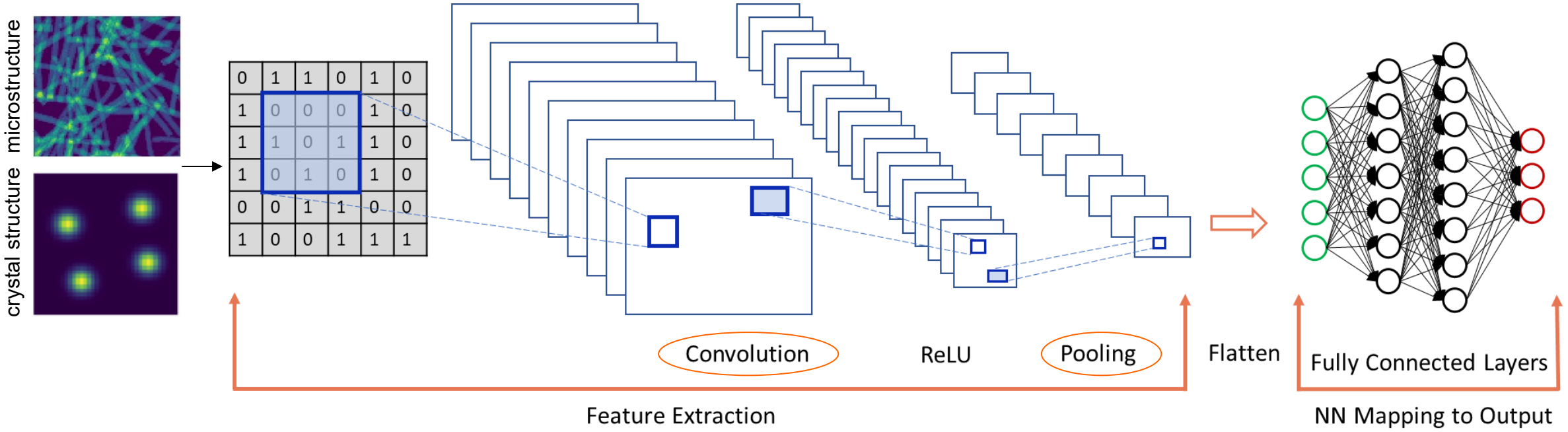
- Simple cells are formed by receive fields lined up.(orientation selective)
- Complex cells receive inputs from a number of simple cells. (orientation selective but only sensitive to cumulative light in that direction)





https://www.cns.nyu.edu/~david/courses/perception/lecturenotes/V1/lgn-V1.html

Typical Structure of a CNN



Variations:

- Multiple Convolution layers in sequence are possible.
- The package of Conv + ReLU + Pooling can also be repeated.
- Output can be regression (z.B. microstructure-property) or classification via softmax (crystal-phase)

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1, \dots, W$

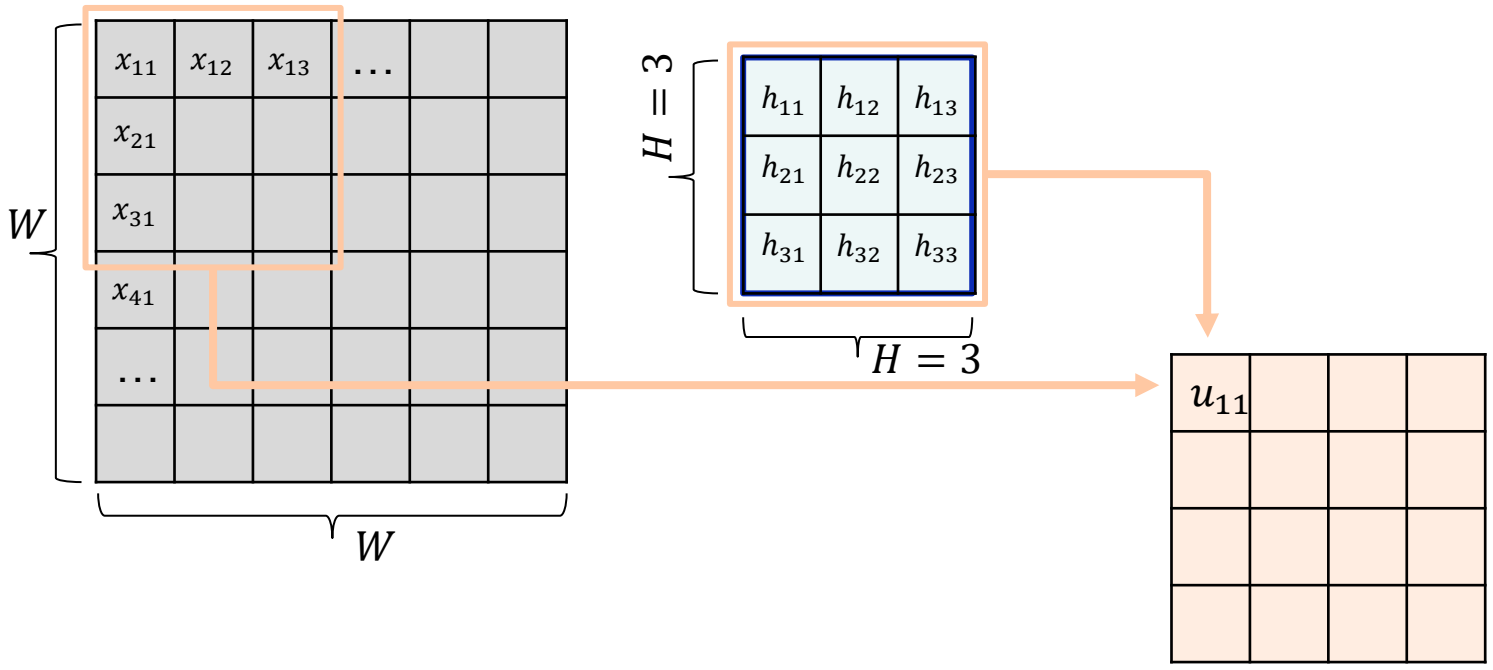
Kernel or Filter

$h_{pq} \quad p,q = 1, \dots, H$

Convolution

$u_{ij} \quad i,j = 1, \dots, H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{11} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p,q} h_{pq}$$
$$= x_{11}h_{11} + x_{12}h_{12} + x_{13}h_{13}$$
$$+ x_{21}h_{21} + x_{22}h_{22} + x_{23}h_{23}$$
$$+ x_{31}h_{31} + x_{32}h_{32} + x_{33}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

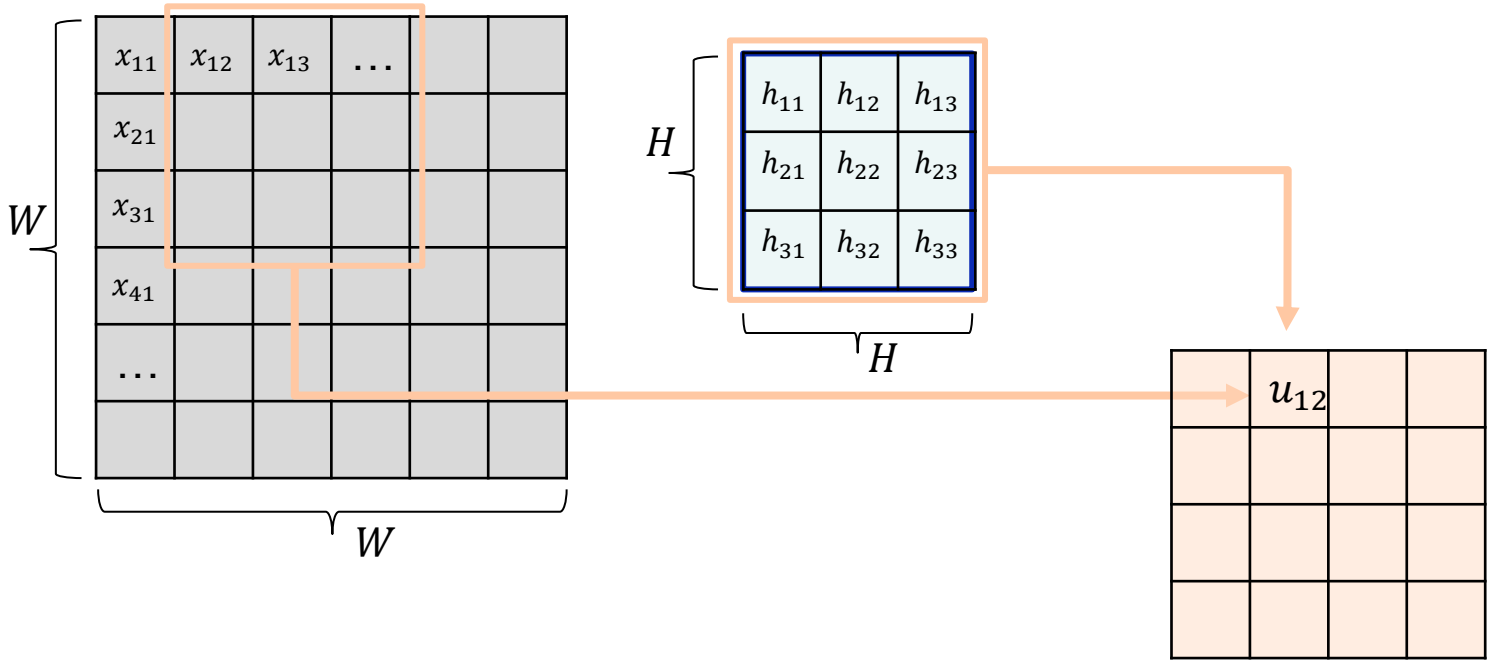
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{12} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p,q+1} h_{pq}$$
$$= x_{12}h_{11} + x_{13}h_{12} + x_{14}h_{13}$$
$$+ x_{22}h_{21} + x_{23}h_{22} + x_{24}h_{23}$$
$$+ x_{32}h_{31} + x_{33}h_{32} + x_{34}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

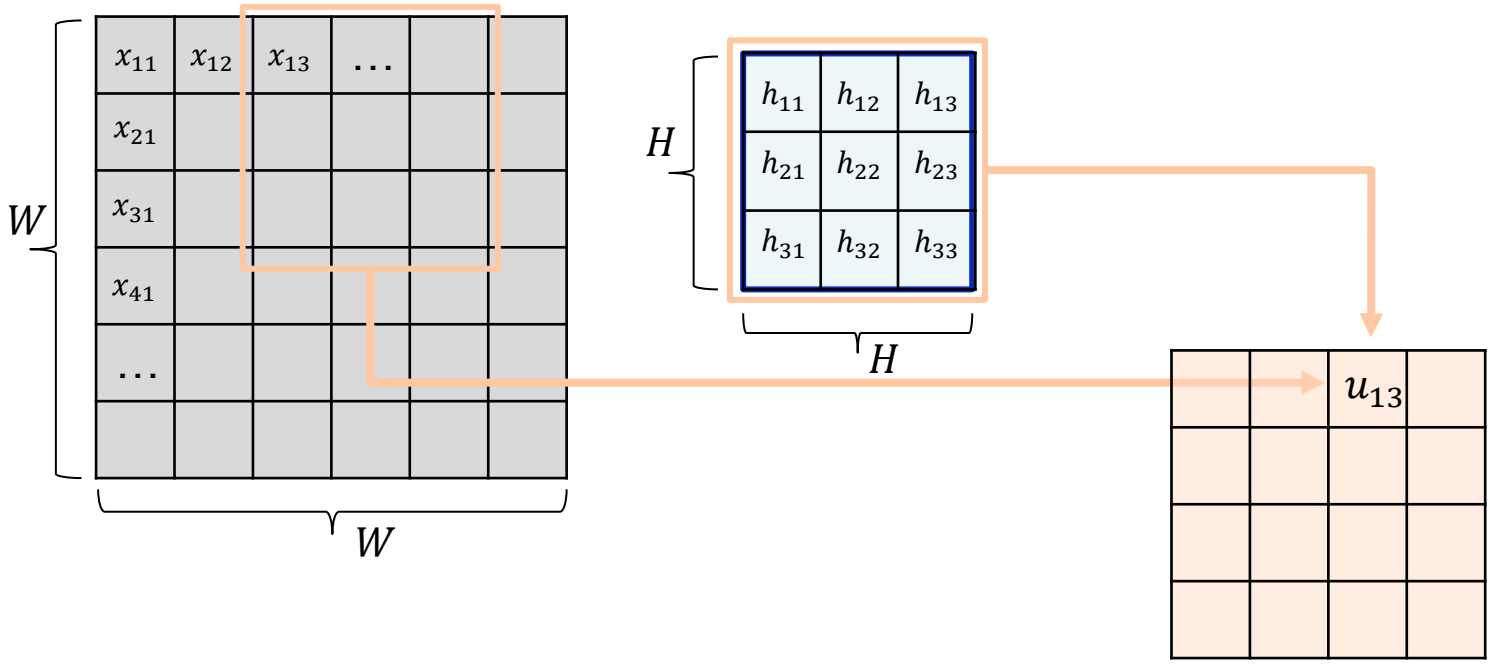
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{13} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p,q+2} h_{pq}$$
$$= x_{13}h_{11} + x_{14}h_{12} + x_{15}h_{13}$$
$$+ x_{23}h_{21} + x_{24}h_{22} + x_{25}h_{23}$$
$$+ x_{33}h_{31} + x_{34}h_{32} + x_{35}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

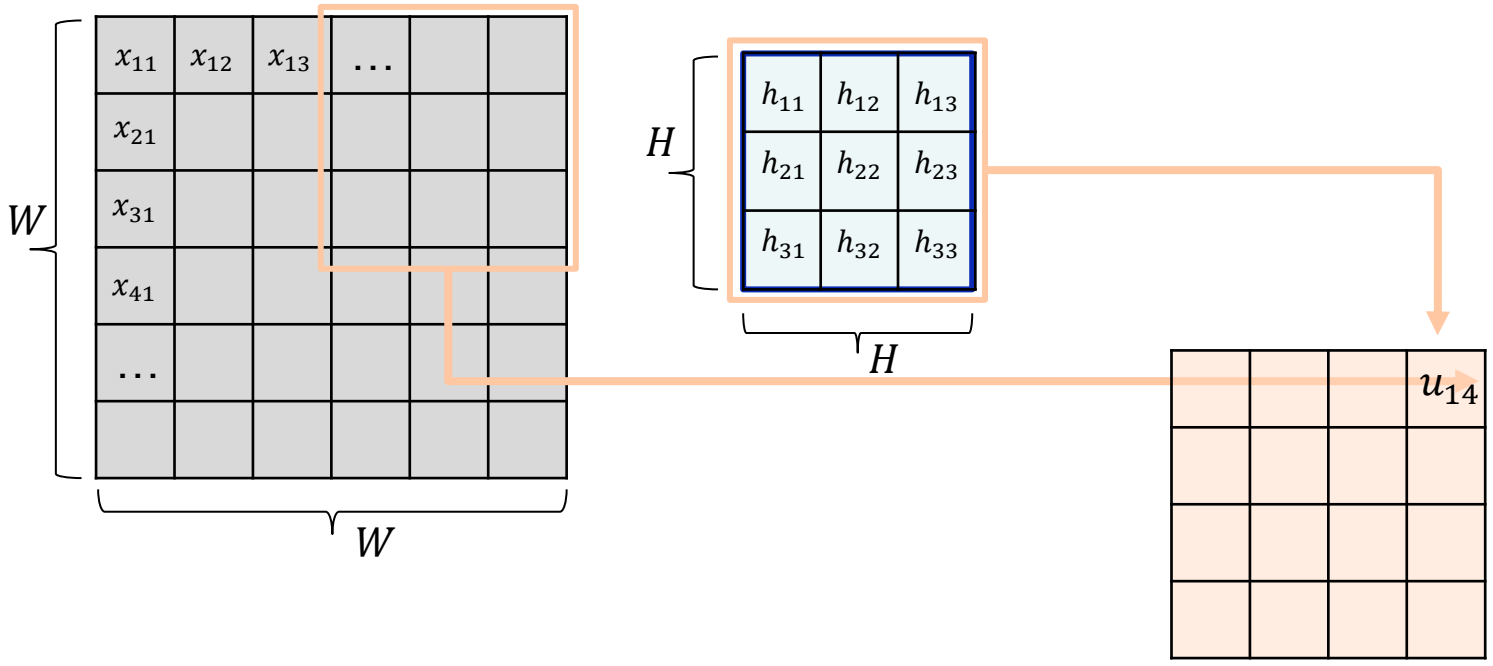
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{14} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p,q+3} h_{pq}$$
$$= x_{14}h_{11} + x_{15}h_{12} + x_{16}h_{13}$$
$$+ x_{24}h_{21} + x_{25}h_{22} + x_{26}h_{23}$$
$$+ x_{34}h_{31} + x_{35}h_{32} + x_{36}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

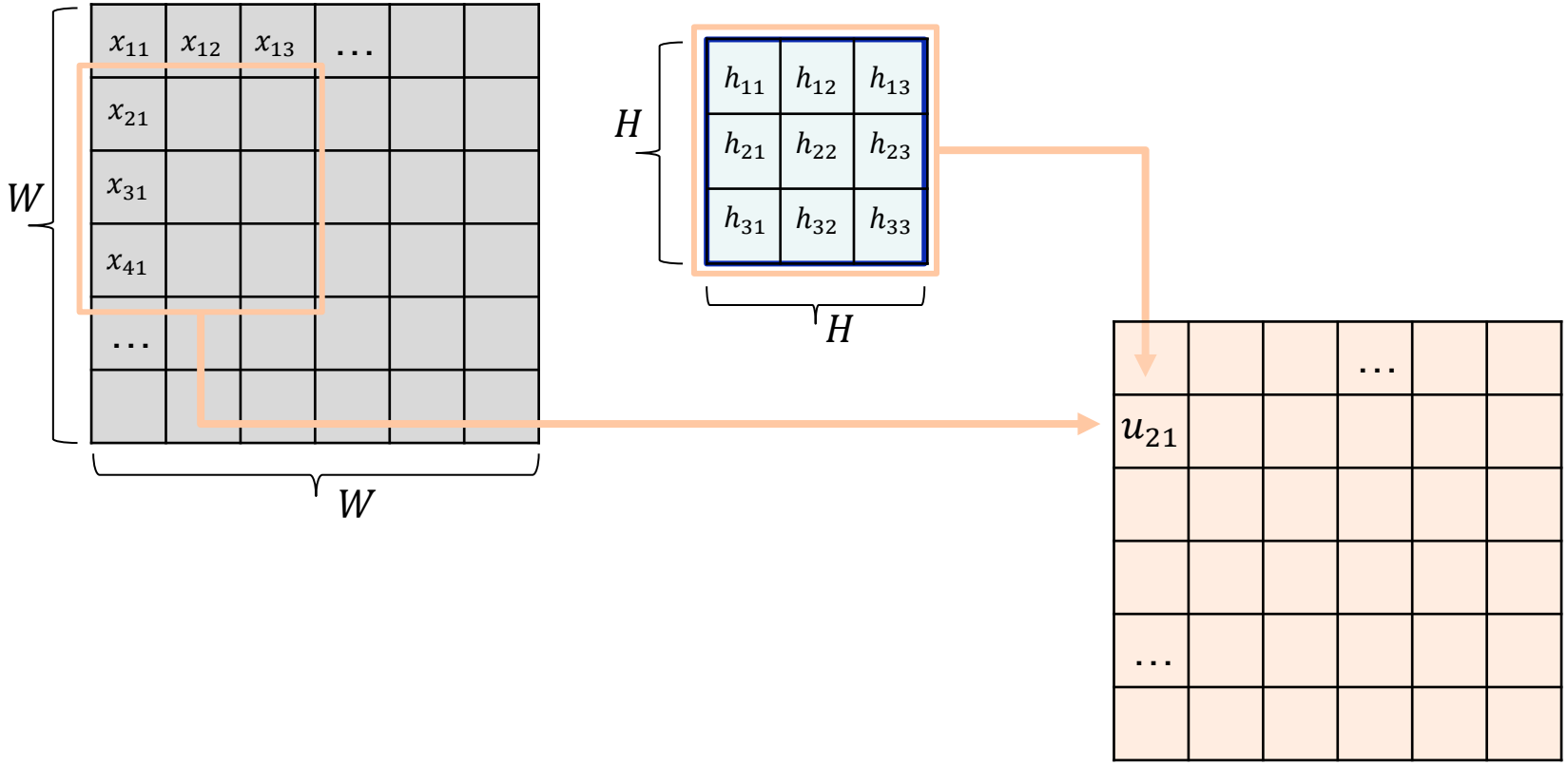
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{21} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p+1,q} h_{pq}$$
$$= x_{21}h_{11} + x_{22}h_{12} + x_{23}h_{13}$$
$$+ x_{31}h_{21} + x_{32}h_{22} + x_{33}h_{23}$$
$$+ x_{41}h_{31} + x_{42}h_{32} + x_{43}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

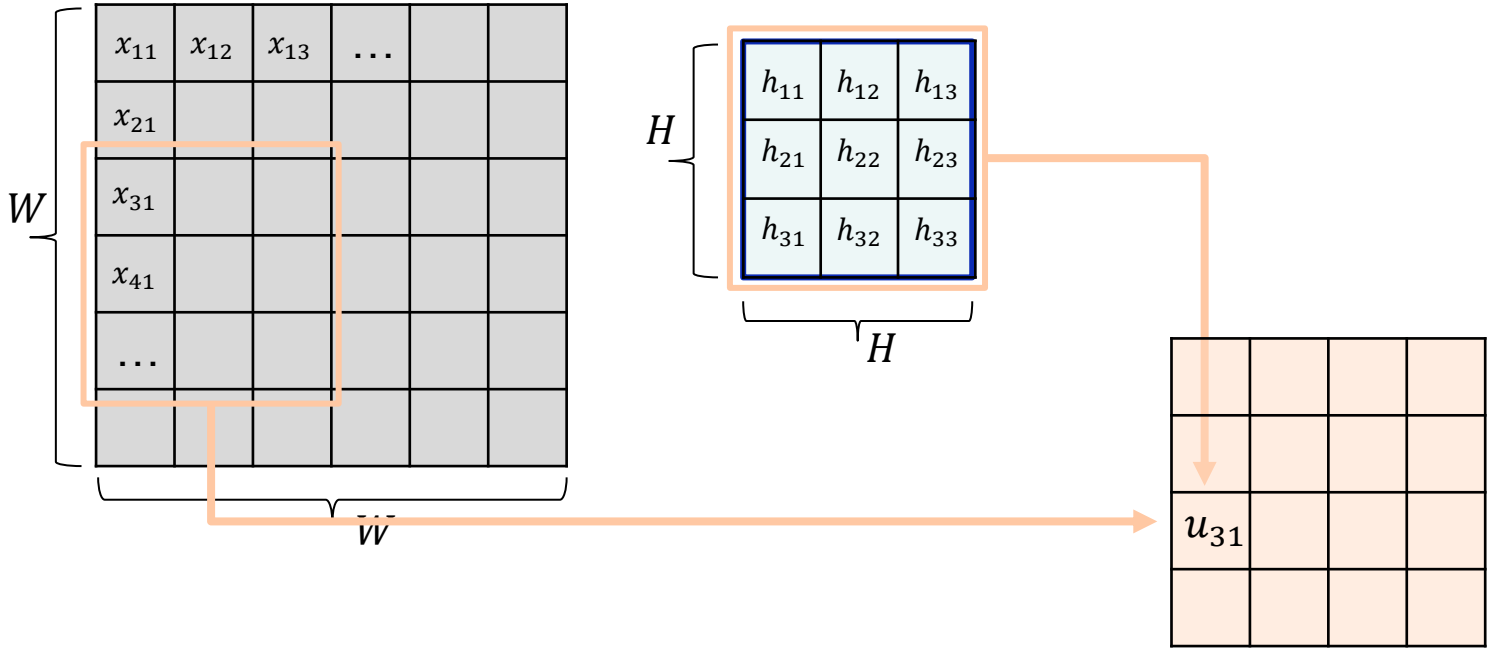
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{31} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p+2,q} h_{pq}$$
$$= x_{31}h_{11} + x_{32}h_{12} + x_{33}h_{13}$$
$$+ x_{41}h_{21} + x_{42}h_{22} + x_{43}h_{23}$$
$$+ x_{51}h_{31} + x_{52}h_{32} + x_{53}h_{33}$$

Convolution: Definition

Input data

$x_{ij} \quad i,j = 1,\dots,W$

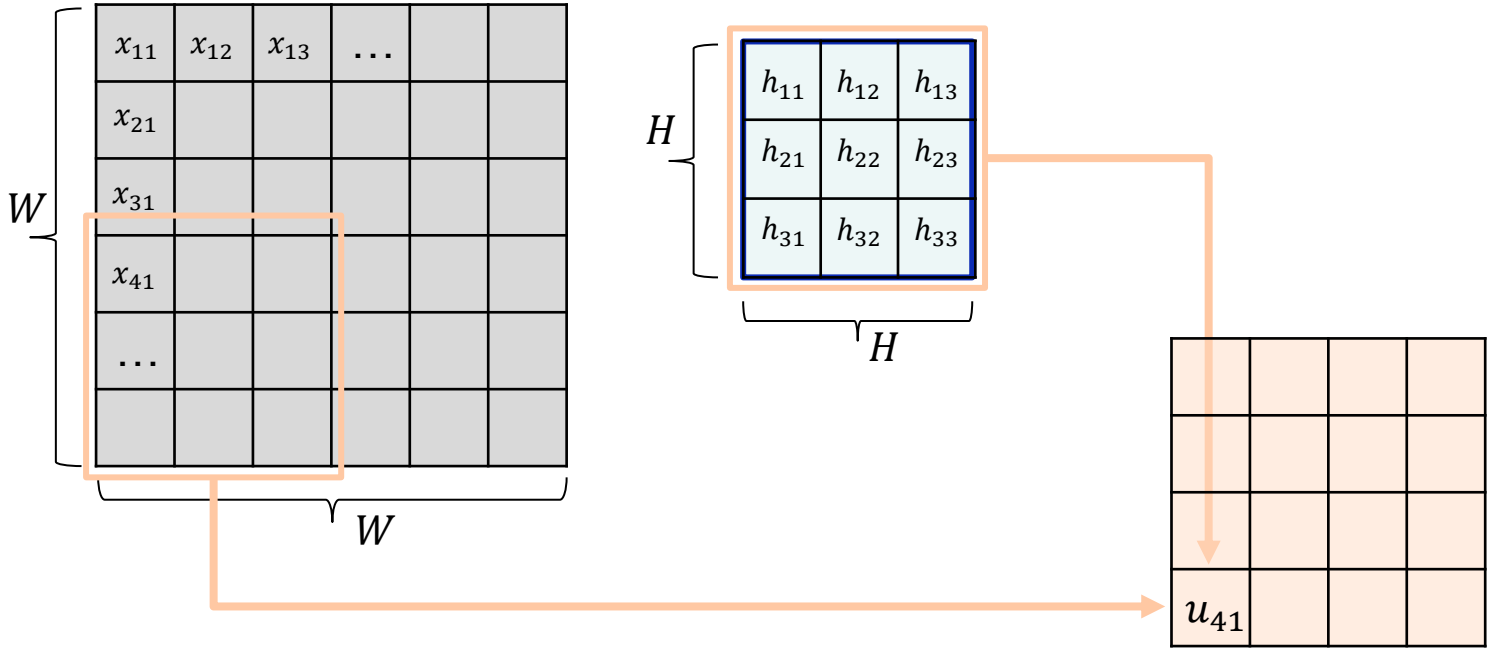
Kernel or Filter

$h_{pq} \quad p,q = 1,\dots,H$

Convolution

$u_{ij} \quad i,j = 1,\dots,H$

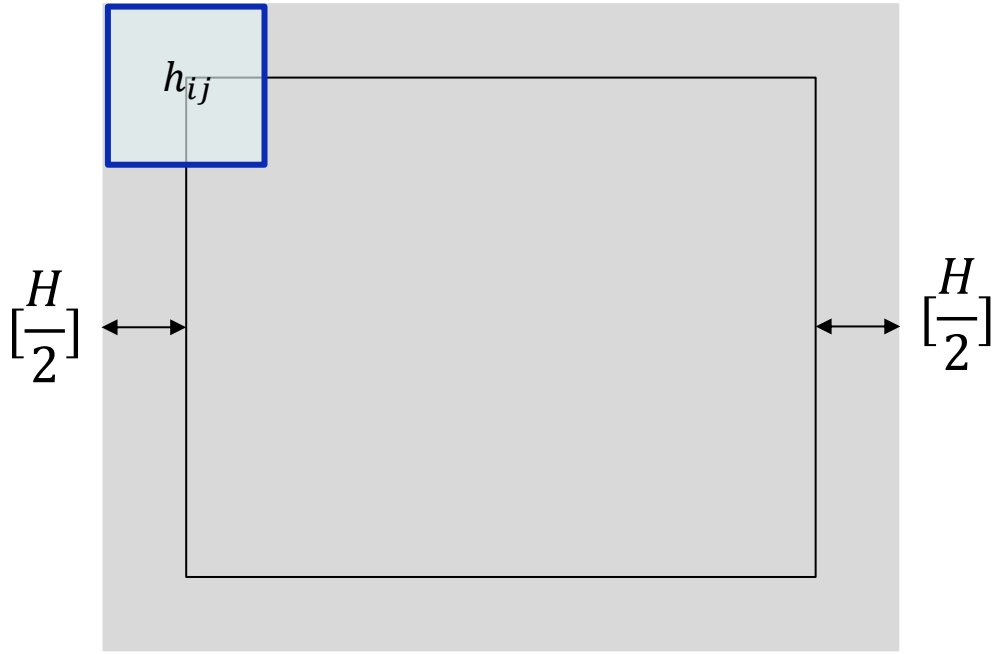
$$u_{ij} = \sum_p \sum_q x_{i+p-1,j+q-1} h_{pq} + b_{ij}$$



$$u_{41} = \sum_{p=1}^{H} \sum_{q=1}^{H} x_{p+3,q} h_{pq}$$
$$= x_{41}h_{11} + x_{42}h_{12} + x_{43}h_{13}$$
$$+ x_{51}h_{21} + x_{52}h_{22} + x_{53}h_{23}$$
$$+ x_{61}h_{31} + x_{62}h_{32} + x_{63}h_{33}$$

Convolution: Padding

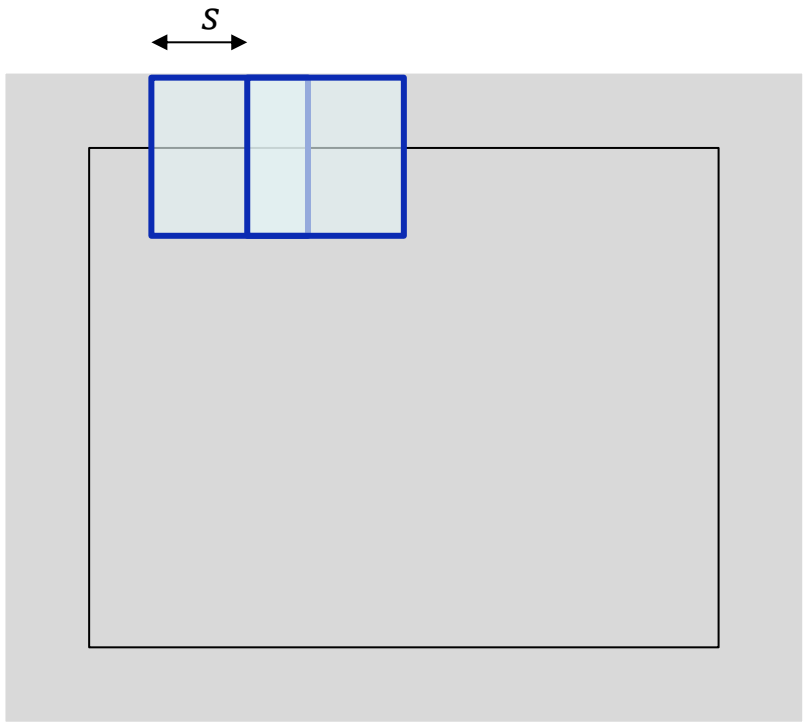
The width shrinks by $H-1$ after each convolution over the image. It limits the size of the number of the convolution layers and the chosed size of kernels. Therefore Padding strategies are proposed.



- Zero-padding: Adding implicit zeros before convolution keeps the representation size unchanged.
- Filling the additional margin by pixel values of most outside or other variants.

Convolution: Stride

For large size image, one may shift the kernel by a few pixels once, instead of one-by-one.



$$u_{ij} = \sum_p \sum_q x_{i*s+p-1,j*s+q-1} h_{pq}$$

s: stride

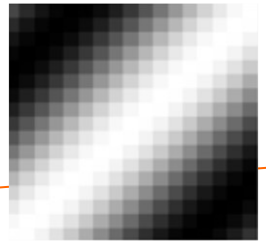
Output image size becomes then

$$\left(\left[\frac{W-1}{s}\right]+1\right) \text{X} \left(\left[\frac{W-1}{s}\right]+1\right)$$

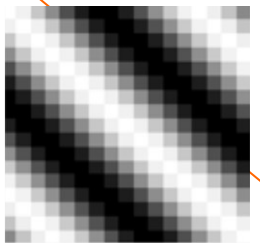
Convolution: Effect



Kernel 1



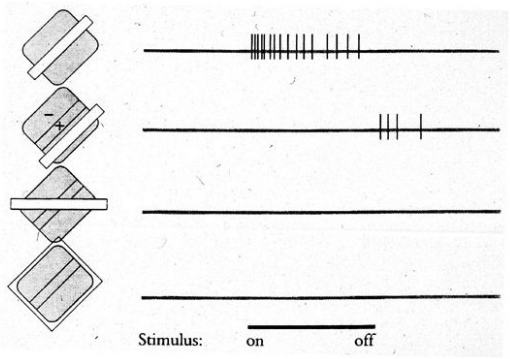
Kernel 2







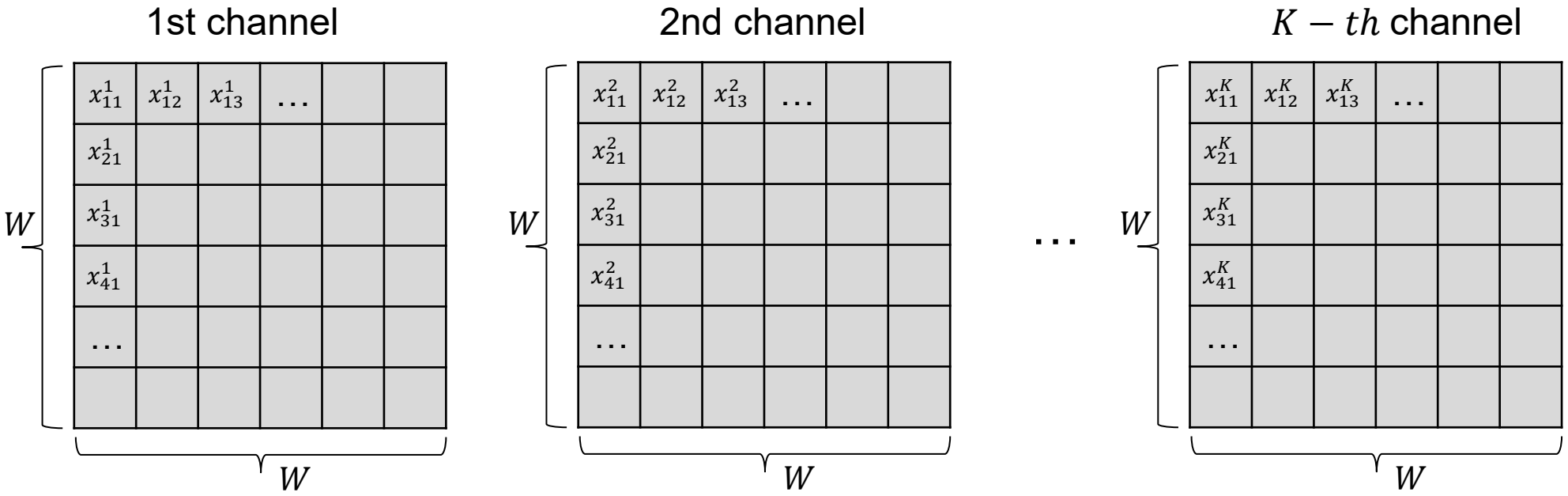
Simple cell



https://gist.github.com/matsuken92/5b78c792f2ab98576c5c

Convolution: multiple kernels and convolution layer

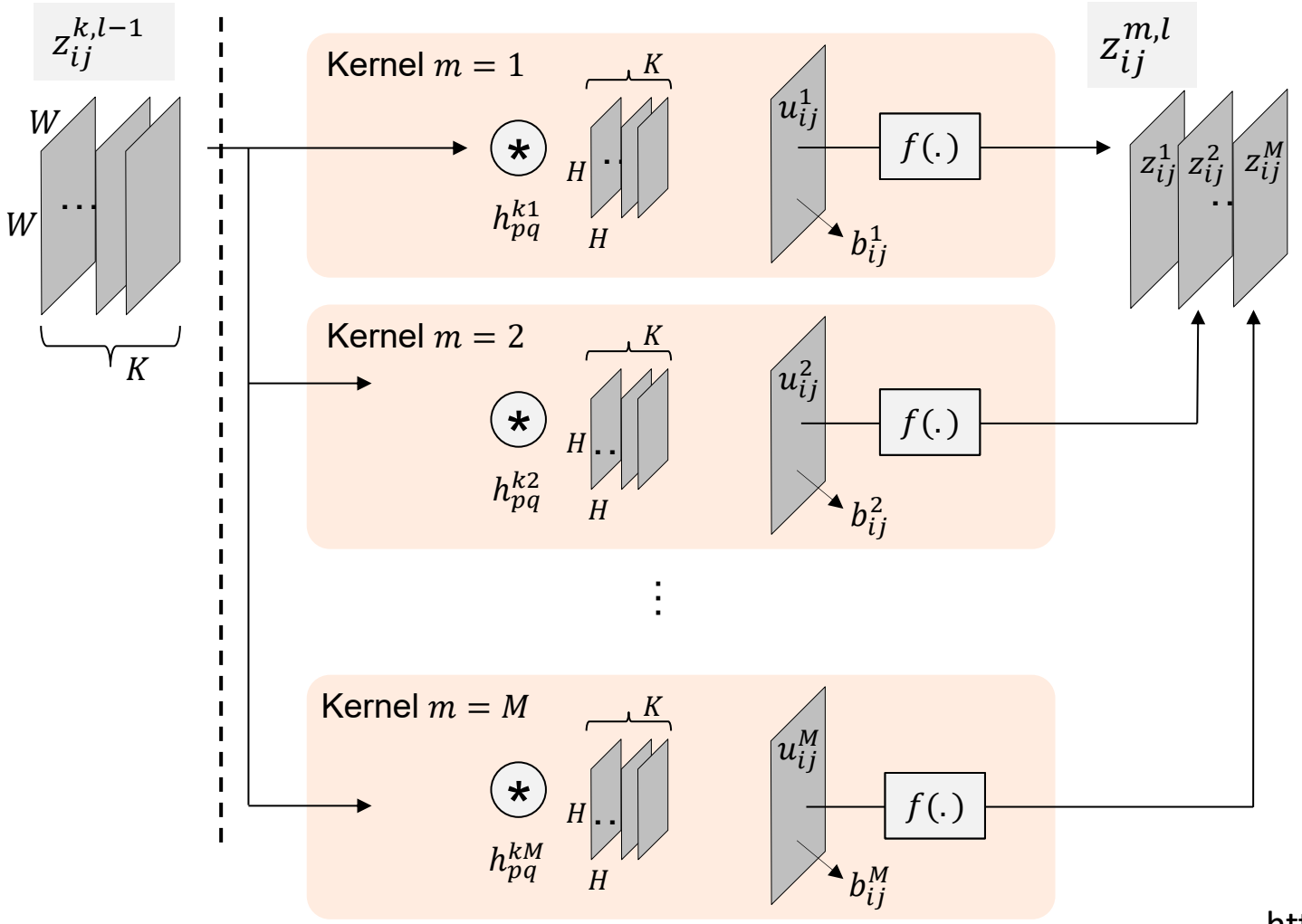
- K multi channel image (e.g. RGB)



The size of the image: W x W x K

- M parallel kernels: $h_{ij}^m \quad m = 1,2,\dots,M$

Convolution: multiple kernels and convolution layer



the $l-th$ convolution layer

$$u_{ij}^{m} = \sum_{k}^{K} \sum_{p}^{H} \sum_{q}^{H} z_{i+p,j+q}^{k,l-1} h_{pq}^{km} + b_{ij}^{m}$$

$$z_{ij}^{m,l} = f(u_{ij}^{m})$$

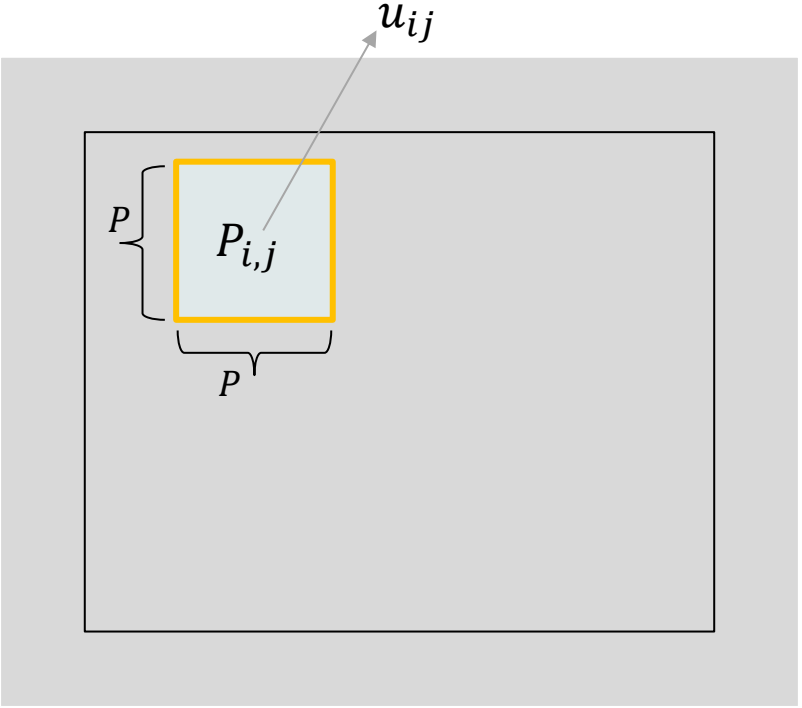
- $z_{ij}^{k,l-1}$: output values of the previous layer:
- $z_{ij}^{m,l}$: input values for next layer
- $f(.)$: activation function
- The size of the output: W x W x K.
- Gradient Descent based Backprop is also used to determine model parameters h_{pq}^{km}, b_{ij}^{m}.
- Hyperparameters: kernels, and kernel number m and kernel sizes.

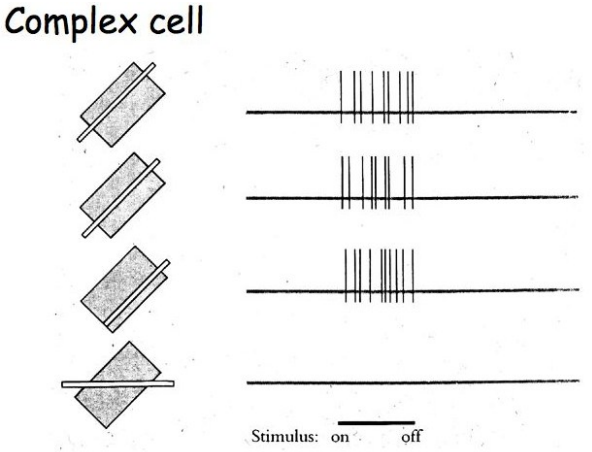
https://gist.github.com/matsuken92/5b78c792f2ab98576c5c

Pooling

A pooling function is designed to replace the output of the net at a certain location with a summary statistic of the nearby outputs.

Pooling can be independently applied or skipped for different channels, depending on the invariant features.





A pixel value u_{ij} is be obtained from the region $P_{i,j}$ of the size P x P, on which the pooling function works on at that moment.

Different type of pooling functions:

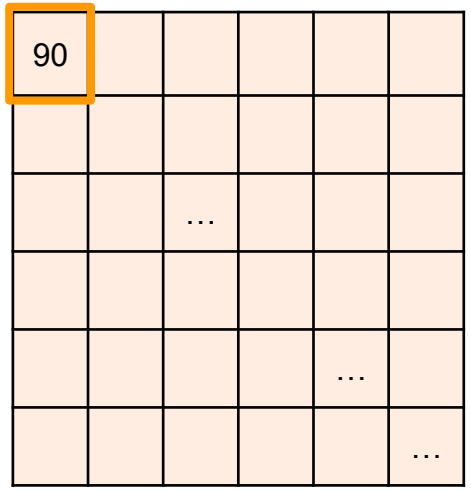
- Max pooling
- Average pooling
- Lp pooling

Pooling

$z_{ij}^{k,l}$ zero padding

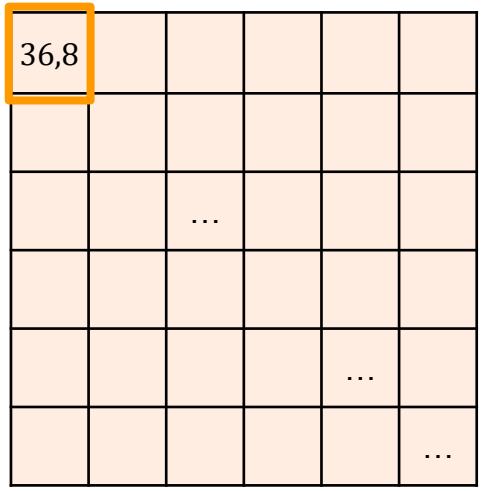
90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling



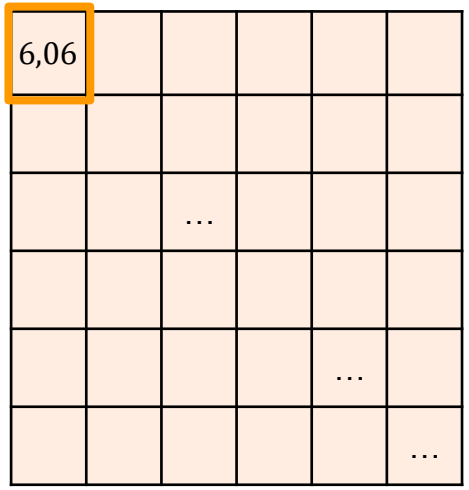
$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling



$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$



$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

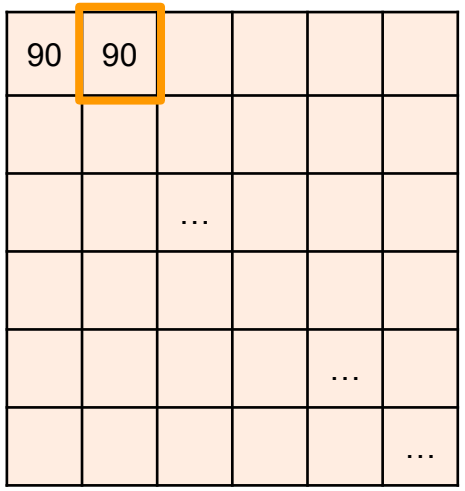
If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

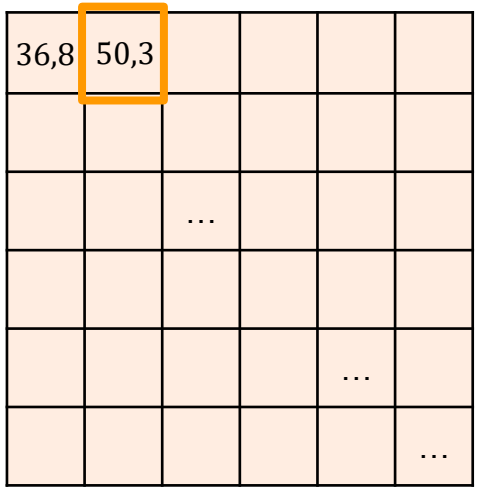
90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling



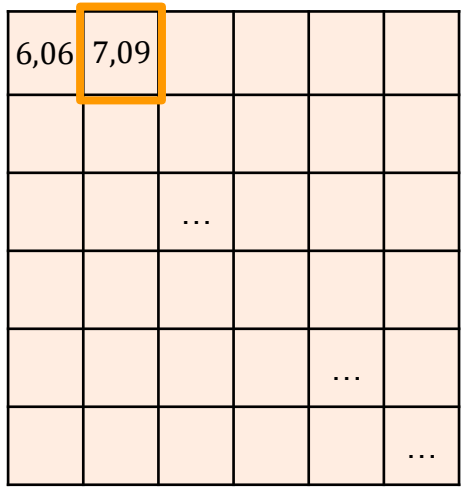
$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling



$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$



$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

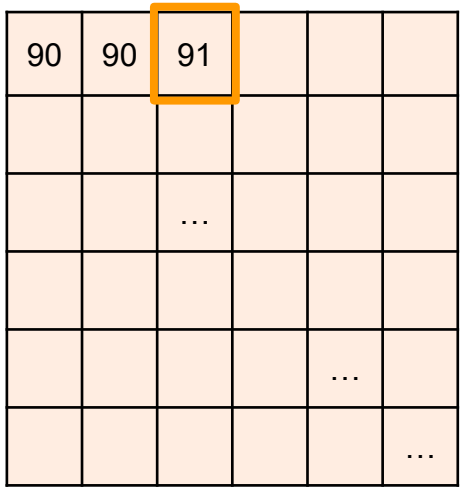
If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

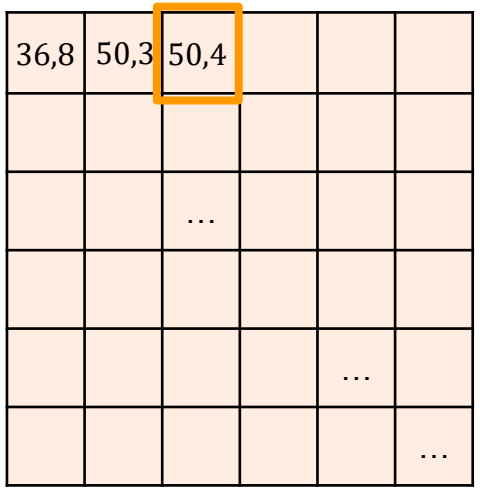
90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling



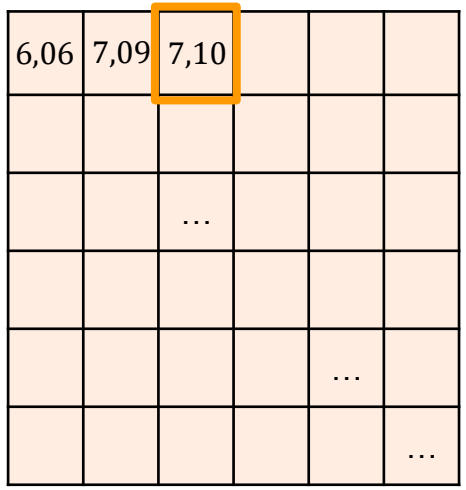
$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling



$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$



$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

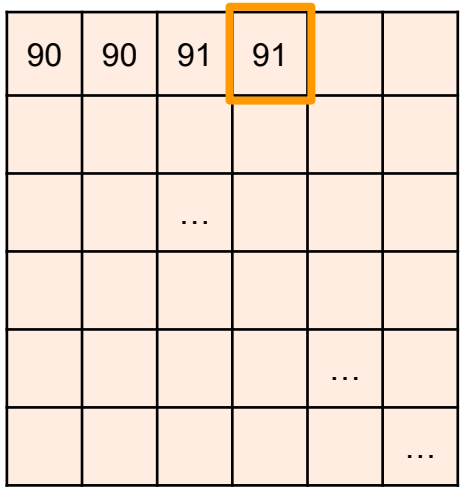
If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

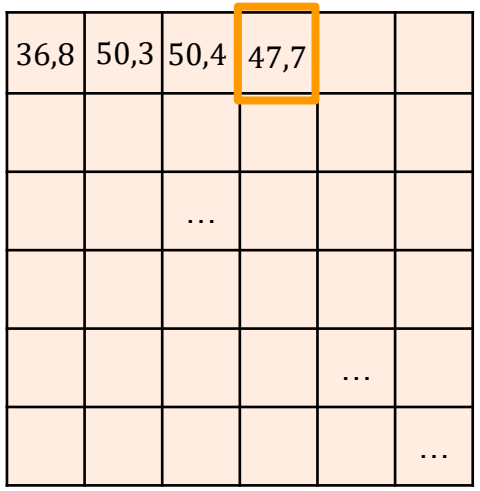
90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling



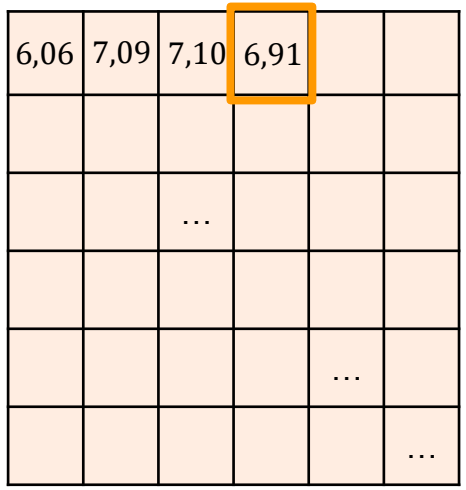
$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling



$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$



$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}\right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	
		…			
				…	
					…

Average pooling

36,8	50,3	50,4	47,7	47,7	
		…			
				…	
					…

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	
		…			
				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
		…			
				…	
					…

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
		…			
				…	
					…

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
		…			
				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
90					
		…			
				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
47,9					
		…			
				…	
					…

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
6,92					
		…			
				…	
					…

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}\right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
90					
99		…			
				…	
					…

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
47,9					
49,8		…			
				…	
					…

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
6,92					
7,06		…			
				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
90					
99		…			
99					
				…	
					…

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
47,9					
49,8		…			
47,1					
				…	
					…

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
6,92					
7,06		…			
6,86					
				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
90					
99		…			
99					
99				…	
					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
47,9					
49,8		…			
47,1					
46,5				…	
					…

$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
6,92					
7,06		…			
6,86					
6,82				…	
					…

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

$z_{ij}^{k,l}$ zero padding

90	80	86	91	82	83
83	78	36	83	52	36
43	57	27	97	80	87
88	99	96	58	85	46
43	94	84	37	44	54
59	36	42	42	72	46

Max pooling

90	90	91	91	91	83
90					
99		…			
99					
99				…	
94					…

Average pooling

36,8	50,3	50,4	47,7	47,7	28,1
47,9					
49,8		…			
47,1					
46,5				…	
25,7					…

Lp pooling $n = 2$

6,06	7,09	7,10	6,91	6,91	5,30
6,92					
7,06		…			
6,86					
6,82				…	
5,07					…

$$u_{ij}^{k,l} = \max_{p,q \in P_{i,j}} (z_{pq}^{k,l})$$

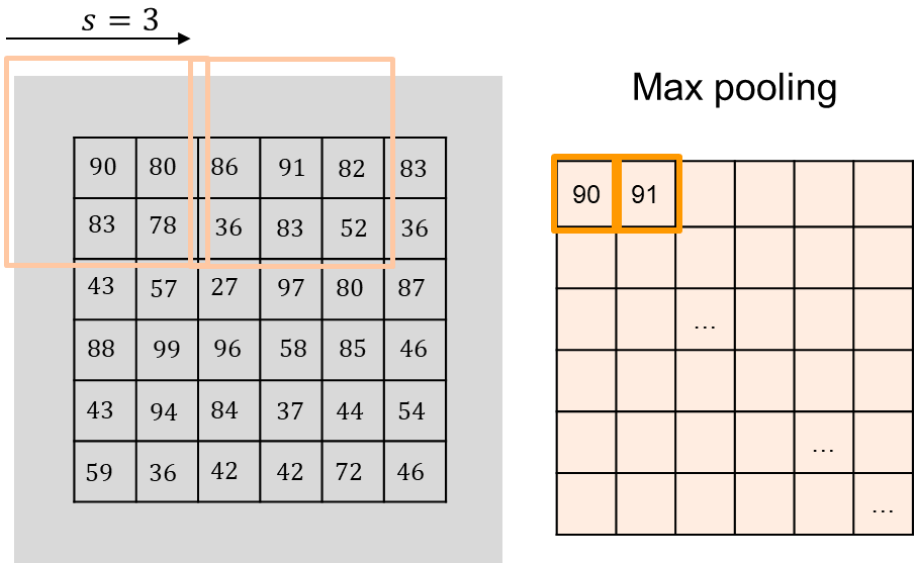
$$u_{ij}^{k,l} = \frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l}$$

$$u_{ij}^{k,l} = \left(\frac{1}{P^2} \sum_{p,q \in P_{i,j}} z_{pq}^{k,l} \right)^{1/n}$$

If $n = 1$, Lp pooling reduces to Average pooling.
If $n \to \infty$, Lp pooling reduces to Max pooling.

Pooling

- There is no hyperparameter in the Max or Average pooling, while in Lp pooling the hyperparameter is n.
- Generally pooling calculation is done on each input channel independently. Thus the number of channel remains the same after pooling.
- Normally no activation function is applied after pooling.
- Stride of the pooling, similar as in convolution



Comments

- The convolution layer is not fully connected.
- Flatten is the process of converting all the resultant 2D or 3D arrays from pooled feature map into a single long continuous linear array. The array goes afterwards into a fully connected NN.
- Besides the convolution, activation, pooling, there can be more steps like normalization.
- There are different variations of CNN for specific tasks, e.g. Region based CNN, Mask R-CNN, Mesh R-CNN.

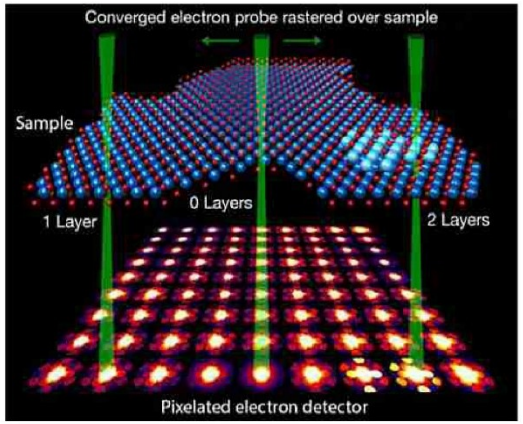
Motivation: Example 3 Crystal structure characterization ?

Conventional template matching

- Templates are generated via crystal structure simulation
- Calculation of *Correlation Index* (*CI*)

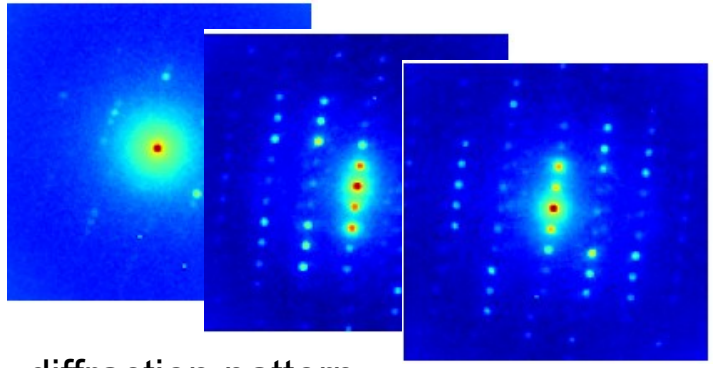
$$CI\ (i) = \frac{\sum_{j=1}^{m} P(x_j, y_j) T_i(x_j, y_j)}{\sqrt{\sum_{j=1}^{m} P^2(x_j, y_j)} \sqrt{\sum_{j=1}^{m} T_i^2(x_j, y_j)}}$$

- Slow, computational expensive and potentially imprecise



Scanning Transmission Electron Microscopy 4DSTEM

Ophus 2019 - Four-Dimensional Scanning Transmission Electron Microscopy.jpg (Ophus 2019, p. 564)



diffraction pattern

?

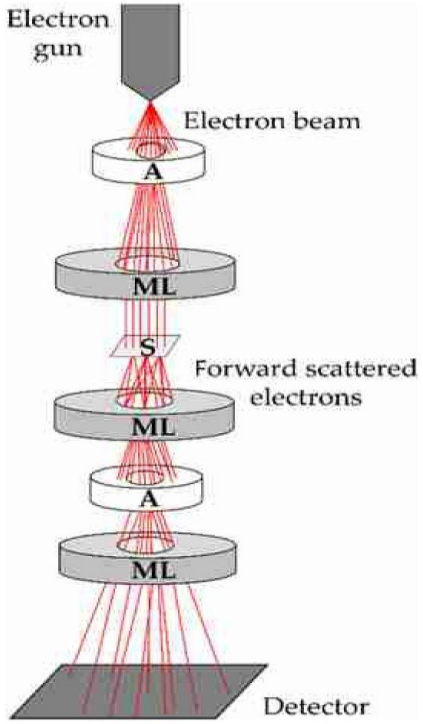
Euler angles
grain orientation

MECHANICS of FUNCTIONAL MATERIALS

CNN analysis of diffraction pattern and 4D TEM

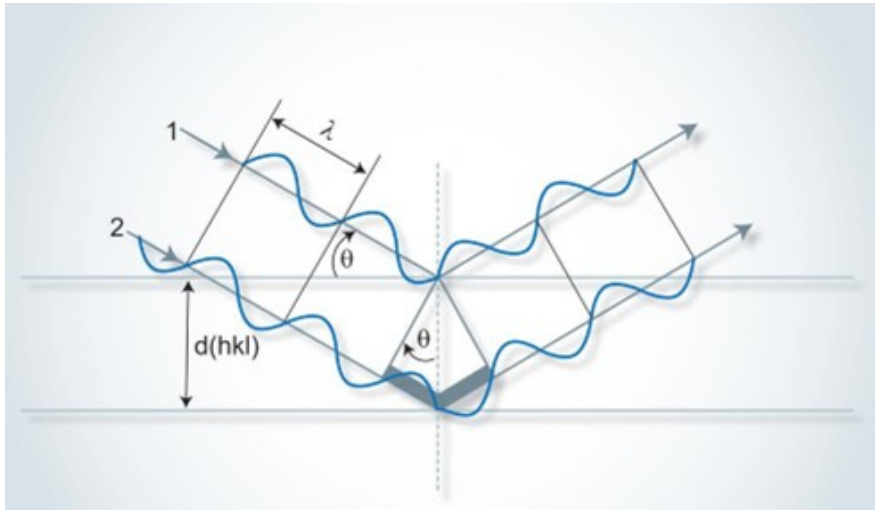
TECHNISCHE UNIVERSITÄT DARMSTADT

Conventional Transmission Electron Microscopy (TEM)

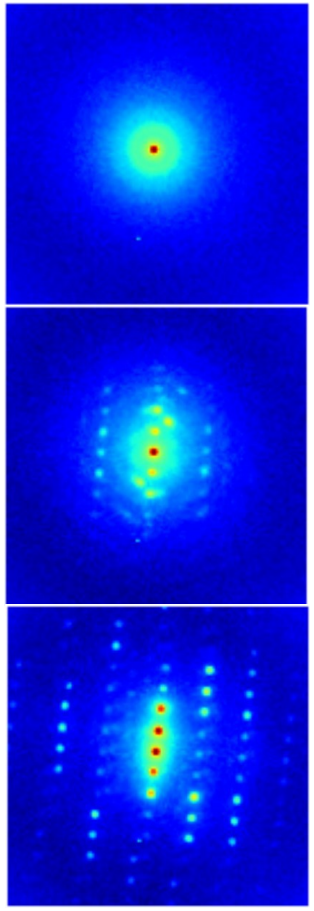


Malenica, Vukomanović et al 2021 - Perspectives of Microscopy Methods.jpg (Malenica et al. 2021)

Braggs Diffraction
Equation: $n\lambda = 2 \cdot d \cdot sin\theta$



Charles Campana 2015 - Single Crystal Diffraction.jpg (Charles Campana 2015)

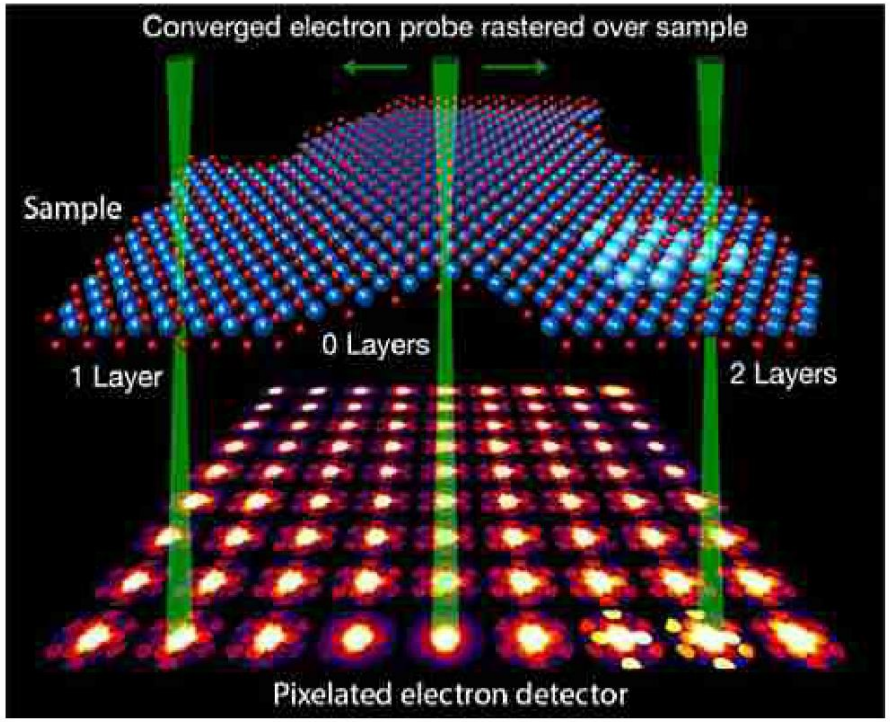


MECHANICS of FUNCTIONAL MATERIALS

CNN analysis of diffraction pattern and 4D TEM

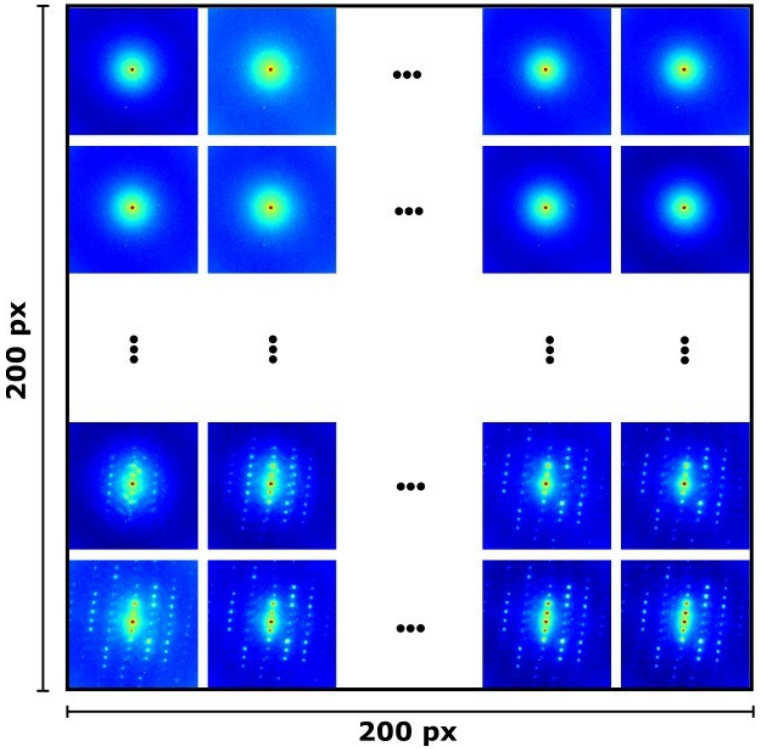
TECHNISCHE UNIVERSITÄT DARMSTADT

Scanning Transmission Electron Microscopy STEM



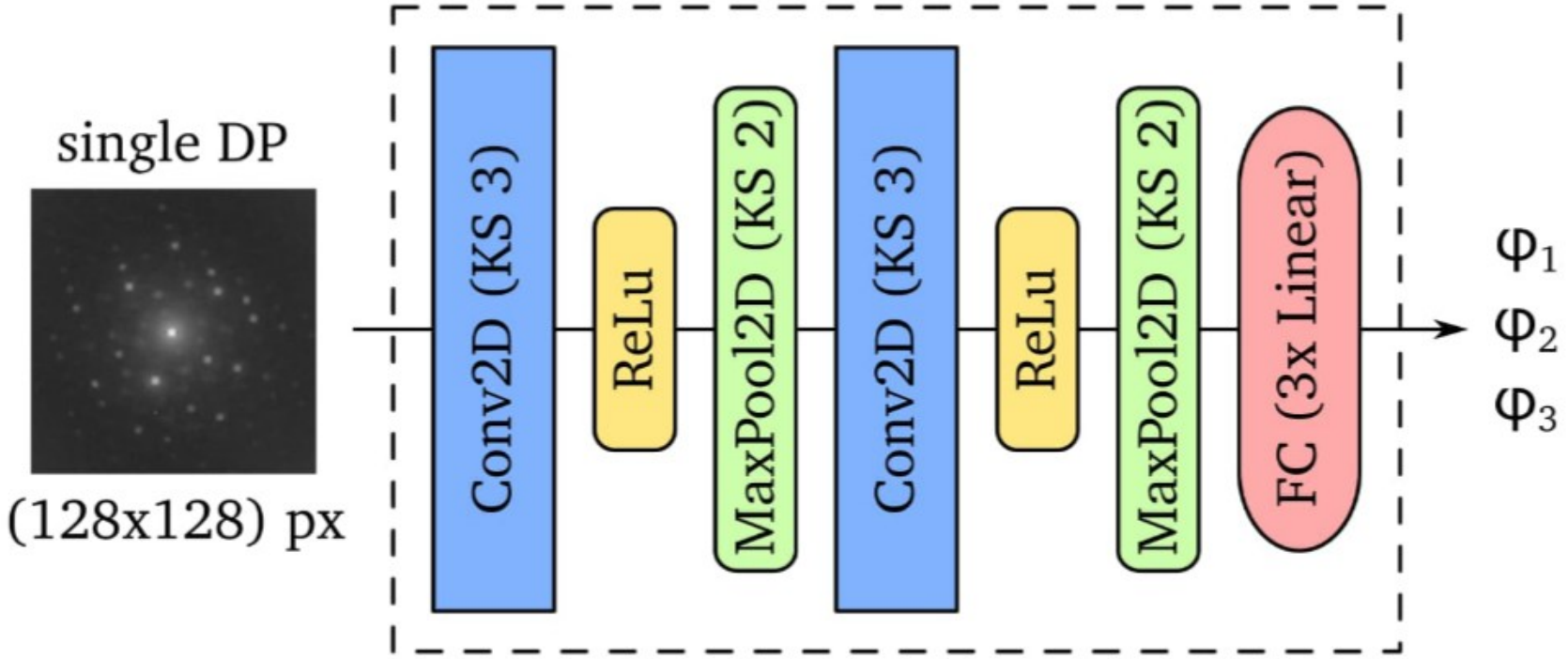
Ophus 2019 - Four-Dimensional Scanning Transmission Electron Microscopy.jpg (Ophus 2019, p. 564)

Scanning in x- and y-direction
Diffraction pattern at every position
4d dataset (x, y, k_x and k_y direction)



MECHANICS of FUNCTIONAL MATERIALS

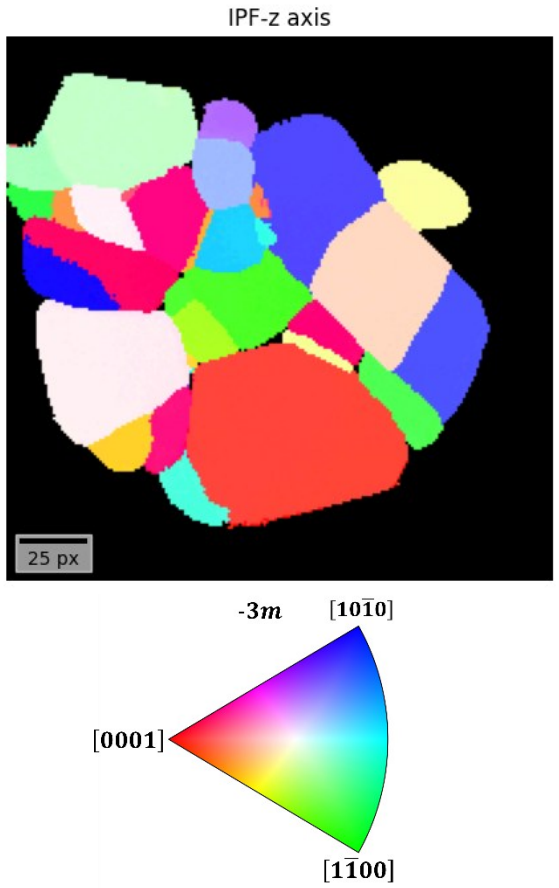
CNN analysis of diffraction pattern



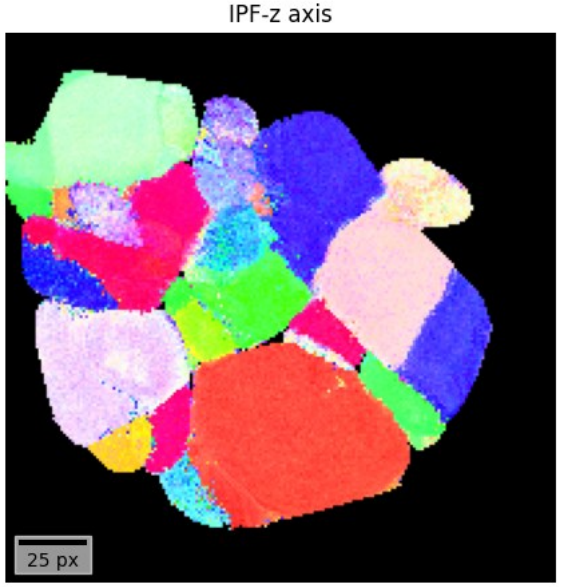
Wissel, Scheunert, Dextre, Ahmed, Beyer, Volz, Xu, https://arxiv.org/pdf/2507.01889

Comparison CNN vs. Template matching result

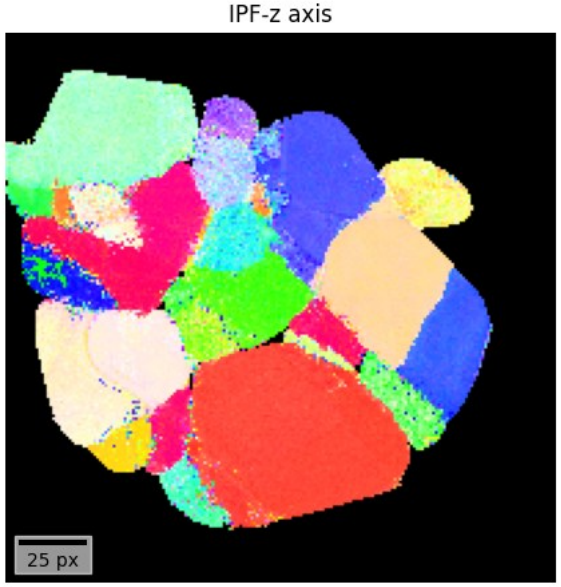
Grundtruth from template matching



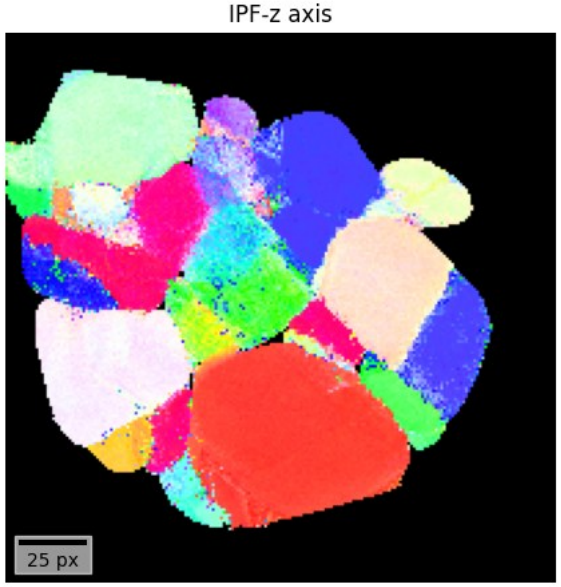
CKS 7; 150 epochs
R2 = 0.97



CKS 3; 270 epochs
R2 = 0.97

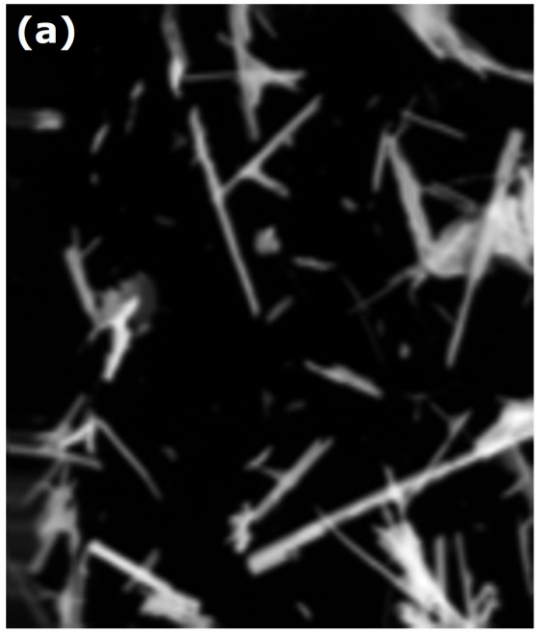


CKS 25; 400 epochs
R2 = 0.96

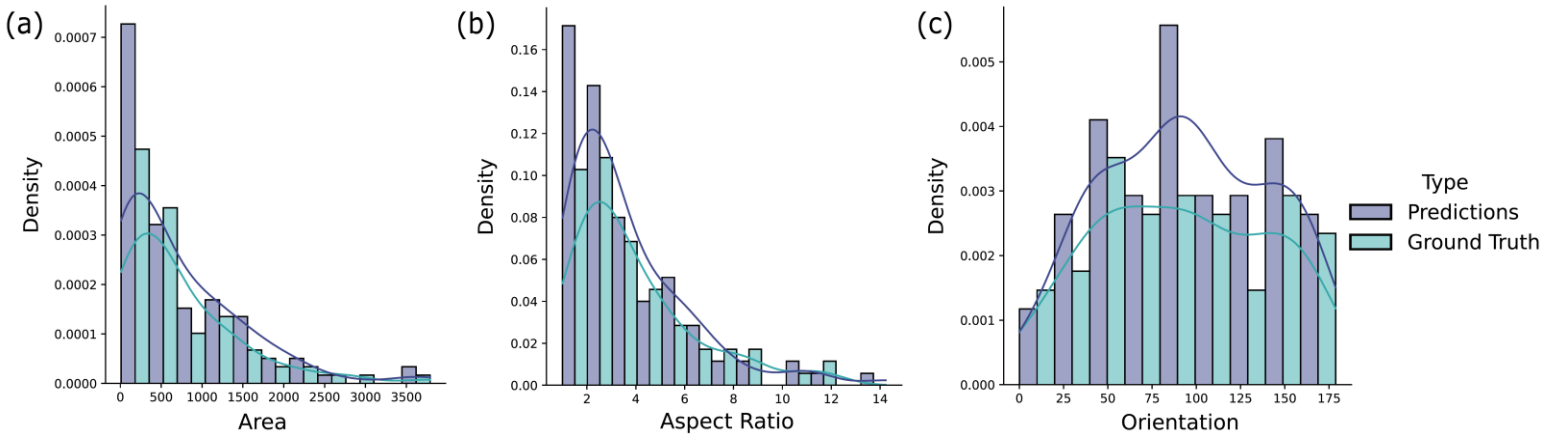


Wissel, Scheunert, Dextre, Ahmed, Beyer, Volz, Xu, https://arxiv.org/pdf/2507.01889

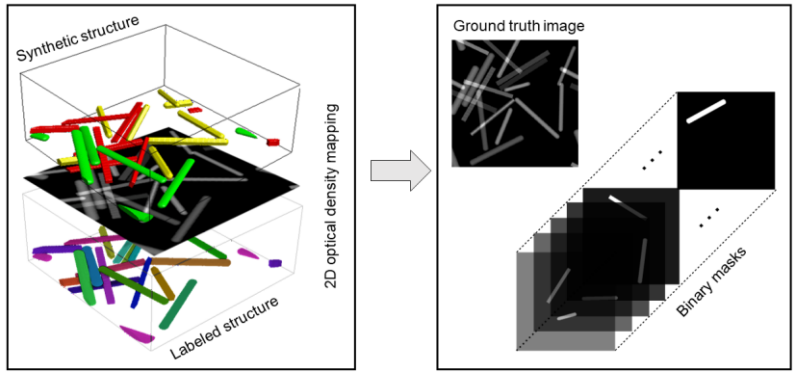
Motivation: Example 2 Microstructure characterization ?

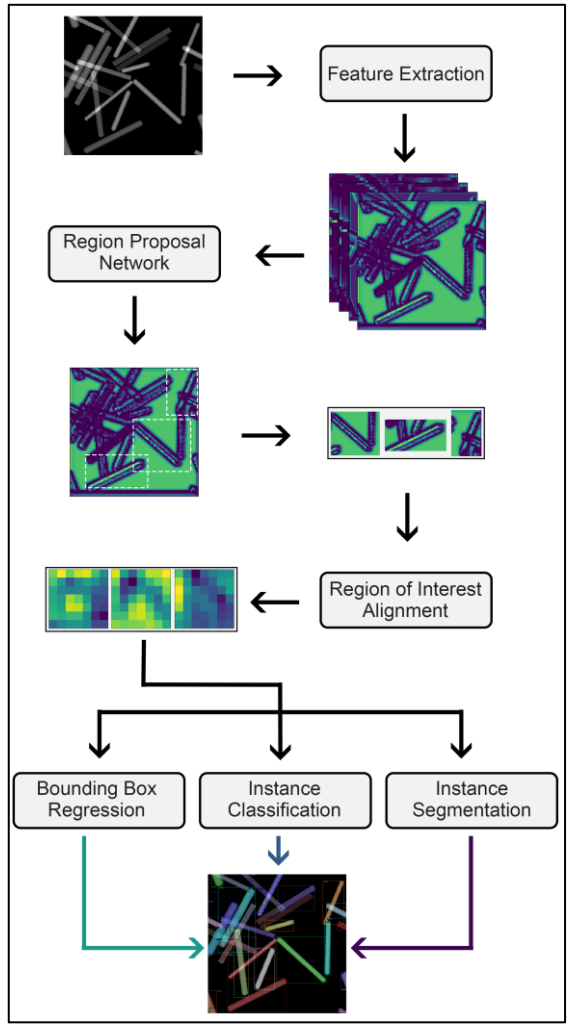


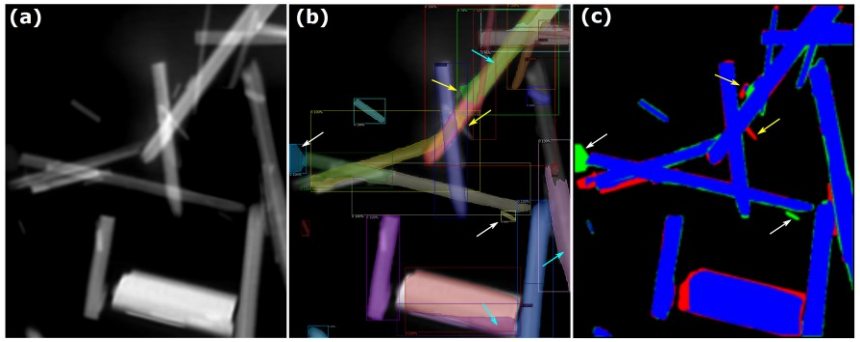
?

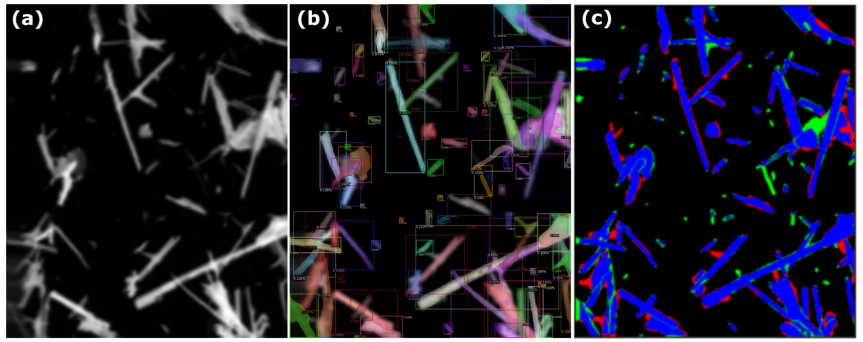


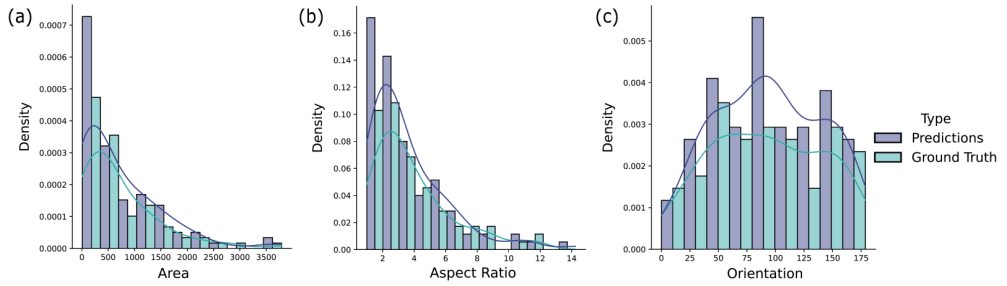
Mask R-CNN enabled Fiber Segmentation Model







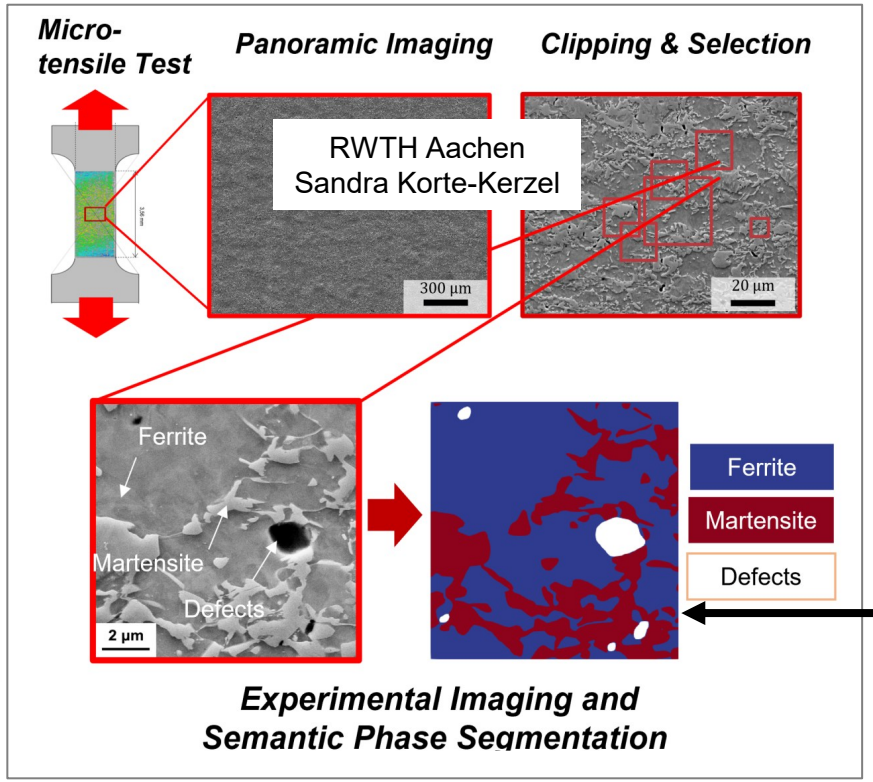


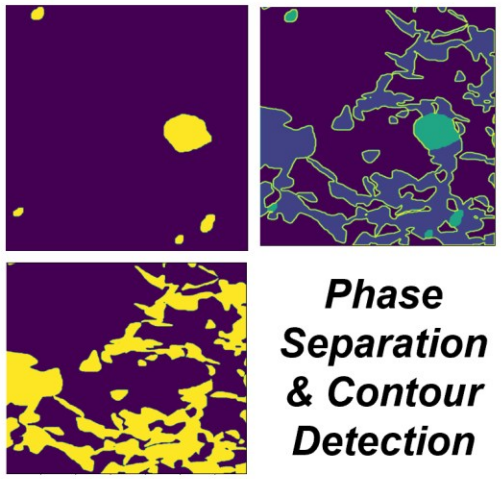


The web-App based on the trained ML model is available here: https://share.streamlit.io/linbinbin92/V2O5_app/V2O5_app.py

Lin, B., … Banerjee S. Xu, B.-X. A deep learned nanowire segmentation model using synthetic data augmentation. *npj Comput Mater* **8,** 88 (2022). https://doi.org/10.1038/s41524-022-00767-x

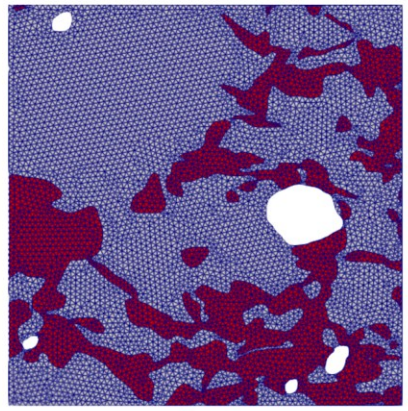
Elastoplastic simulation of two-phase steel



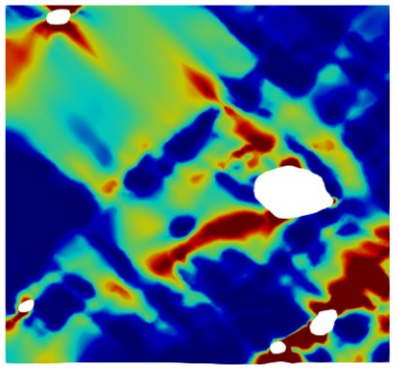


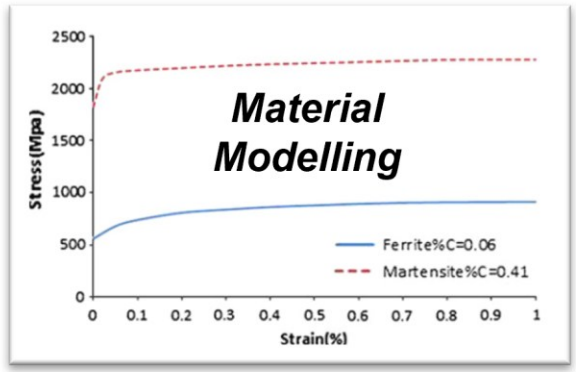
Phase Separation & Contour Detection

FE Meshing



Stress/Strain field Prediction



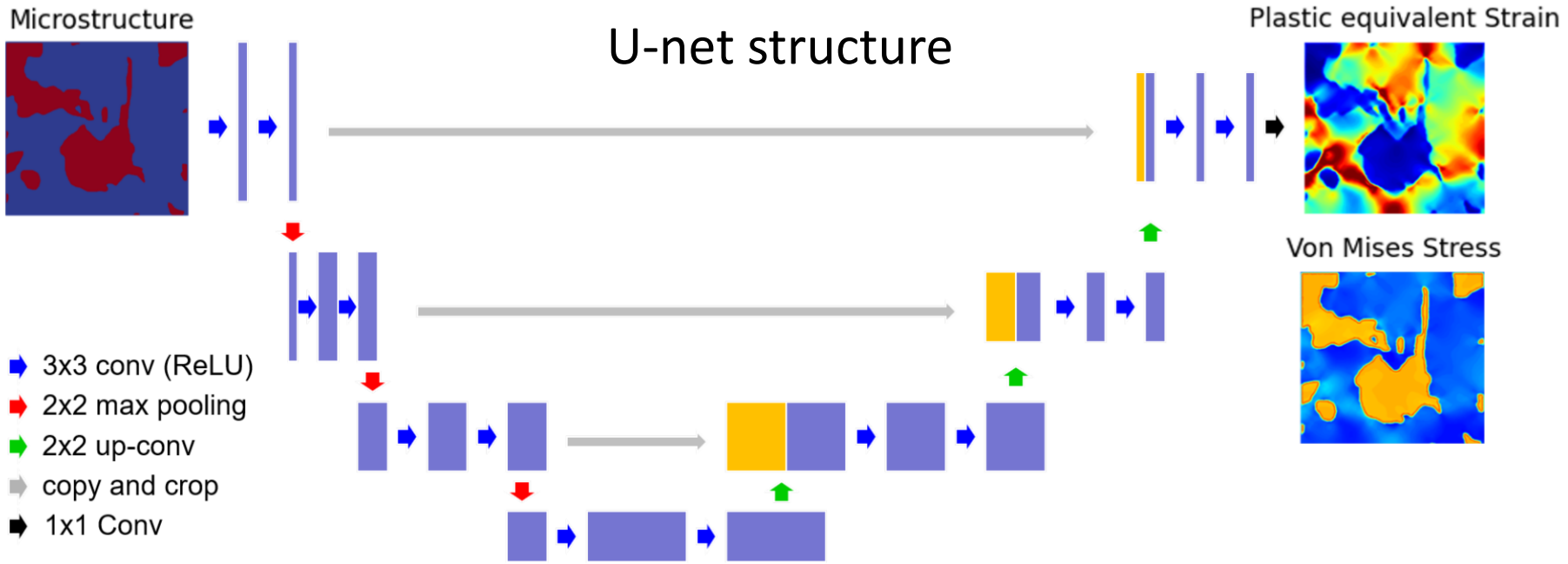


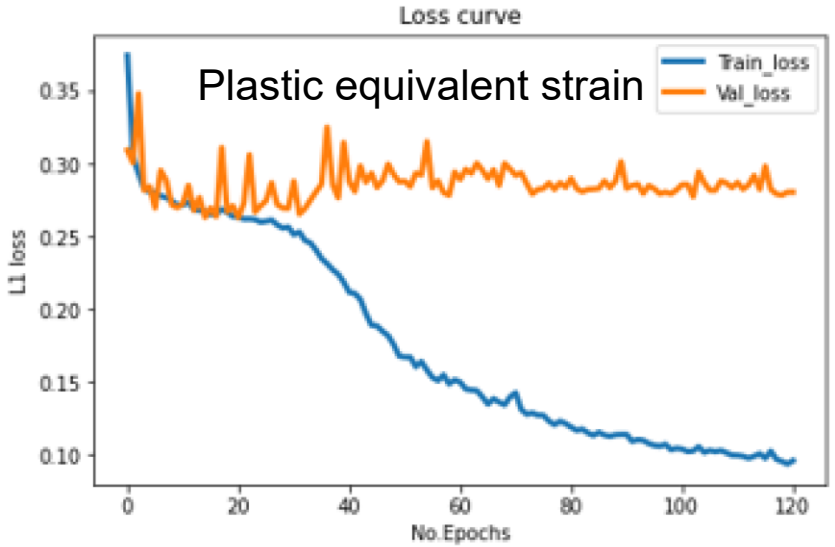
FE – Simulation of Microstructural Images

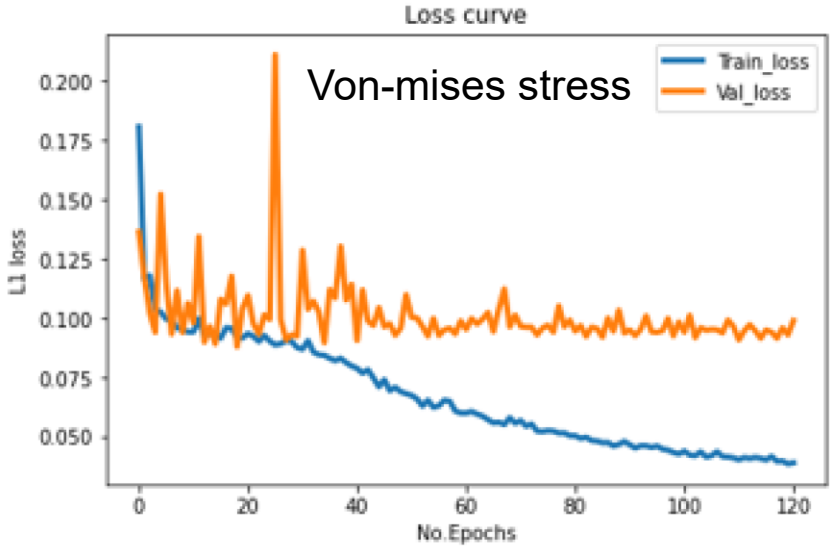
plasticity (von Mise, isotropic hardening)

$$\boldsymbol{\sigma} = \boldsymbol{E}(\rho, T) : \left(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^{\mathrm{pl}}\right)$$

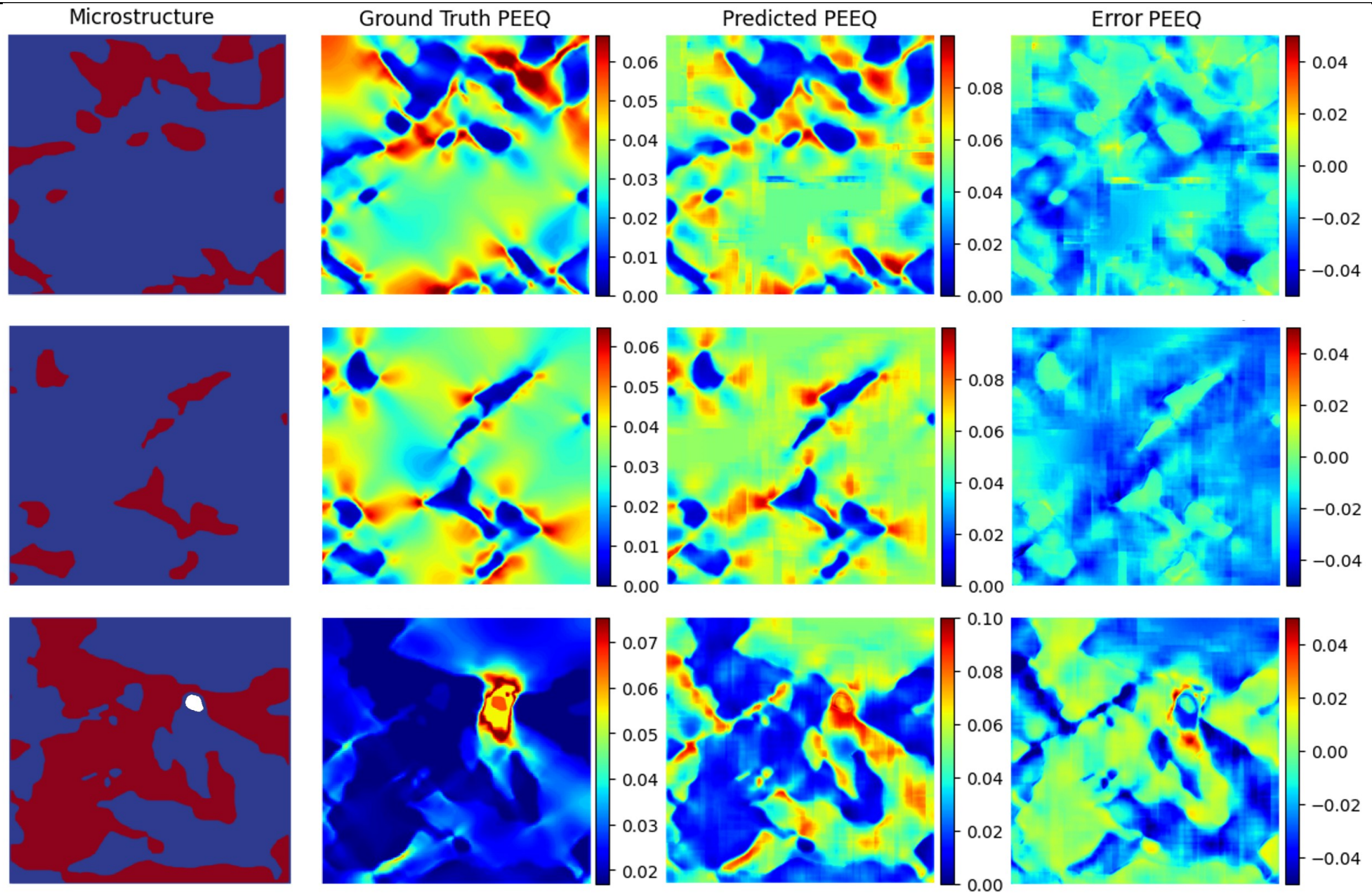
$$f(\sigma, p_e) = \sigma_e - (\sigma_Y + H\alpha)$$
$$\sigma_e = \sqrt{s_{ij} s_{ij}/2},$$
$$\dot{\alpha} = |\dot{p}_e|, \quad p_e = \sqrt{3\boldsymbol{\varepsilon}^{pl} : \boldsymbol{\varepsilon}^{pl}/2}$$







Lin, Medghalchi, Korte-Kerzel, Xu, PAMM 2023, acccepted. A Machine Learning Enabled Image-data-driven End-to-end Field Predictor For Dual-Phase Steel.
online available.


Microstructure
Ground Truth PEEQ
Predicted PEEQ
Error PEEQ
0.06
0.05
0.04
0.03
0.02
0.01
0.00
0.08
0.06
0.04
0.02
0.00
0.04
0.02
0.00
−0.02
−0.04
0.07
0.10

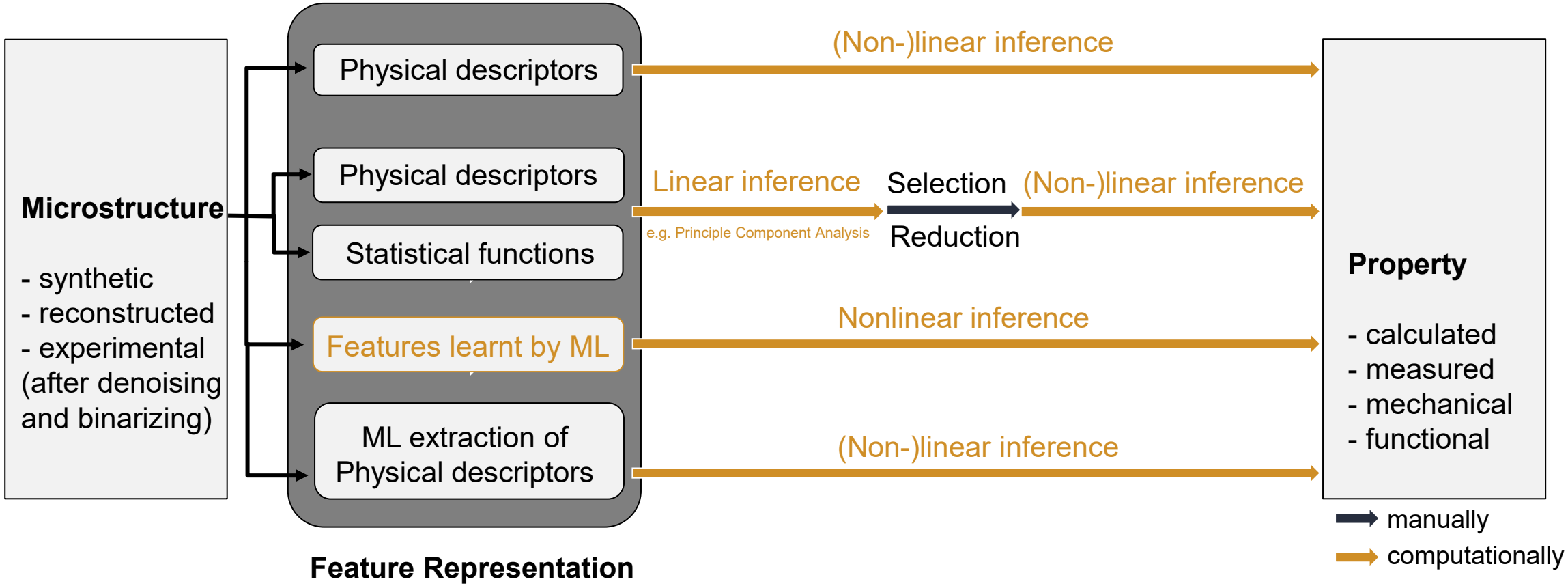
TECHNISCHE UNIVERSITÄT DARMSTADT

Microstructure Representation

- Physical descriptors
- Statistical functions
- Features presented by ML

MECHANICS of FUNCTIONAL MATERIALS

Microstructure – Property Relation

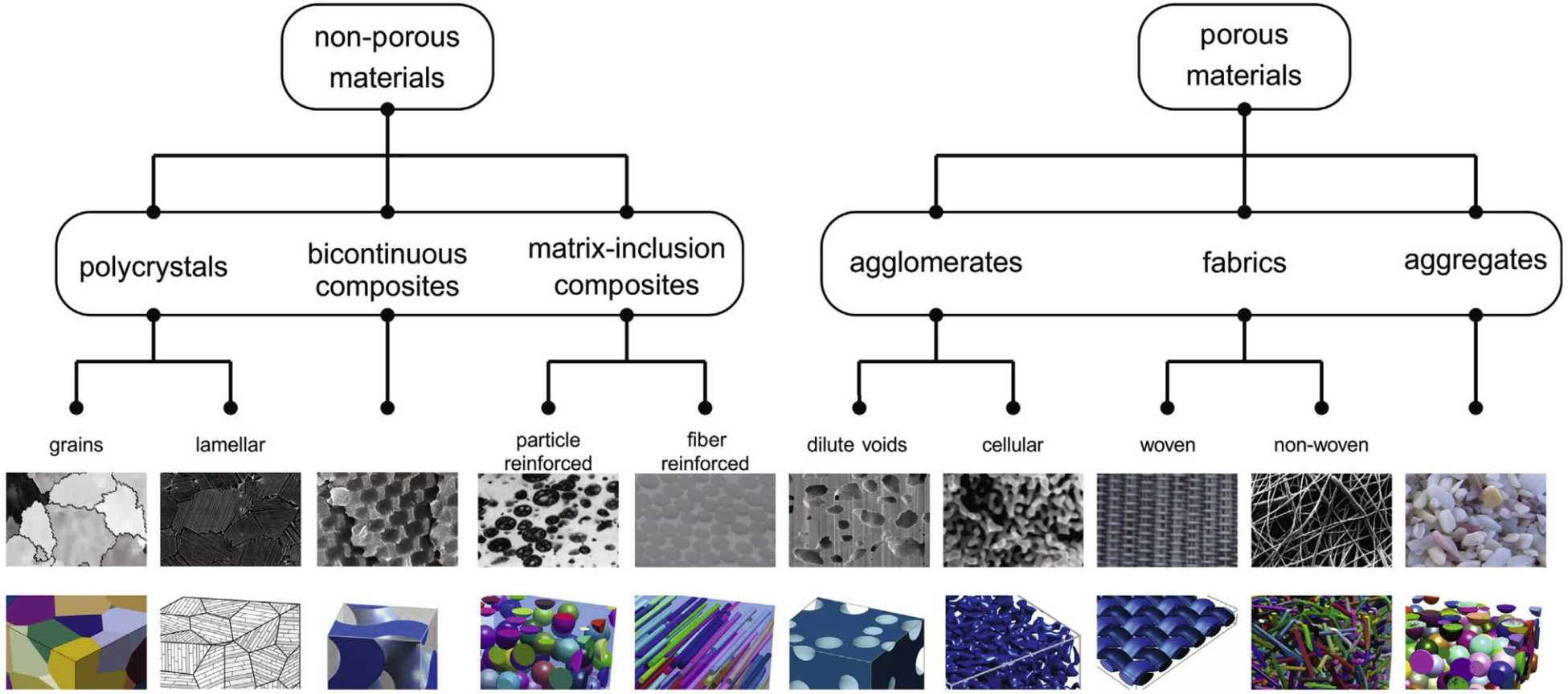


Comment: Feature Representation is not only needed for Microstructure-Property relation, but also can be necessary for other purposes, e.g. *microstructure reconstruction* (for subsequent computation of property calculation, for 2D-to-3D generation, or for visualization of optimized features).

I. Physical Descriptors

TECHNISCHE UNIVERSITÄT DARMSTADT

- They depend strongly on microstructure, materials and the target property.



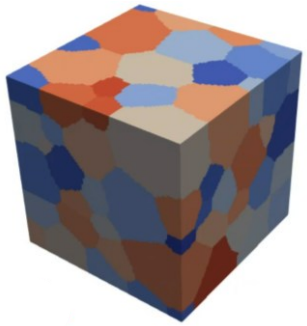
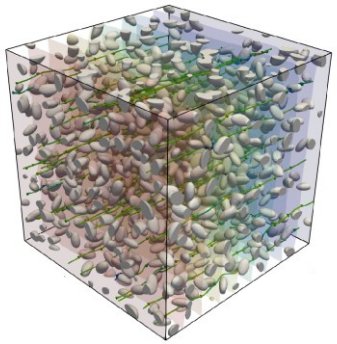
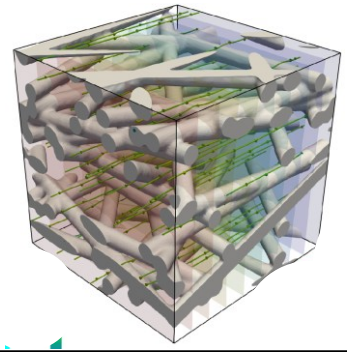
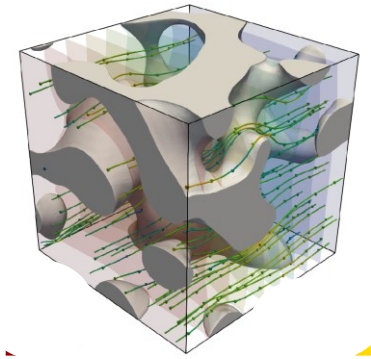
S. Bargmann et al. / Progress in Materials Science 96 (2018) 322–384

Fig. 2. Classification of heterogeneous solids into porous and non-porous media. Based on various morphological features, these categories are further subdivided into different microstructure classes. See acknowledgements for information on figure copyrights and permissions to reprints.

I. Physical Descriptors

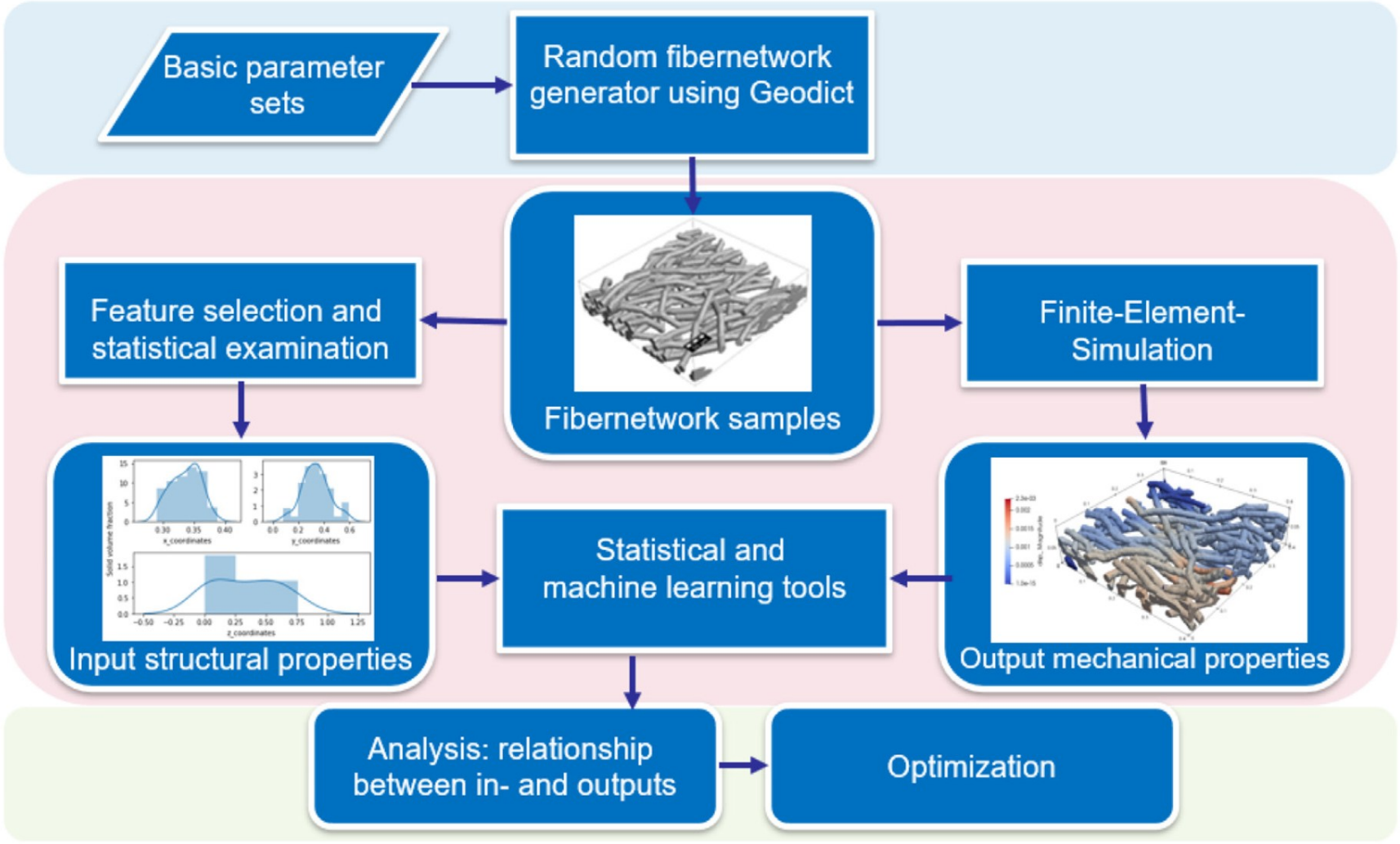
TECHNISCHE UNIVERSITÄT DARMSTADT

- They can be generally classified deterministic or statistical; geometric, compositional/chemical or dispersional.

Type Yang et al. Scripta Materialia 212 (2022) 114537	Polycrystalline	Matrix-inclusion composite	Fiberous material	Cellular materials
				
Deterministic	_	Volume fraction; Inclusion aspect ratio; Number of inclusion particles	Grammage; Building direction	Porosity
Statistical	Grain size, Grain orientation, Grain boundary feature (character, area, composition, structure)	Cluster‘s nearest center distance	Fiber length; Fiber orientation; Contact area, Fiber diameter	Pore surface: Tortuosity; Pore size, Ligament geometry, Surface property (orientation, structure)

- Physical descriptors can be correlated and have different level of impact on properties.
- To improve predictive performance and efficient design, there should be selection process based on e.g. variable ranking approach and exploratory factor analysis.

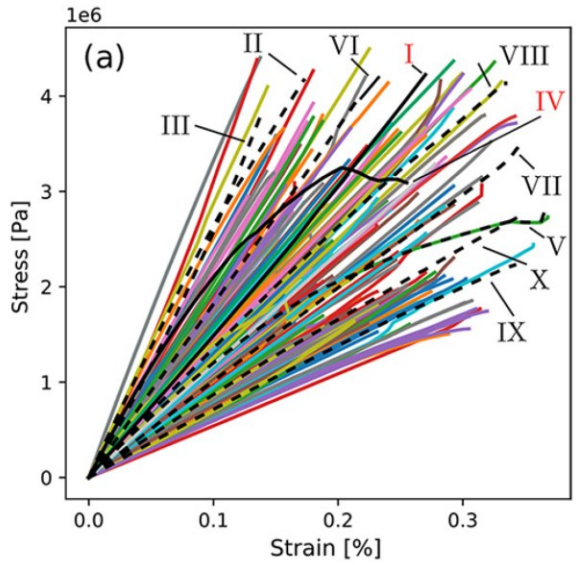
I. Physical Descriptors: Application case

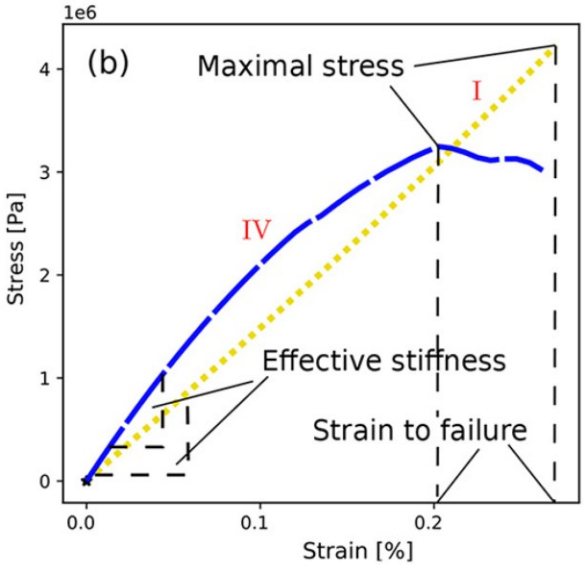


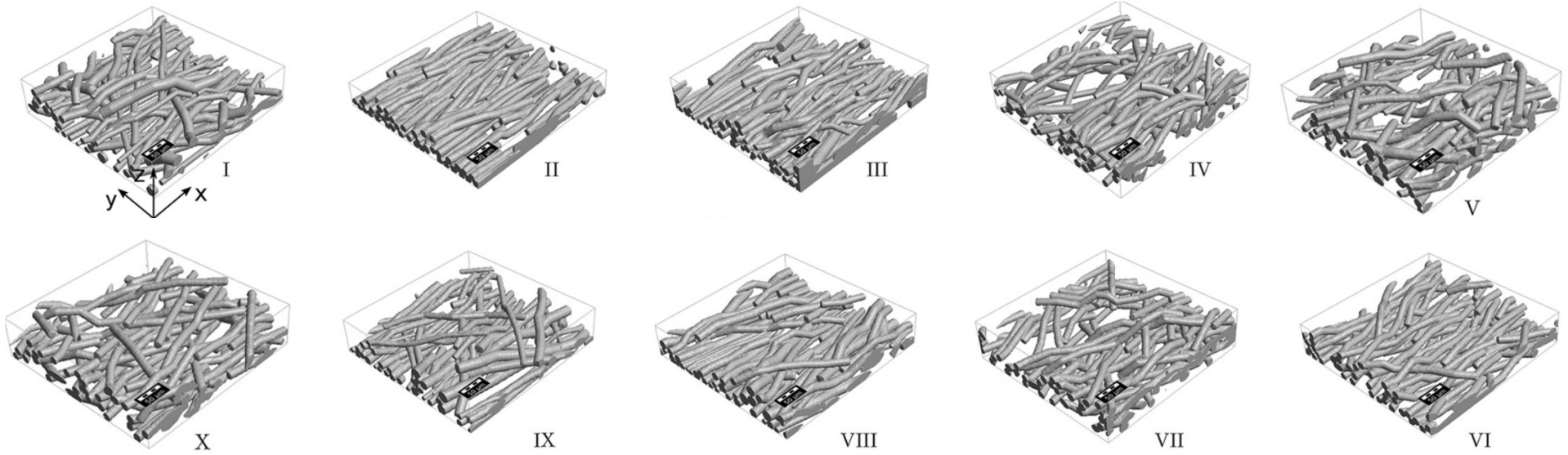
Lin, Binbin; Bai, Yang; Xu, Bai-Xiang. Data-driven microstructure sensitivity study of fibrous paper materials. In: Materials and Design 197 (2021) 109193

https://doi.org/10.1016/j.matdes.2020.109193

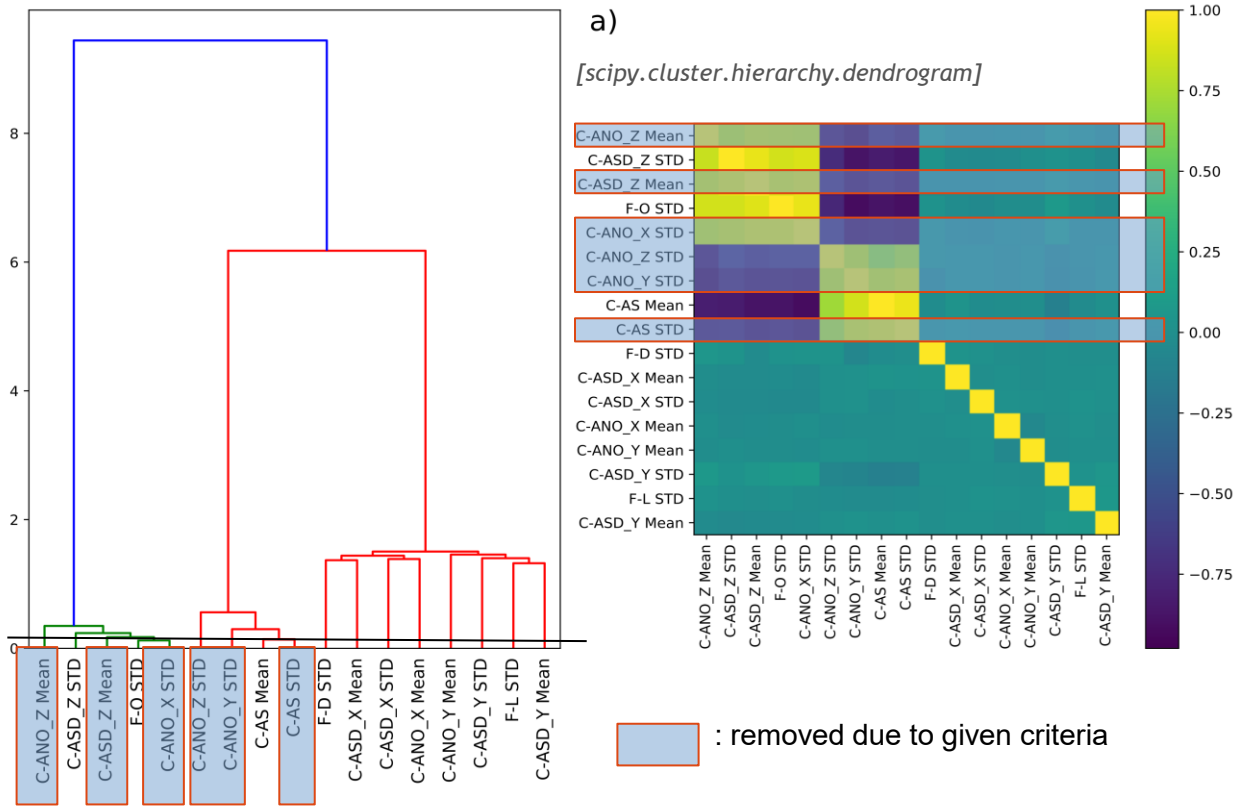
Fixed fiber network setting		Structural features as Input		
Domain size x, y	400 μm	*Basic structural distributions*		
Distribution in orientation, length, diameter	Gaussian	Std orientation	[−90, 90] μm	F-O STD
		Std fiber length	[−100,100] μm	F-L STD
		Std diameter	[−10,10] μm	F-D STD
Grammage	30 g/m^2	*Derived contact area features*		
Mean orientation	0° in x	Mean contact area size		C-AS Mean
Mean fiber length	300 μm	Std of contact area size		C-AS STD
Mean diameter	17 μm	Mean spatial projection of contacts (X,Y,Z)		C-ASD_ X,Y,Z Mean
Mechanical target variables as Output		Std spatial projection of contacts (X,Y,Z)		C-ASD_ X,Y,Z STD
Strain to failure		Mean contact area normal (X,Y,Z)		C-ANO_ X,Y,Z Mean
Effective stiffness		Std contact area normal (X,Y,Z)		C-ANO_X,Y,Z STD
Maximal stress				







MECHANICS of FUNCTIONAL MATERIALS



Dimensionality reduction:

- Spearmann rank-order correlation (right figure)

$$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$$

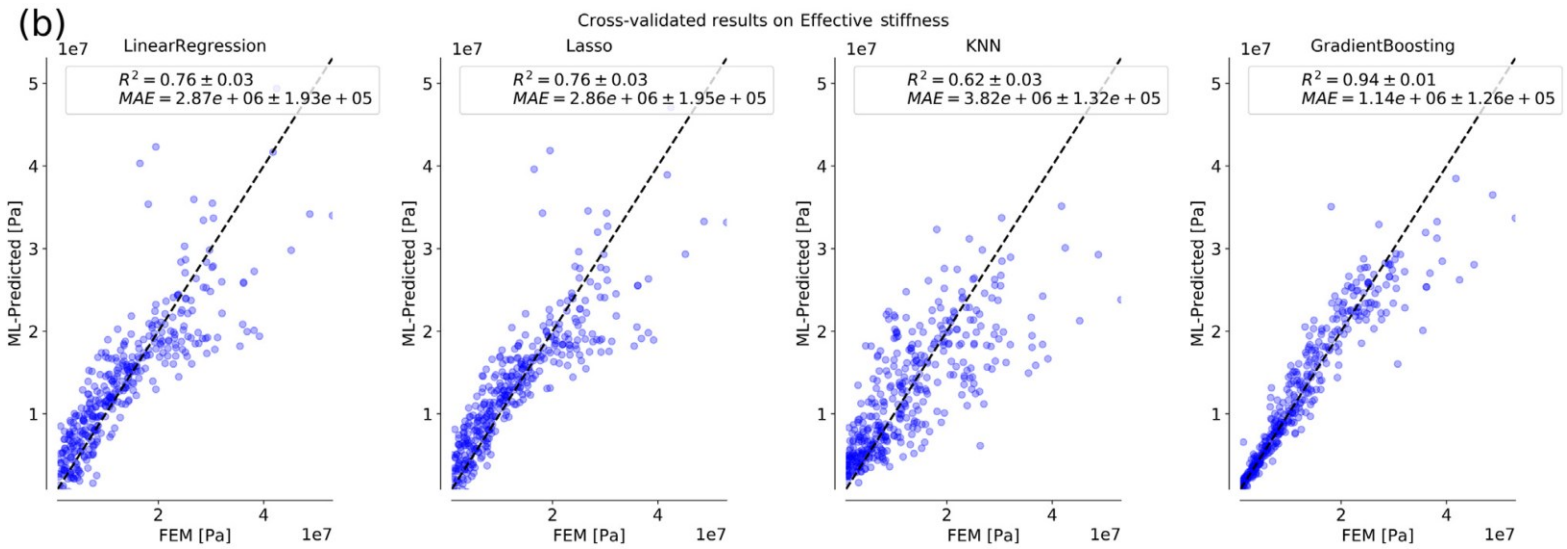
- Hierarchical clustering based on rank-order correlation matrix (left figure)

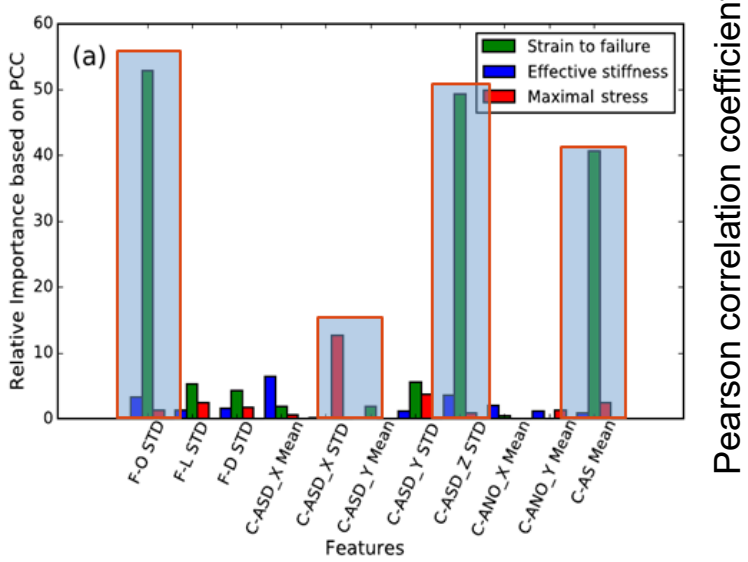
- Threshold setting for ρ > 0.9
- Manually inspection

(keeping at least one feature for each cluster)

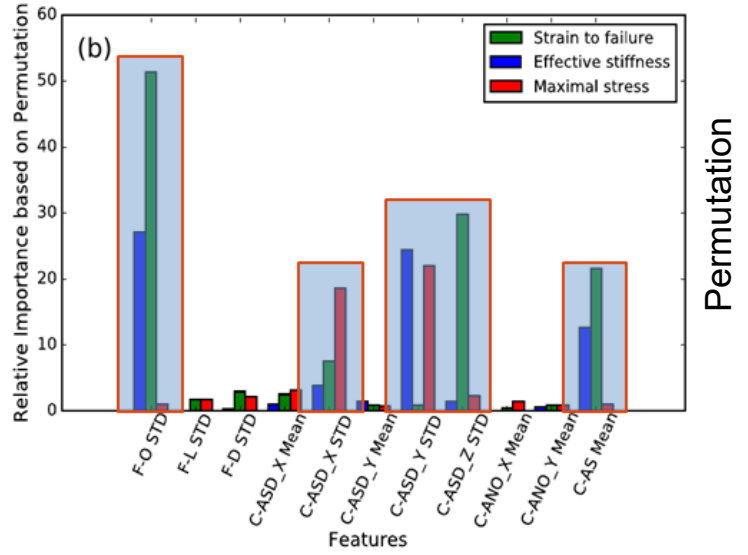
Feature selection: From 17 → 11 parameters

Feature importance analysis





Pearson correlation coefficient



Permutation

$$R_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

Permutation: Shuffle the feature values & rank decrease or increase of model score

www.mawi.tu-darmstadt.de/mfm
https://github.com/linbinbin92/data_driven_microstructure_paper_material

MECHANICS of FUNCTIONAL MATERIALS

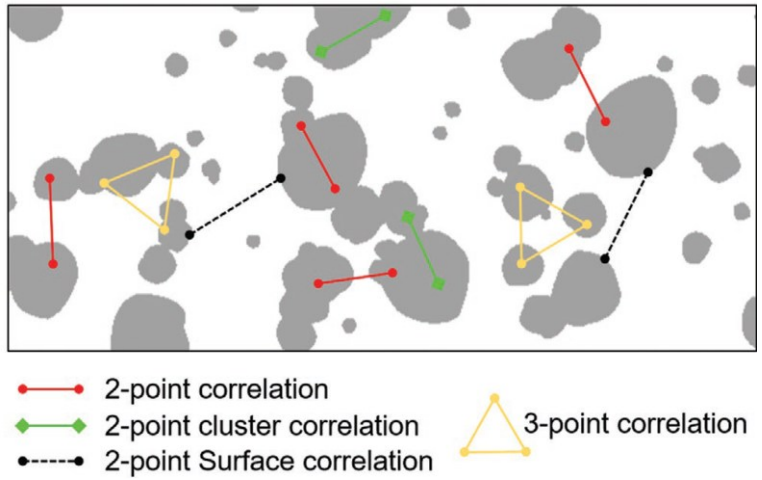
II. Statistical functions

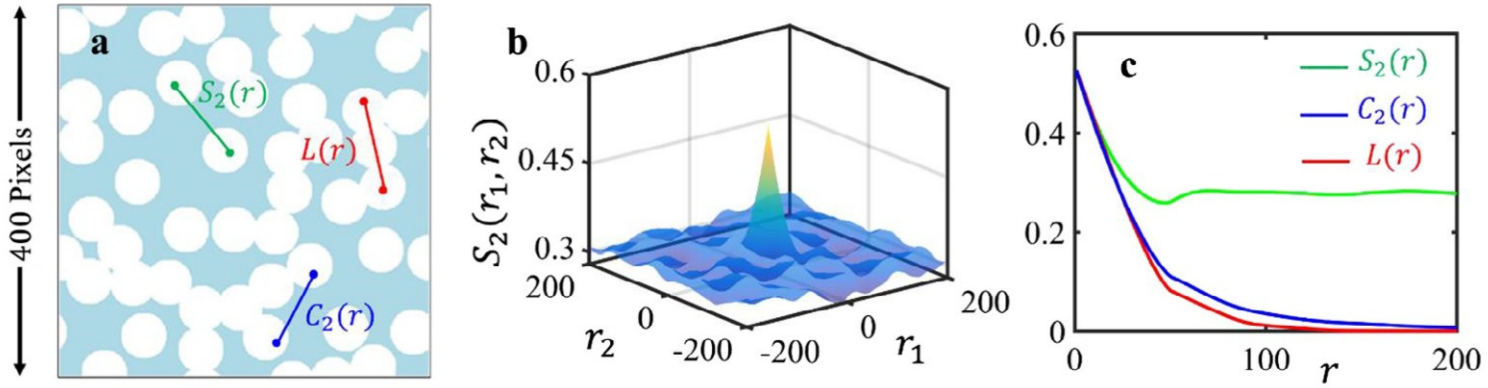
1) n-**point correlation functions** (n-point), which provides information on defined pair/group distributions regarding phase i

2-point correlation function: $S_2^i(\boldsymbol{r}_1, \boldsymbol{r}_2) =< X_{\boldsymbol{r}_1}, X_{\boldsymbol{r}_2} >, \quad X_{\boldsymbol{r}} = \begin{cases} 1, & \text{if in phase } i \\ 0, & \text{if not in phase } i \end{cases}$

2) **Lineal-path function** $L^i(\boldsymbol{r})$, which quantifies the amount of clusteredness along straight lines within phase i

3) **Two-point cluster correlation function** $C_2(\boldsymbol{r})$, quantifying the probability of finding two sites within phase i that are at distance r and in the same cluster





Xu et al. Adv. Engng. Mat. 2021, 2003908; Bostanabad et al. Progress in Materials Science 95 (2018) 1–41

II. Statistical functions

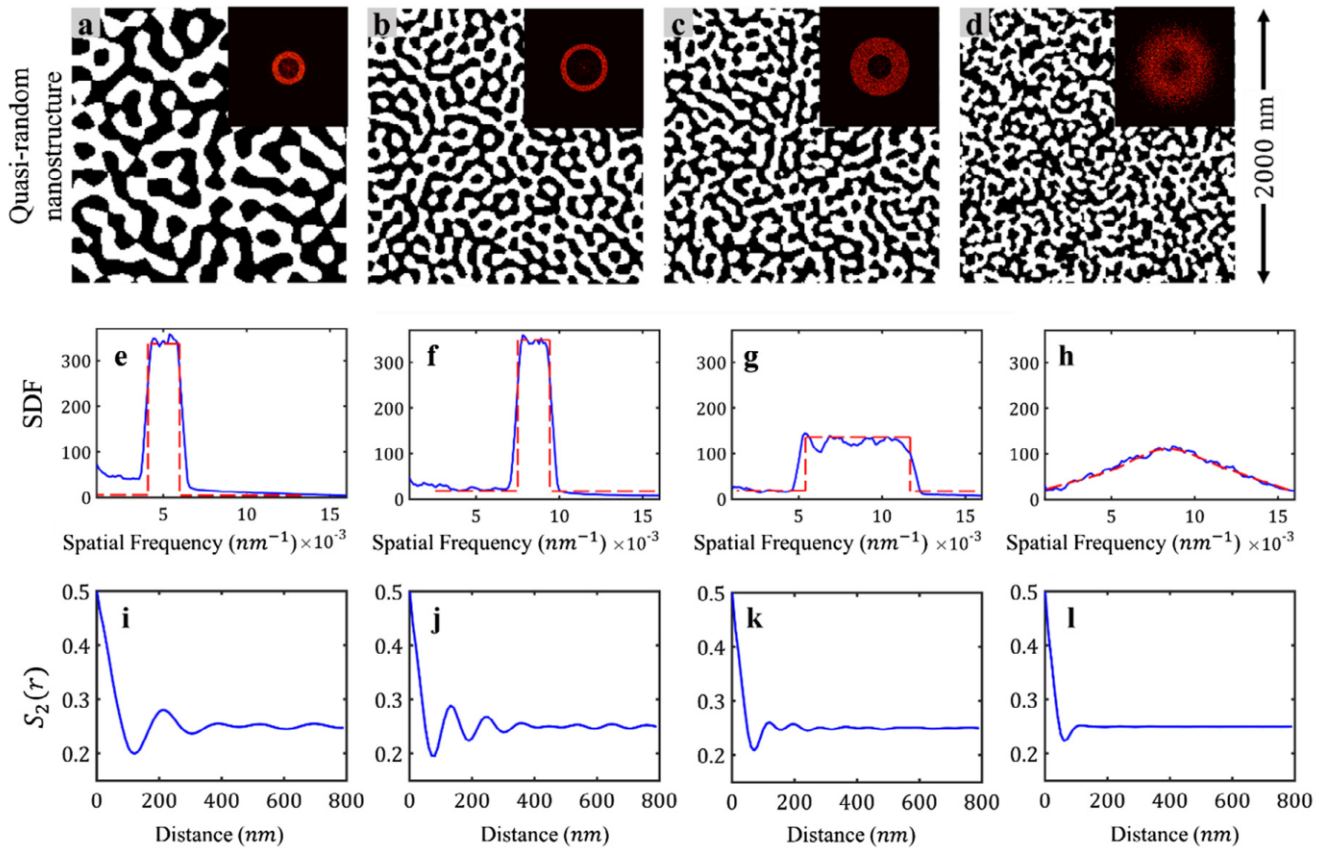
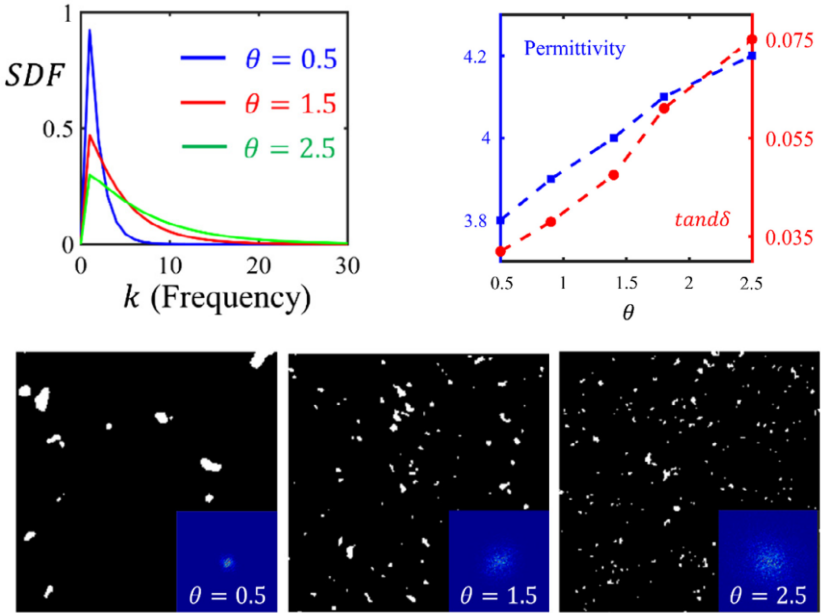


Fig. 15. Comparison between SDF and $S_2(r)$ characterization of four quasi-random microstructures: The differences in the microstructures such as feature scale and degree of randomness are better represented via SDF rather than $S_2(r)$. The insets in a through d are the 2*D* SDF's of the microstructures and due to isotropy, these 2*D* SDF's reduce to the 1*D* SDF's in e through h. See the text for more details.

4) **Spectral density function**, which is similar to the two-point correlation function but describes the structure correlations in a reciprocal frequency space rather than the real space

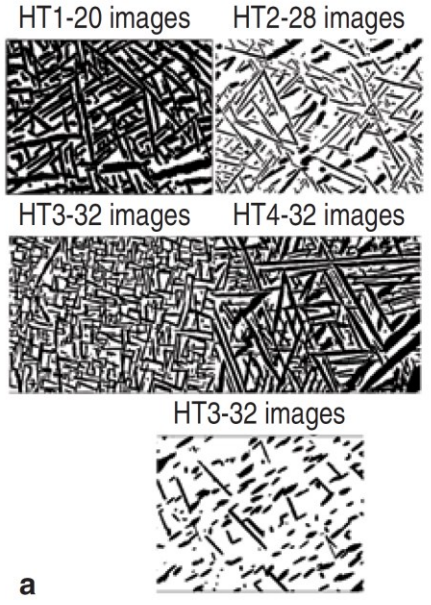


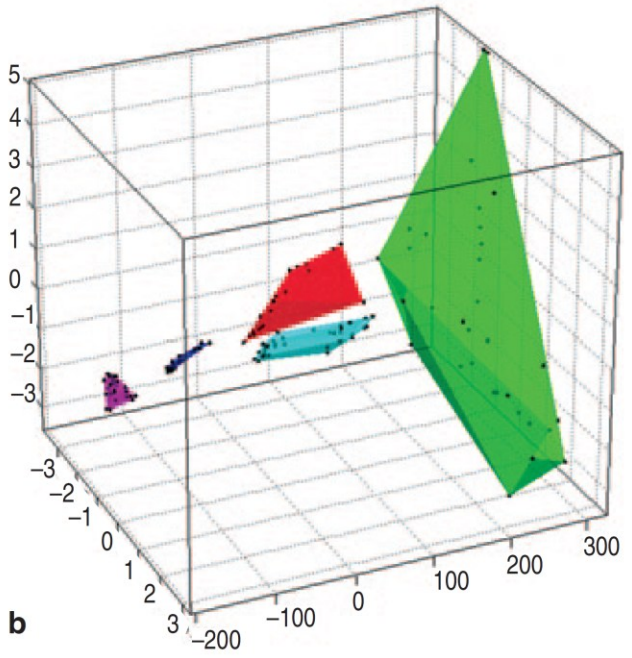
Bostanabad et al. Progress in Materials Science 95 (2018) 1–41;

II. Statistical functions

Comments

- There are various other statistical functions available. The choice depends on the microstructure and multiple types are recommended.
- Such statistical functions are usually lengthy and abstract, and difficult to be used directly for microstructure-property relation or microstructure reconstruction.
- To overcome this challenge, dimension reduction techniques, such as Principle Component Analysis (PCA, linear embedding) or nonlinear embedding methods, can be applied.
- The selected major principle components after dimention reduction are then used to build the relation model to property or microstructure reconstruction.



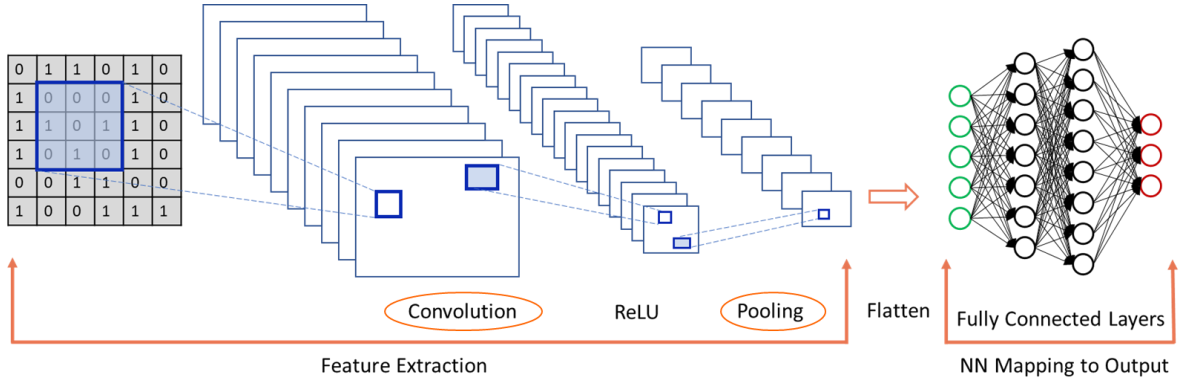


Kalidindi et al. JOM, 2011, Research Summary, Enabling ICME.

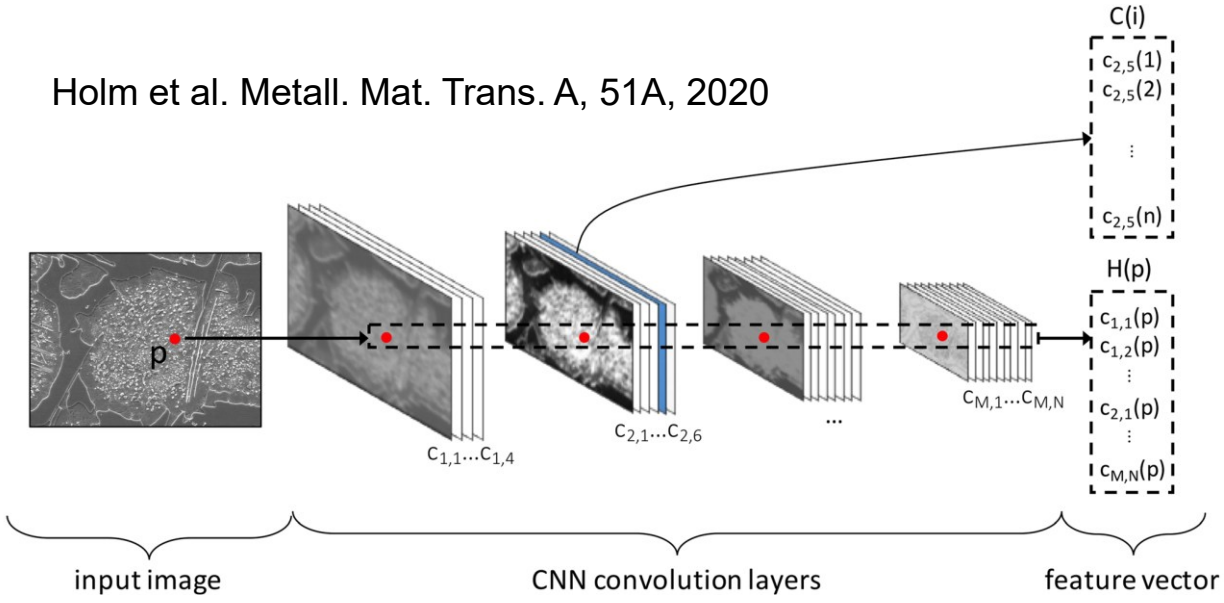
a) The backscatter electron micrographs of 5 distinct heat treatments of Ti-5553.
b) Visualization of microstructure in PCA space (the first three components) of the n-point correlation functions

III. Features represented by ML

- The network applies signal processing operations on the input image to encode it as a **feature vector**.
  - Filter activation in shallow layers represents local environments, while pooling causes pixels in deep layers to represent large areas i.e. deeper layers recapture features at larger length scales.
  - Features by CNN are translation invariant but are not rotation or scale invariant. (It implies that a feature may require muliple representations to capture its various geometric orientations within one microstructure)
  - To combine the information from all of the convolution layers, one approach is the hypercolumn pixel feature (by stacking the activations of the convolution layers at the one pixel‘s real space position and storing the pixel‘s feature membership at multiple length scales.)



Holm et al. Metall. Mat. Trans. A, 51A, 2020



III. Features represented by ML

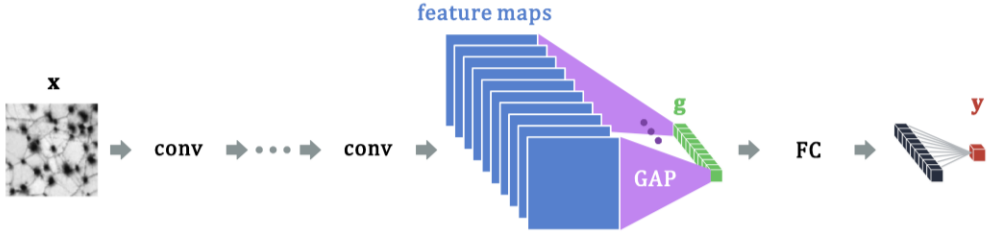
- It requires a large microstructure data set to train.
- Very well trained CNNs on natural image seem to work surpringly good also for microstructure data. This is the so-called Transfer Learning, which may be related to common fundamantal geometric features like edges, texture and so on.

- Using the CNN feature vector, different tasks (property as named generally previously) can be achieved:
  - classification (e.g. of different phases)
  - association with mesoscopic properties (e.g. the overall conductivity, the overall yield strength)
  - determination of physical descriptors of the microstructure (e.g. grain size distribution, surface detection)
  - …

  Depending on the task, the subsequent ML model applied to the feature vector can be supervised or unsupervised.

III. Features represented by ML

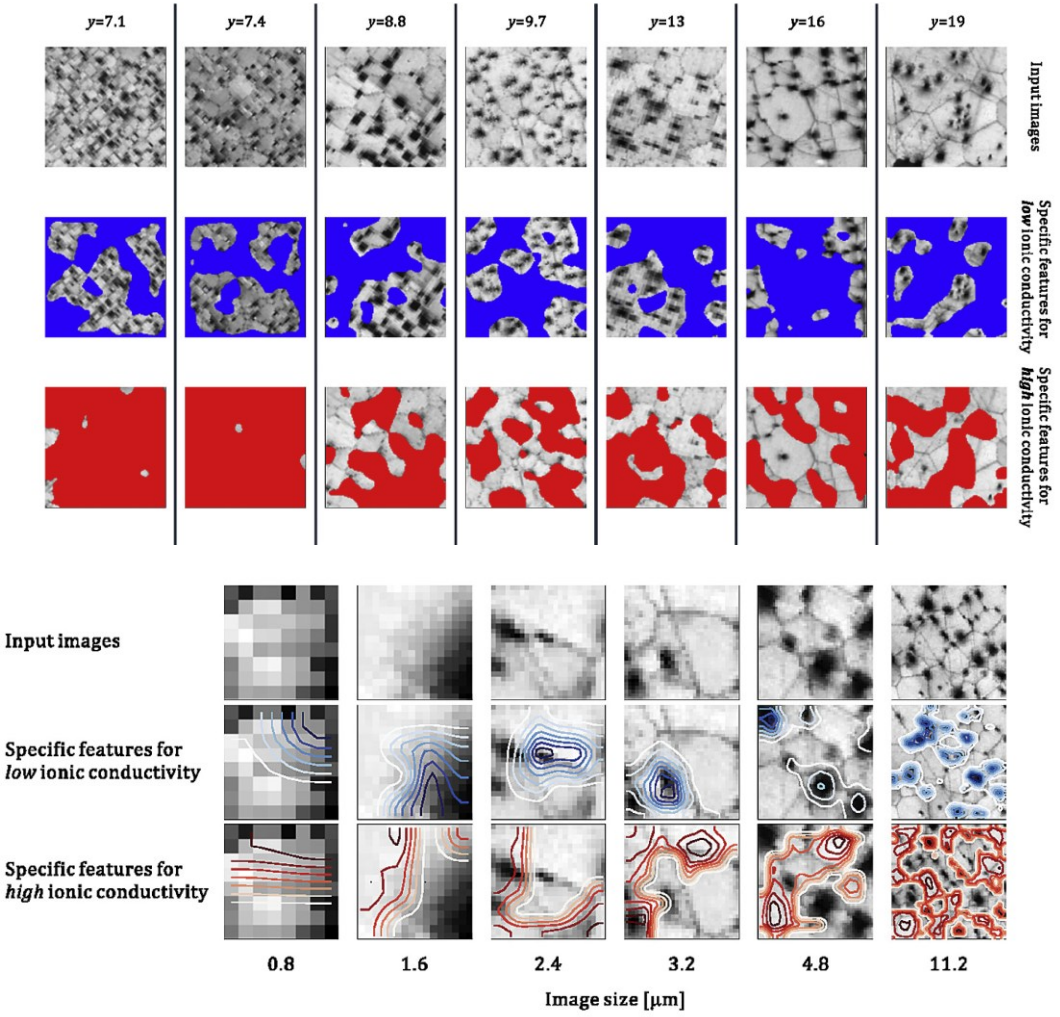
Kondo et al. Acta Materialia 141 (2017), pp. 29–38.
→ supervised learning of the microstructure-ionic conductivity relation; forward prediction



$$y = a_i g_i + b_i$$

y: ionic conductive
g_i: components of the global average pooling (GAP)

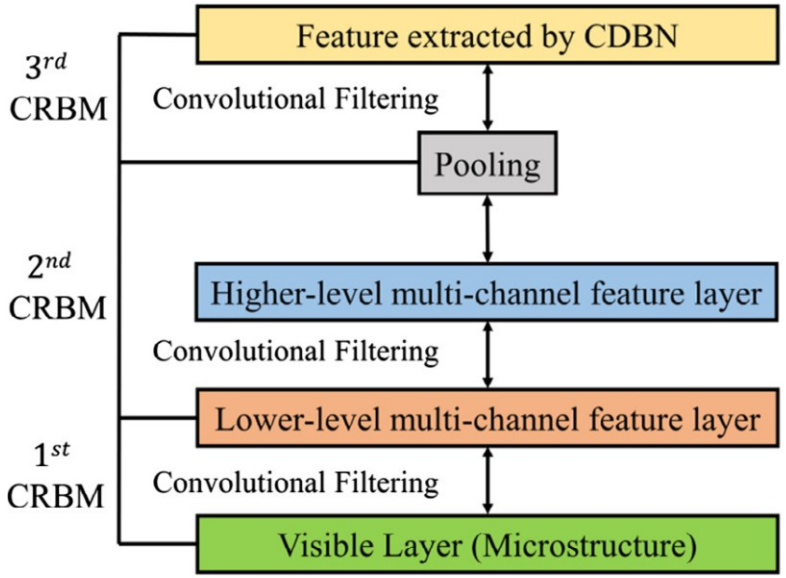
Based on the sign of the fitted a_i, the features are grouped into low (negative) and high (positive) ionic conductivity regions, respectively. Then, the component of the feature map just before GAP layer is averaged accordingly. The procedure resembles the class activation mapping.



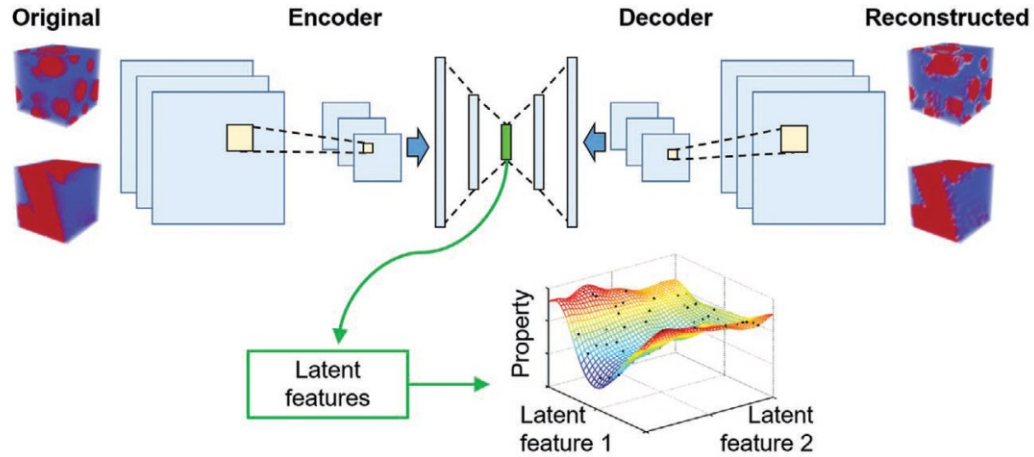
III. Features represented by ML

Cang and Ren, American Society of Mechanical Engineers. 2016

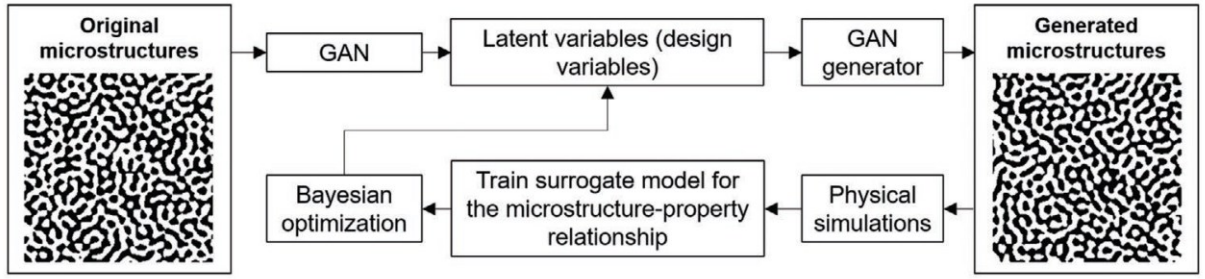
Convolutional deep belief network (CDBN)
Convolutional restricted Boltzman machine (CRBM)



Unsupervised learning; microstructure optimization
Jung et al., Materialia 2020, 11, 100690
Xu et al. Adv. Energy Mater. 2021, 2003908



(a) Autoencoder for microstructure feature extraction and design optimization



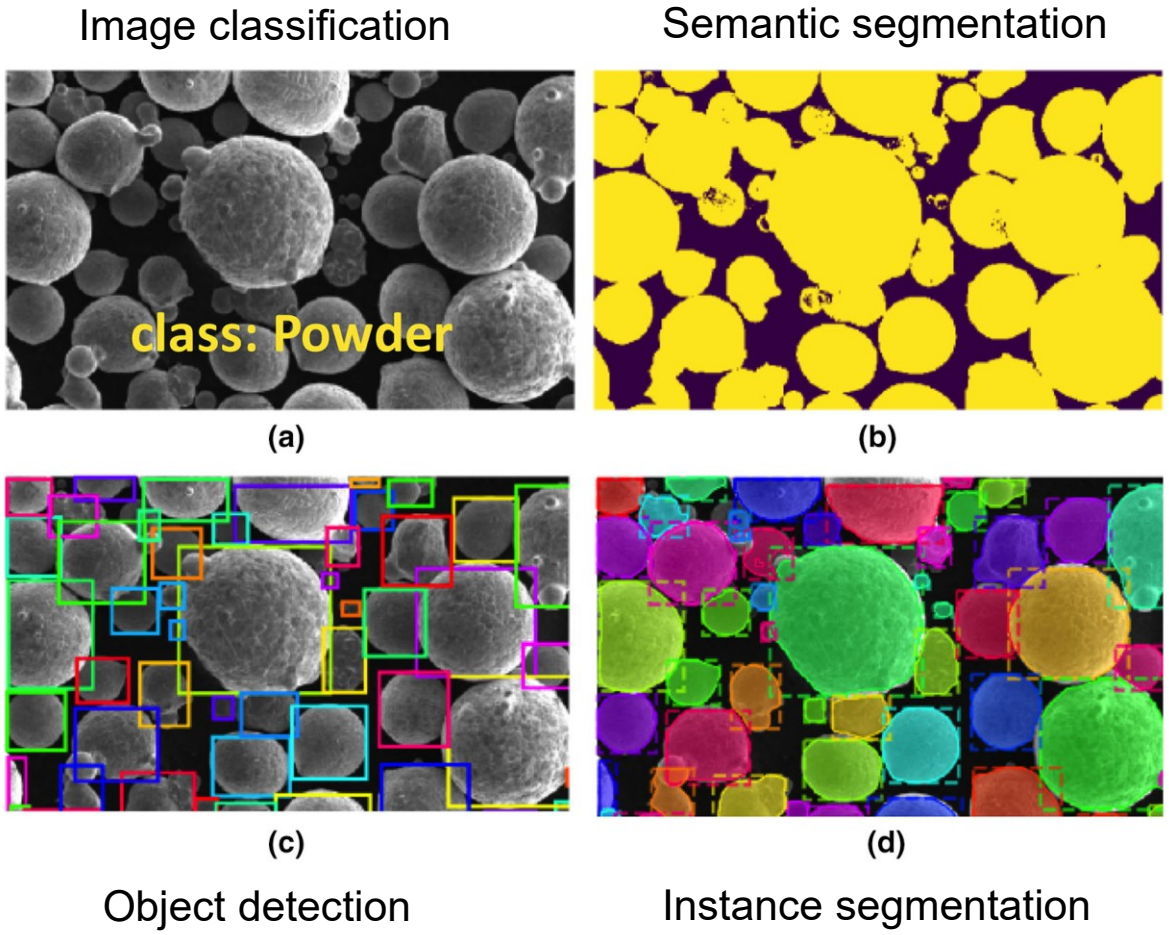
(b) Generative adversarial network-based microstructure design workflow

III. Features represented by ML

A hierarchy or taxonomy of microstructural analysis tasks:

- Image classification: identifying the content of an image
- Semantic segmentation: associating each pixel in an image with a constituent of the image
- Object detection: locating individual objects in an image
- Instance segmentation: assigninng pixel to individual objects

Holm et al. Metall. Mat. Trans. A, 51A, 2020



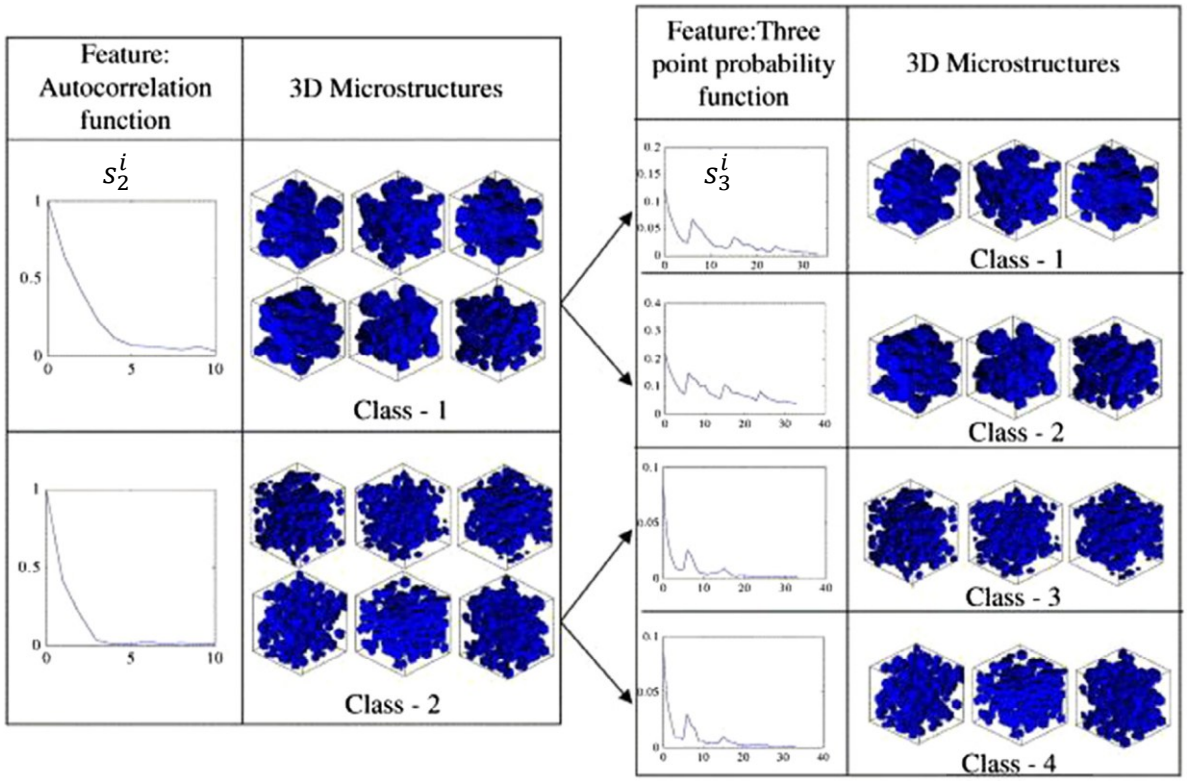
III. Features represented by ML
III.2 Instance-based learning

Instance-based learning performs instance similarity search in databases of microstructures.

1. first creating a database that contains representative sample instances
2. then defining the feature space (e.g. the statistical functions) and similarity metrics applicable to all the samples in the database (e.g. hierarchical support vector machine classification model)
3. finally conducting either bruteforce or algorithmic search to find the most similar instance in the database upon query

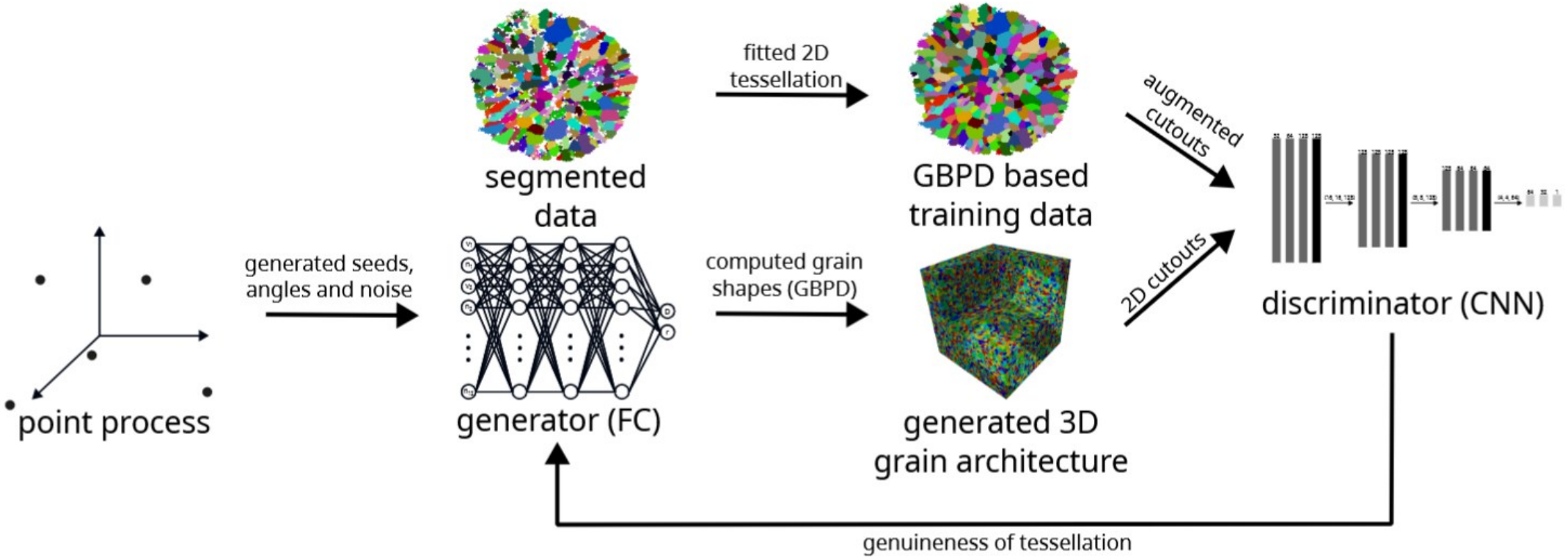
Comments:

- data sufficiency: not only the database should be large enough, but also the feasible space must be well covered
- search efficiency: the size of database increases exponentially and so it is essential to develop a rapid searching mechanism that can efficiently locate the most similar instance in the database upon a query



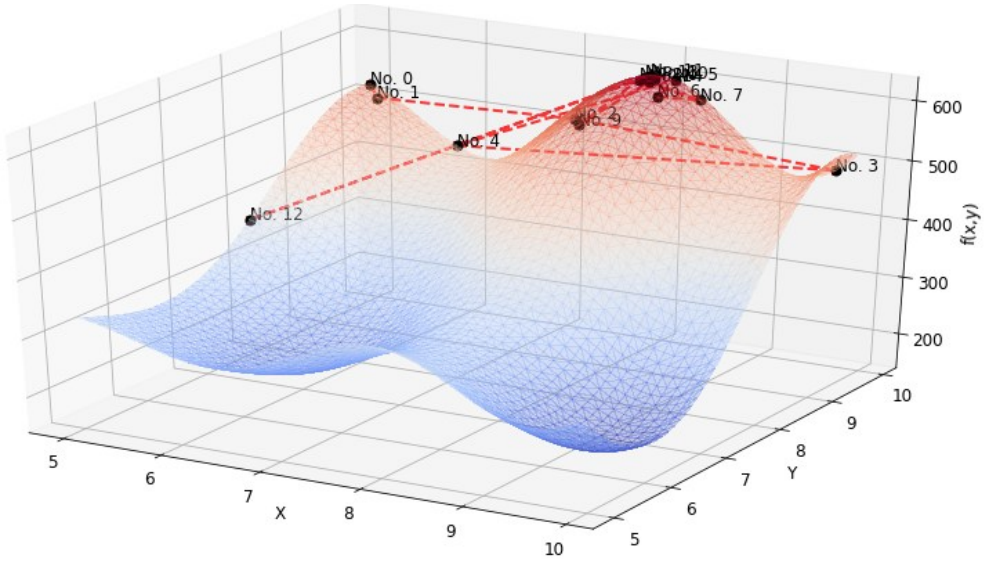
Sundararaghavan V, Zabaras N. Classification and reconstruction of 3D microstructures using support vector machines. Comput. Mater. Sci. 2005;32(2):223–39.

Generative Adversarial Networks (GAN) training procedure for 3D microstructure generation from 2D images

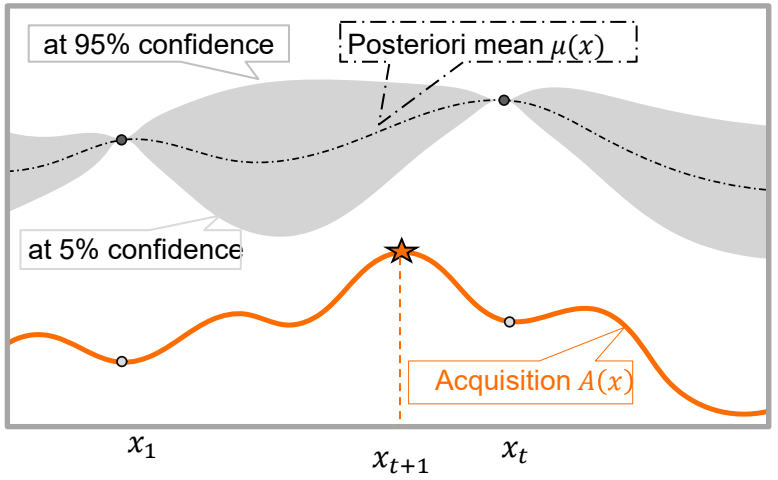


Fuchs et al. Commun. Mater. 2025. doi: 10.1038/s43246-024-00728-5

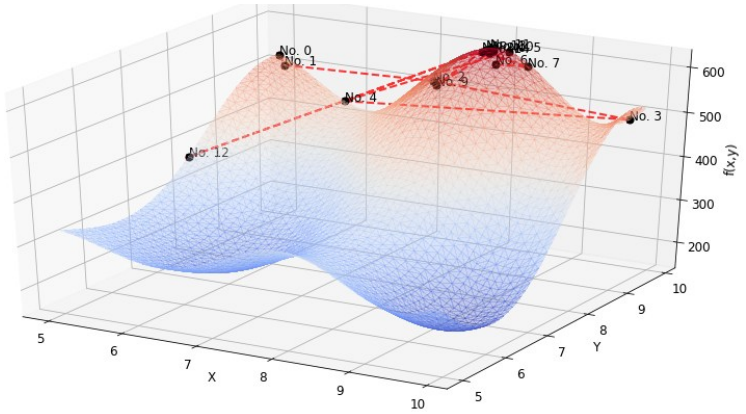
Bayesian Optimization with Gaussian Process



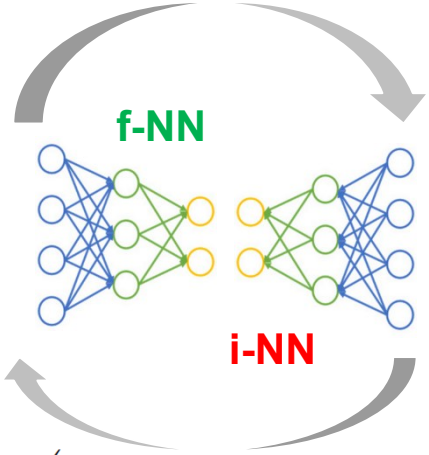
Indirect and Direct Inverse Design



Bayesian Optimization
(indirect method)
suitable for earlier stage with limited datasets



Average-based DOE or empirical methods are not suitable for such high-dimensional optimization/design.



$$\mathcal{L} = \min_{\nu} \frac{1}{n} \sum^{n} \left(\lambda \underbrace{\|\mathcal{I}_\nu[\boldsymbol{Y}] - \boldsymbol{X}\|^2}_{\mathrm{SE}_i} + \underbrace{\|\mathcal{F}_\omega[\mathcal{I}_\nu[\boldsymbol{Y}]] - \boldsymbol{Y}\|^2}_{\mathrm{SE}_f} \right)$$

Gradient-based Method
(direct method)
suitable for later stage with large datasets

Peng, Fathidoost, Lin, Yang, **Xu**, MRS Bulletin 2025

Gaussian Process

Model: **Gaussian Process Regression** (probability distribution of the to-be-determined function)

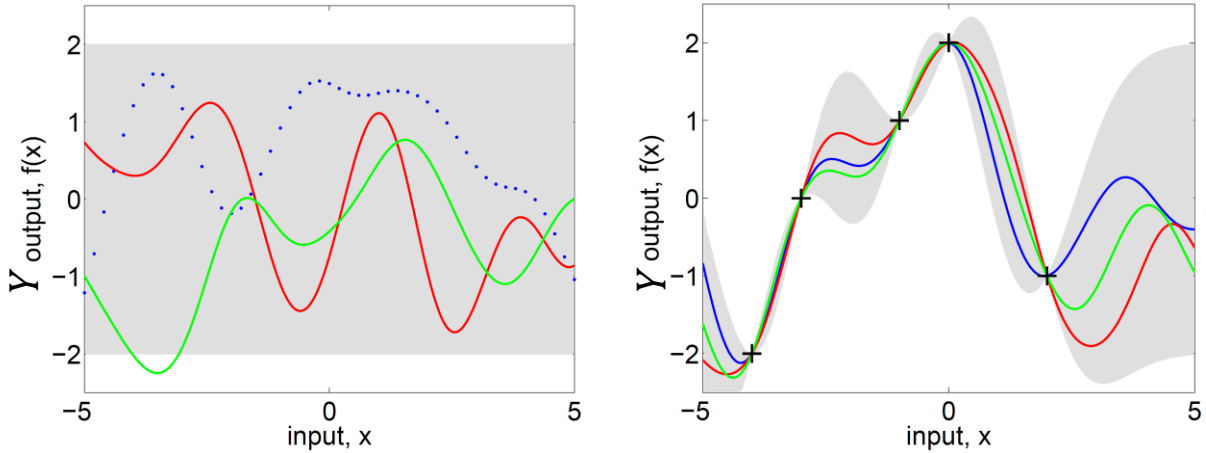
Objective: **maximum a posteriori** (maximum likelihood penalized)

Iterative Graussian Process:

$$P(y_{t+1}|D_{1:t}, x_{t+1}) = \aleph(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1}) + \sigma_{noise}^2)$$

$$\mu_t(x_{t+1}) = \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1} y_{1:t} \quad \sigma_t^2(x_{t+1}) = k(x_{t+1}, x_{t+1}) - \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1}$$

Squared-exponential covariance $\mathbf{K}(x_p, x_q) = \sigma_f^2 \exp\left[-\frac{1}{2l^2}(x_p - x_q)^2\right]$



Distribution regions of the output function: Initial/prior (gray on left) and Posteriori (gray on right) after considering three data points

No model parameter

Hyperparameters in iterative process:

$\mu_0(x), \sigma_0(x)$: initial mean/variance functions
σ_{noise}^2 : noise variance
σ_f^2 : signal variance
l : length-scae

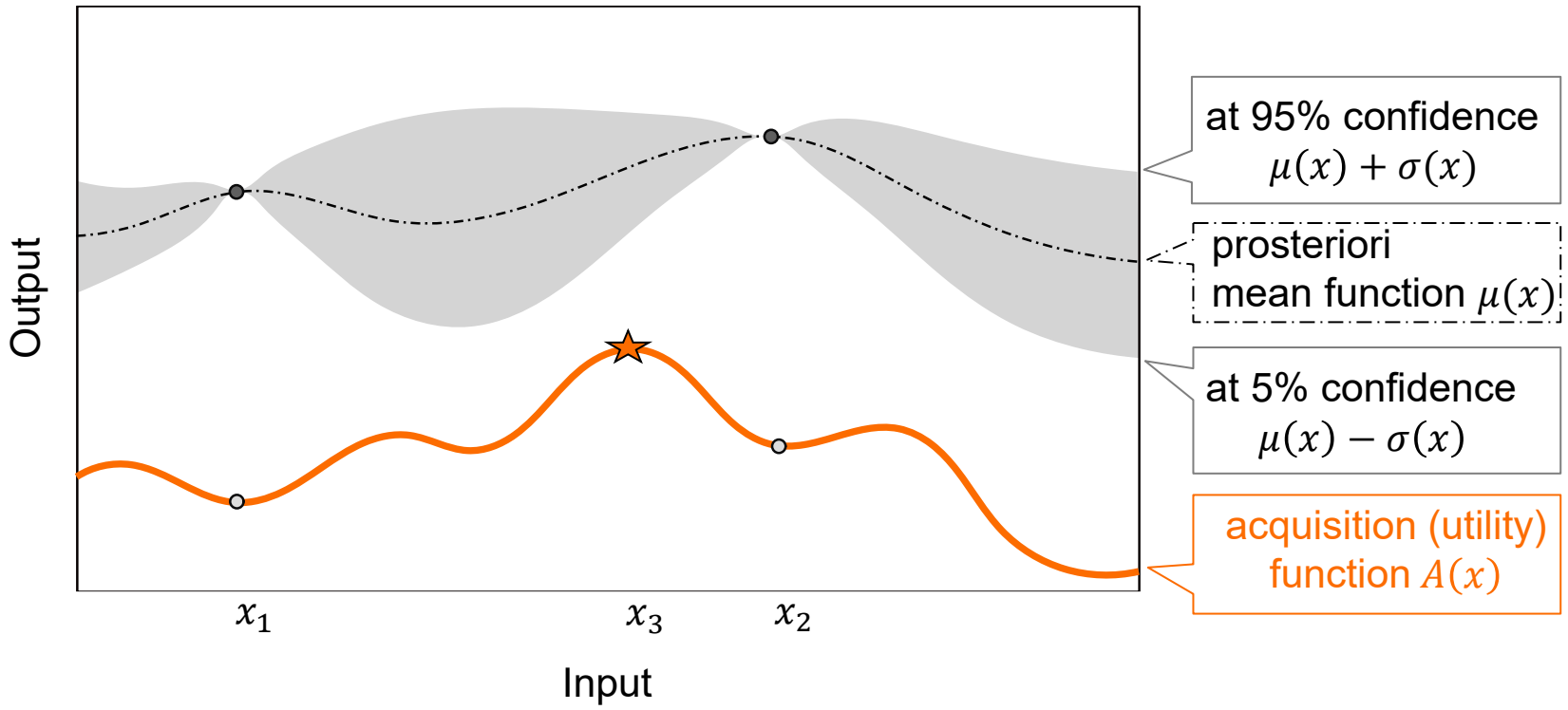
Bayesian Optimization (in W9), used for searching best property, optimizing hyperparameters (also for NN), inverse design, is closely based on Gaussian Process.

Thereby the so-called acquisition or utility function should be defined, e.g.

$$u(x) = \mu(x) + \alpha\,\sigma(x)$$

Maximizing $u(x)$ to find the optimum.

Bayesian Optimization



$\mu(x)$ and $\sigma(x)$ are prosteriori mean function and variance after Gaussian Process after considering the data till now $D_{1:2} = \{(x_1, y_1), (x_2, y_2)\}$

Acquisition function $A(x)$ can be readily calculated from the current $\mu(x), \sigma(x)$.

Find the next input x_3 which maximizes the current $A(x)$: $x_3 = \text{argmax}_x A(x \mid D_{1:2})$

- Acquisition function $A(x)$ can take different forms depending on the objective and the approach choice, as shown later.
- In general, it imposes a kind of trade-off between the exploitation (high mean) and the exploration (high variance).

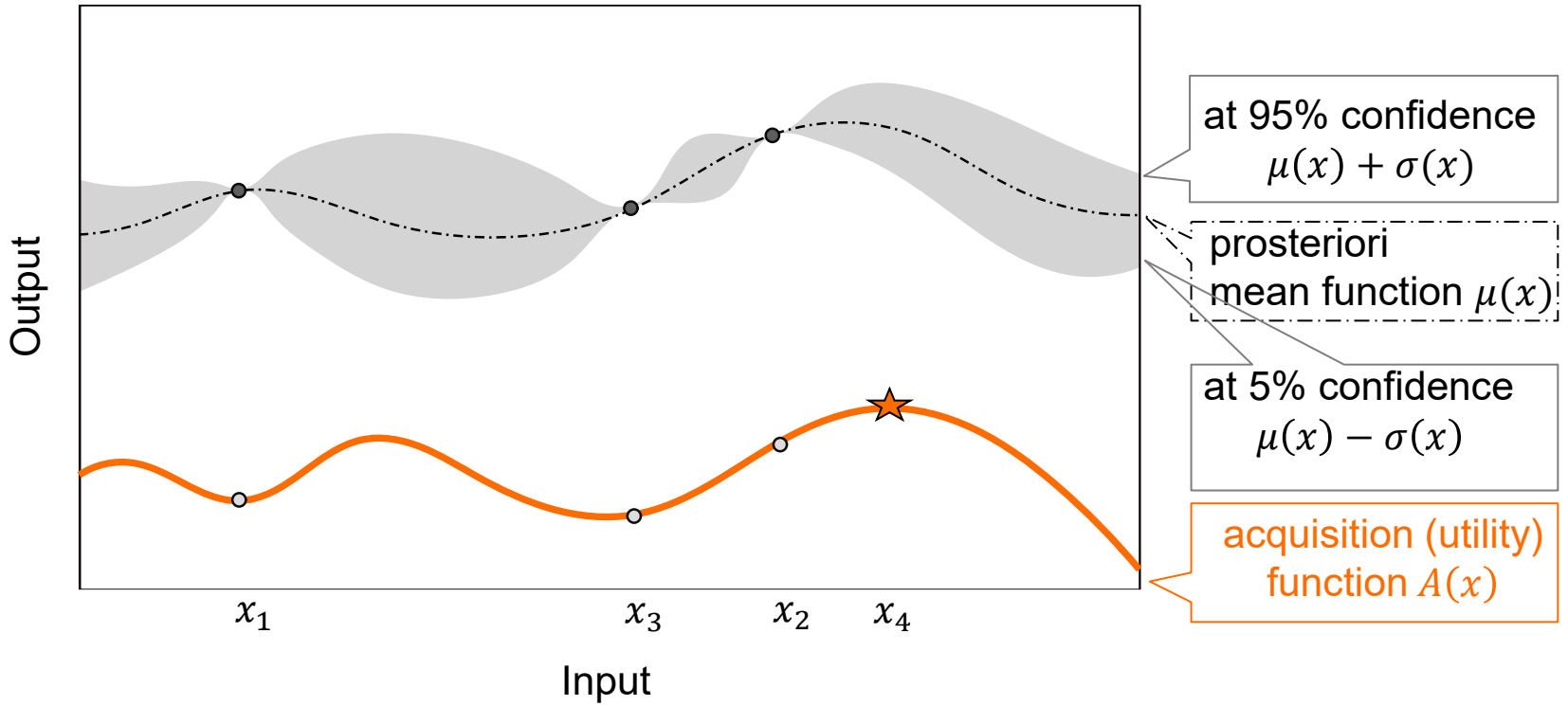
Bayesian Optimization

Gaussian Process

$$P(y_{t+1}|D_{1:t}, x_{t+1}) = \aleph(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1}) + \sigma_{noise}^2)$$

$$\mu_t(x_{t+1}) = \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1} y_{1:t}$$

$$\sigma_t^2(x_{t+1}) = k(x_{t+1}, x_{t+1}) - \boldsymbol{k}^T\left[\mathbf{K} + \sigma_{noise}^2\mathbf{I}\right]^{-1}$$



$\mu(x)$ and $\sigma(x)$ are updated via Gaussian Process after data augment

$$D_{1:3} = \{D_{1:2}, (x_3, y_3)\}$$

Acquisition function $A(x)$ is also updated through updated $\mu(x), \sigma(x)$.

The procedure is then repeated. Find the next input x_4 which maximizes the current $A(x)$:

$$x_4 = \text{argmax}_x A(x \mid D_{1:3})$$

In general

$$x_{t+1} = \text{argmax}_x A(x \mid D_{1:t})$$

- While Gaussian Process provides a regression model from given data, Bayesian Optimization guide the efficient data acquisition towards the optimal output, without fully presenting the regression model.

$A(x)$: Probability of Improvement

$A(x)$ is assumed to the Probability of Improvement, which is defined, for the given mean $\mu(x)$ and variance $\sigma(x)$ at x as the probability of its output y can be larger than the maximal output which has been seen currently. More exactly,

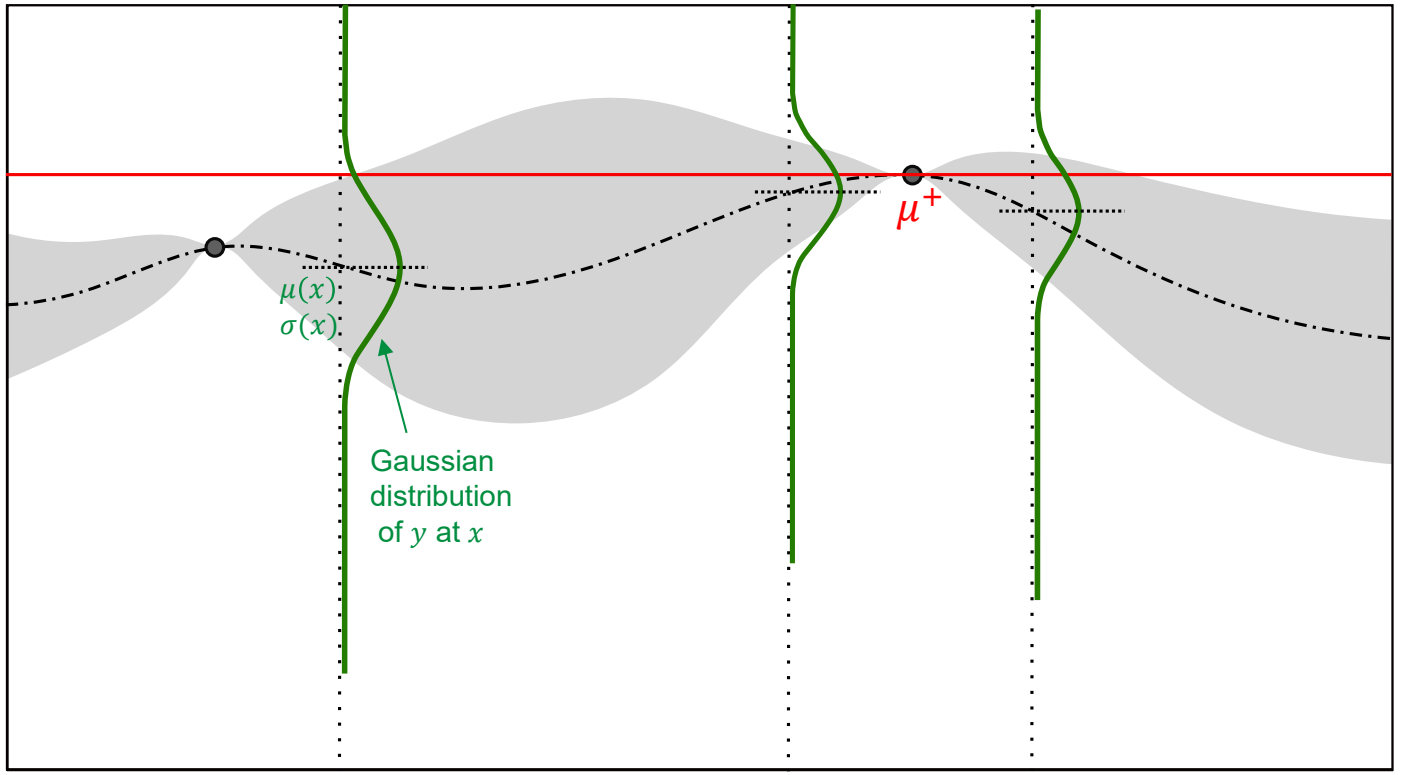
$$PI(x) = P(y \geq \mu^+ + \xi) = \Phi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right)$$

μ^+ : maximal value of y in the current data $D_{1:t}$

ξ : a small numerical number to void rigid issue

$\Phi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$:

the cumulative distribution function of the Gaussian distribution at x .



$A(x)$: Probability of Improvement

$A(x)$ is assumed to the Probability of Improvement, which is defined, for the given mean $\mu(x)$ and variance $\sigma(x)$ at x as the probability of its output y can be larger than the maximal output which has been seen currently. More exactly,

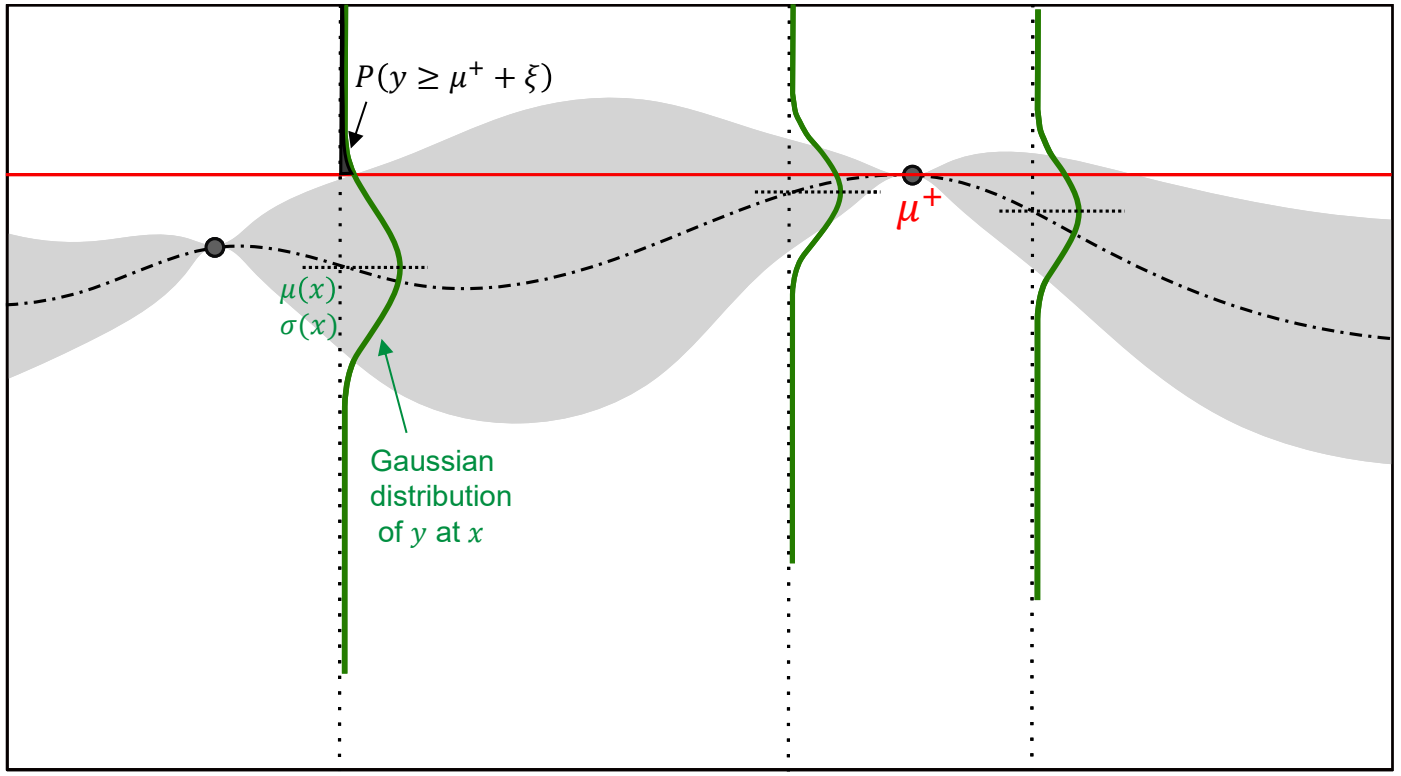
$$PI(x) = P(y \geq \mu^+ + \xi) = \Phi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right)$$

μ^+ : maximal value of y in the current data $D_{1:t}$

ξ : a small numerical number to void rigid issue

$\Phi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$:

the cumulative distribution function of the Gaussian distribution at x .



$A(x)$: Probability of Improvement

$A(x)$ is assumed to the Probability of Improvement, which is defined, for the given mean $\mu(x)$ and variance $\sigma(x)$ at x as the probability of its output y can be larger than the maximal output which has been seen currently. More exactly,

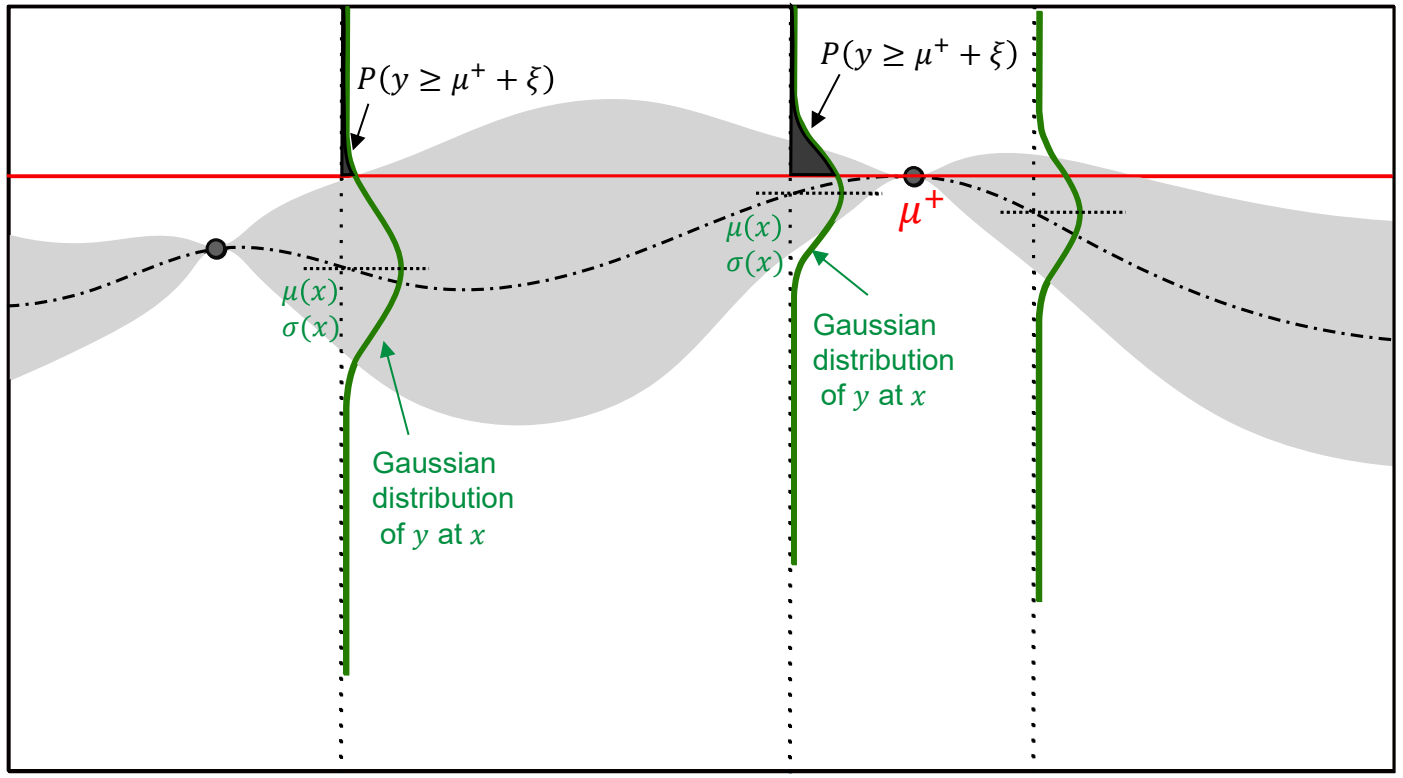
$$PI(x) = P(y \geq \mu^+ + \xi) = \Phi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right)$$

μ^+ : maximal value of y in the current data $D_{1:t}$

ξ : a small numerical number to void rigid issue

$\Phi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$:

the cumulative distribution function of the Gaussian distribution at x .



$A(x)$: Probability of Improvement

$A(x)$ is assumed to the Probability of Improvement, which is defined, for the given mean $\mu(x)$ and variance $\sigma(x)$ at x as the probability of its output y can be larger than the maximal output which has been seen currently. More exactly,

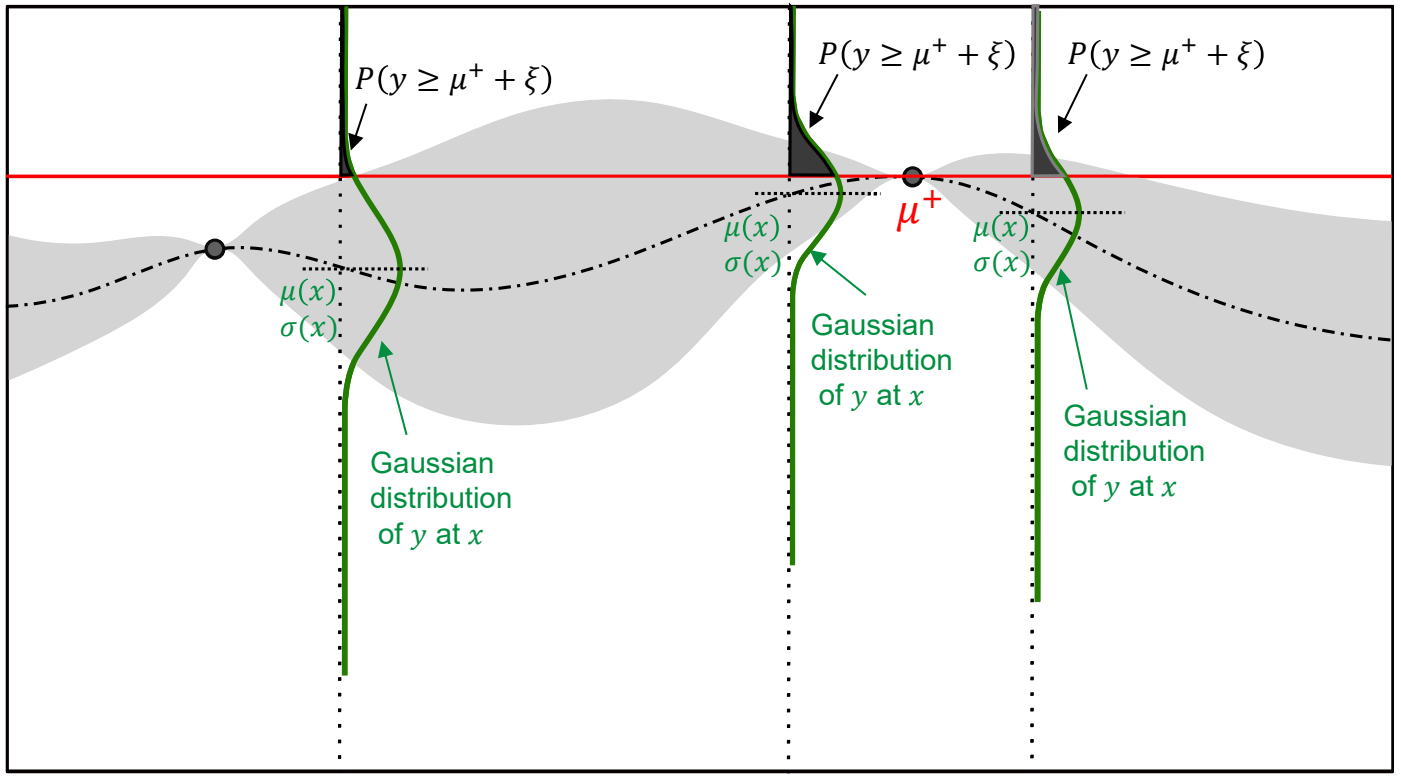
$$PI(x) = P(y \geq \mu^+ + \xi) = \Phi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right)$$

μ^+ : maximal value of y in the current data $D_{1:t}$

ξ : a small numerical number to void rigid issue

$\Phi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$:

the cumulative distribution function of the Gaussian distribution at x .



Find the next input x_{t+1} which maximizes the current $P(y \geq \mu^+ + \xi)$:

$$x_{t+1} = \text{argmax}_x P(y \geq \mu^+ + \xi \mid D_{1:t})$$

$A(x)$: Expected Improvement

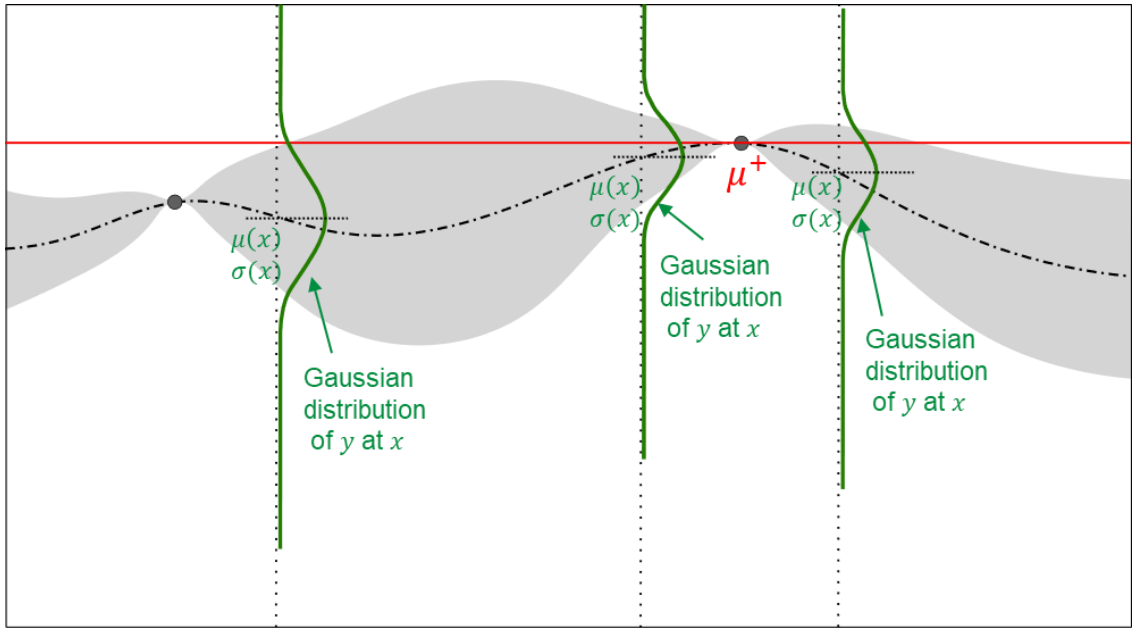
$A(x)$ is assumed to the Expected Improvement which is defined for given the mean $\mu(x)$ and the variance $\sigma(x)$ at x by

$$EI(x) = (\mu(x) - \mu^+ - \xi)\Phi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right) + \sigma(x)\psi\left(\frac{\mu(x) - \mu^+ - \xi}{\sigma(x)}\right)$$

μ^+ : maximal value of y in the current data $D_{1:t}$

ξ : a small numerical number to void rigid issue

$\Phi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$, $\psi\left(\frac{\mu(x)-\mu^+-\xi}{\sigma(x)}\right)$:
the cumulative distribution function and density function of the Gaussian distribution at x, respectively.



Similar idea: For given

$$\begin{cases} P(a = \text{healthy}) = 0.9 \\ P(a = \text{cancer}) = 0.1 \end{cases}$$

$u(a,b)$	a = no treatment	b = treatment
b = healthy	0	-40
b = cancer	-300	-20

Action is maximizíng the expected Utility, or minimizes the expected cost:

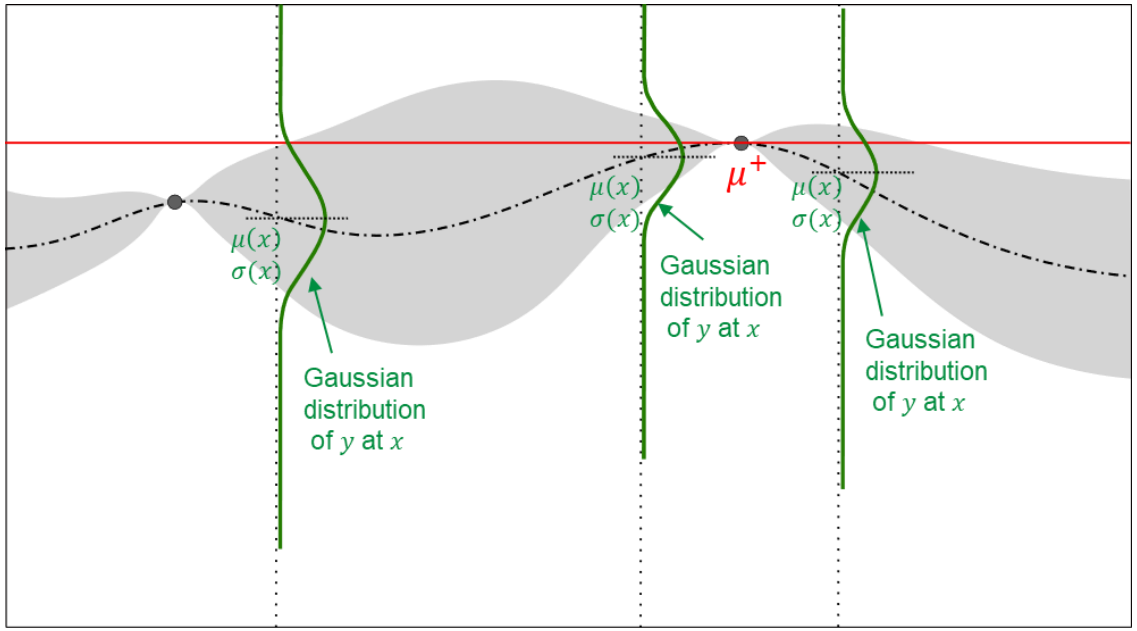
$$EU(a) = \sum_b u(a,b)P(a)$$

$A(x)$: Confidence Bound

For given the mean $\mu(x)$ and the variance $\sigma(x)$ at x, the upper confidence bound is defined by

$$UCB(x) = \mu(x) + \beta\sigma(x)$$

β : a parameter controlling the degree of exploration



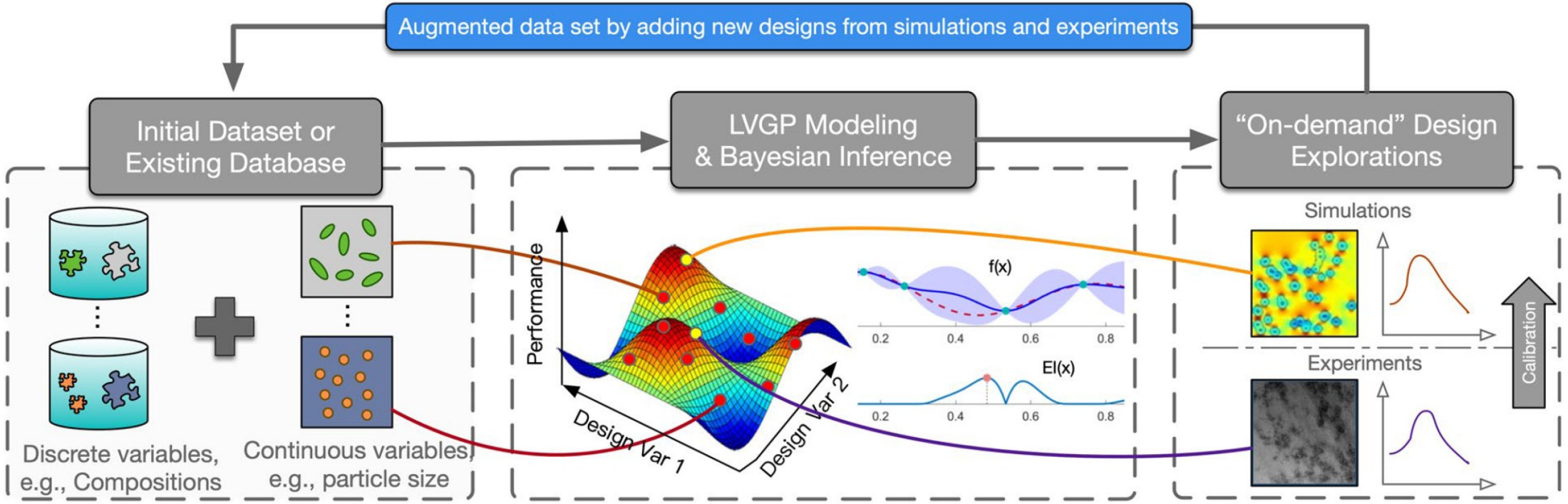
Comments

- It is recommended to use muliple acqusition functions in application case of optimization.
- Instead of Gaussian Process, the Bayesian optimization scheme can be combined with general surrogate ML models (e.g. random forest, automatic relevance detection).
- Gaussian Process regression is mostly for the cases with limited data.

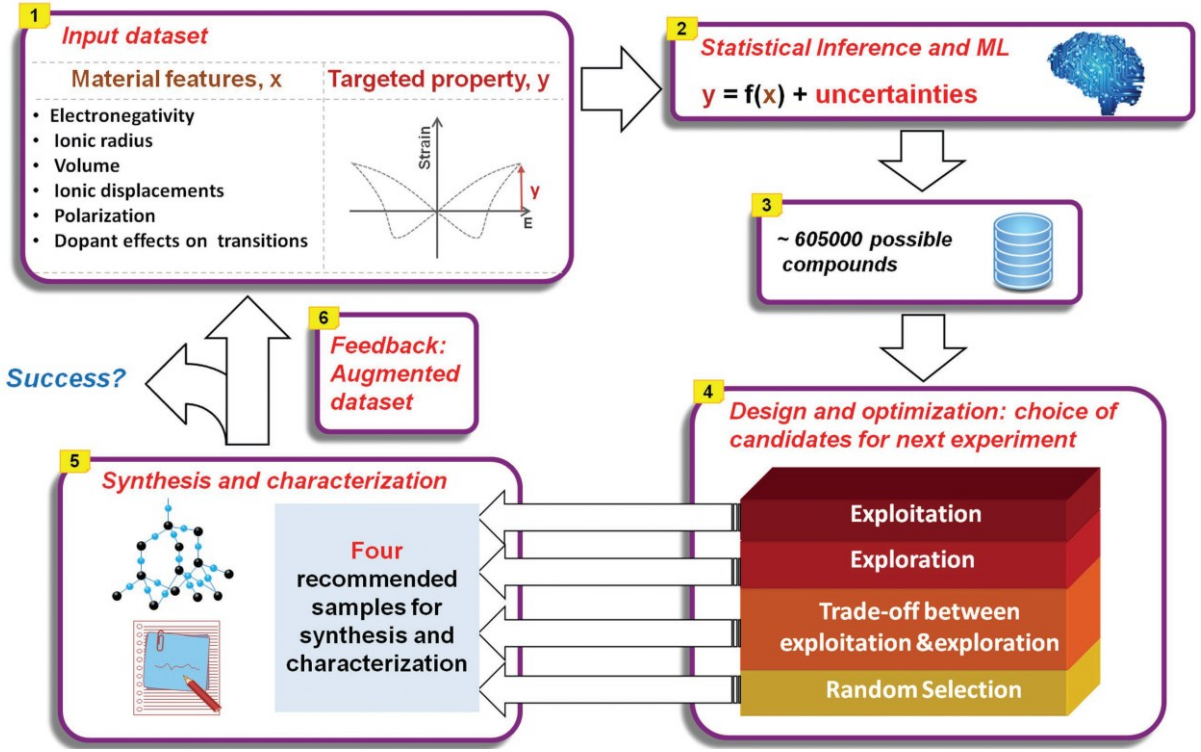
- Bayesian optimization guides the choice of experiments during materials design and discovery to find good material designs in as few experiments as possible.
- It is very useful for optimizing given time- or cost- expensive experiments or simulations.
- It can be also used to guide hyperparameter choice, although not always effective.
- It is important to combine it with additional experimentation or simulation for deeper understanding of the optimization results.

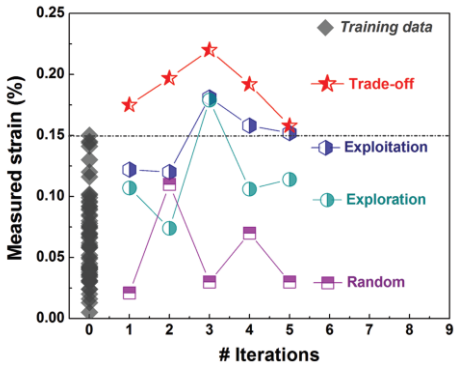
Application in materials science

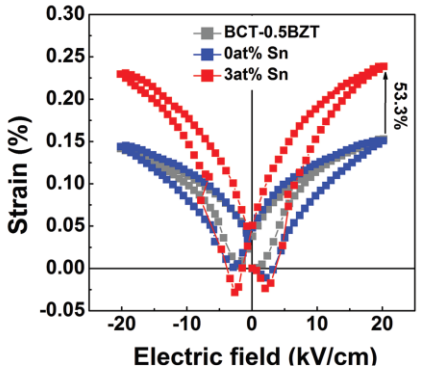
- Zhang, Y., Apley, D.W. & Chen, W. Bayesian Optimization for Materials Design with Mixed Quantitative and Qualitative Variables. *Sci Rep* **10,** 4924 (2020). https://doi.org/10.1038/s41598-020-60652-9

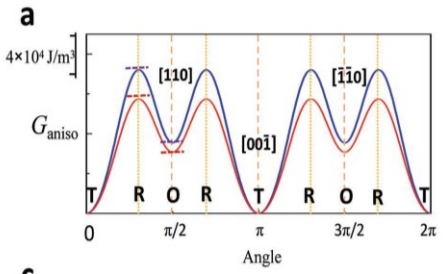


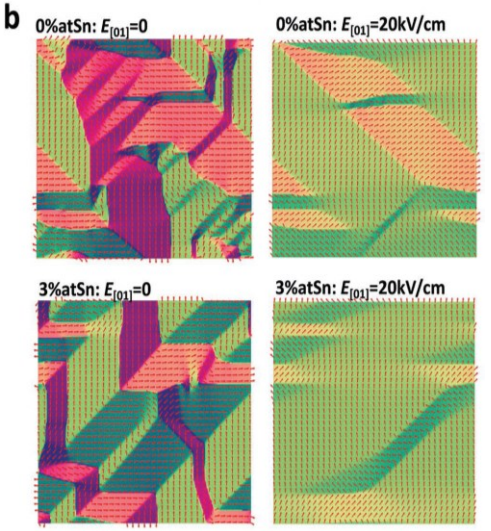
Application in materials science

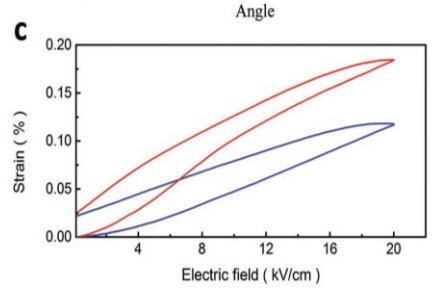












- Yuan et al. Accelerated Discovery of Large Electrostrains in BaTiO3-Based Piezoelectrics Using Active Learning. Adv. Mater. 30, 2018. 1702884.